

ĐỒ ÁN TỐT NGHIỆP

**ỨNG DỤNG ROBOT DI ĐỘNG OMNI TRONG
HỆ THỐNG PHÂN PHỐI THUỐC
TỰ ĐỘNG TẠI BỆNH VIỆN**

Ngành: Robot và trí tuệ nhân tạo

GIẢNG VIÊN HƯỚNG DẪN : TS PHẠM QUỐC THIỆN

Sinh viên thực hiện	MSSV	Lớp
Nguyễn Đình Khiêm	2186300559	21DRTA1
Trần Đăng Khoa	2186300054	21DRTA1
Phạm Văn Huy	2186300537	21DRTA1

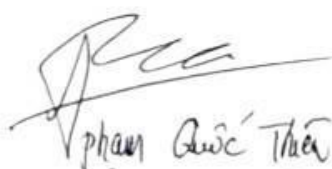
Tp.HCM, tháng 08, năm 2025

VIỆN KỸ THUẬT HUTECH

**PHIẾU ĐĂNG KÝ
ĐỀ TÀI ĐỒ ÁN/KHÓA LUẬN TỐT NGHIỆP**Hệ: CQ (CQ, LT, VB2, VLVH)1. **Họ và tên sinh viên/ nhóm sinh viên đăng ký đề tài (sĩ số trong nhóm...):**(1) Nguyễn Đình Khâm MSSV: 2186300559 .. Lớp: 21DRTA1Điện thoại: 0398089733 Email: duykhamb3@gmail.com ..(2) Phạm Văn Huy MSSV: 2186300537 .. Lớp: 21DRTA1Điện thoại: 0768611506 Email: huy156003@gmail.com ..(3) Trần Đăng Khoa MSSV: 2186300054 .. Lớp: 21DRTA1Điện thoại: 0848889098 Email: TdKhoa18062003@gmail.comNgành : Robot & Trí tuệ nhân tạo ..2. **Mã đề tài :** 013. **Tên đề tài đăng ký :** Ứng dụng robot di động Omni trong hệ thống
phân phối thuốc tự động tại bệnh viện

Sinh viên đã hiểu rõ yêu cầu của đề tài và cam kết thực hiện đề tài theo tiến độ và hoàn thành đúng thời hạn.

Giảng viên hướng dẫn
(Ký và ghi rõ họ tên)


Phạm Quốc Thái

TP. HCM, ngày 1. tháng 3. năm 2025

Sinh viên đăng ký
(Ký và ghi rõ họ tên các thành viên)


Phạm Văn Huy
Trần Đăng Khoa
Nguyễn Đình Khâm

Viện Kỹ thuật Hutech

**PHIẾU GIAO NHIỆM VỤ
THỰC HIỆN ĐỒ ÁN TỐT NGHIỆP**

(Mỗi sinh viên một phiếu, GVHD ghi rõ tên đề tài và nhiệm vụ của từng sinh viên. GVHD chuyển cho SV để nộp về VP Viện.)

1. Sinh viên thực hiện đề tài

Họ tên : Nguyễn Đình Khiêm MSSV : 2186300559 Lớp: 21DRTA1
Điện thoại : 0398089733 Email : dinhkhiemk3@gmail.com
Ngành : Robot và trí tuệ nhân tạo

2. Mã đề tài : 01**3. Tên đề tài:**

Ứng dụng robot di động Omni trong hệ thống phân phối thuốc tự động tại bệnh viện

4. Nhiệm vụ thực hiện đề tài:

- Tổng quan
 - Nghiên cứu các phương pháp điều hướng robot di động.
 - Tổng quan về các thuật toán lập kế hoạch đường đi (Dijkstra, A*, RRT...)
- Xây dựng thuật toán lập kế hoạch quỹ đạo
 - Xác định các điểm đích trên bản đồ bệnh viện (các phòng cấp thuốc).
 - Áp dụng thuật toán A*, Dijkstra hoặc RRT để tìm đường đi tối ưu.
 - Xử lý các tình huống tránh vật cản động (sử dụng Lidar, camera, cảm biến siêu âm)
- Điều khiển robot đi theo quỹ đạo mong muốn
 - Xây dựng bộ điều khiển robot sử dụng kinematic model của robot Omni.
 - Điều chỉnh tốc độ, góc quay của robot dựa trên dữ liệu từ quỹ đạo.
 - Xử lý sai số khi robot di chuyển và tinh chỉnh điều khiển.
- Thực nghiệm & kiểm tra.
 - Kiểm tra robot có thể đi theo đường đi mong muốn với độ chính xác cao.
 - Tinh chỉnh thuật toán để đảm bảo robot hoạt động ổn định và mượt mà.

TP. HCM, ngày 3. tháng 6 năm 2025

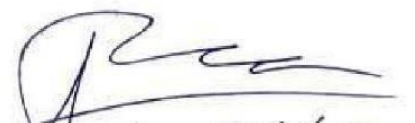
Sinh viên thực hiện

(Ký và ghi rõ họ tên)

Khiêm
Nguyễn Đình Khiêm

Giảng viên hướng dẫn

(Ký và ghi rõ họ tên)


Phạm Quốc Thuận

Viện Kỹ thuật Hutech**PHIẾU GIAO NHIỆM VỤ
THỰC HIỆN ĐỒ ÁN TỐT NGHIỆP**

(Mỗi sinh viên một phiếu, GVHD ghi rõ tên đề tài và nhiệm vụ của từng sinh viên. GVHD chuyển cho SV để nộp về VP Viện.)

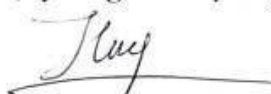
1. Sinh viên thực hiện đề tài

Họ tên : Phạm Văn Huy MSSV : 2186300537 Lớp: 21DRTA1
Điện thoại : 0768611506 Email : huy156003@gmail.com
Ngành : Robot và trí tuệ nhân tạo

2. Mã đề tài : 01**3. Tên đề tài: Ứng dụng robot di động Omni trong hệ thống phân phối thuốc tự động tại bệnh viện****4. Nhiệm vụ thực hiện đề tài:**

- Tổng quan
 - o Nghiên cứu các phương pháp định vị robot trong môi trường trong nhà.
 - o Tổng quan về công nghệ LiDAR trong định vị và xây dựng bản đồ.
 - o Tìm hiểu về thuật toán SLAM (Simultaneous localization and Mapping).
- Xây dựng bản đồ bệnh viện bằng LiDAR
 - o Lựa chọn cảm biến lidar phù hợp (RPLiDAR, Hokuyo, Velodyne...)
 - o Cài đặt cấu hình cảm biến LiDAR.
 - o Xây dựng bản đồ bằng thuật toán SLAM (GMapping, Hector SLAM, Cartographer...)
 - o Xử lý dữ liệu LiDAR để xác định vị trí robot theo thời gian thực.
- Định vị và nhận diện vị trí các phòng cấp thuốc
 - o Xác định vị trí của từng phòng cấp thuốc và lối đi trong bản đồ.
 - o Tích hợp thêm cảm biến hỗ trợ như IMU, camera để tăng độ chính xác.
 - o Xây dựng thuật toán cập nhật vị trí của robot dựa trên bản đồ.
 - o Hiệu chỉnh sai số, tối ưu thuật toán để đảm bảo định vị chính xác trong môi trường bệnh viện.
- Thực nghiệm & kiểm tra độ chính xác.
 - o Kiểm tra khả năng xác định vị trí của robot trong các môi trường khác nhau.
 - o Tinh chỉnh bản đồ và thuật toán để giảm lỗi định vị.

TP. HCM, ngày ... tháng ... năm 20...

Sinh viên thực hiện*(Ký và ghi rõ họ tên)*
Phạm Văn Huy**Giảng viên hướng dẫn***(Ký và ghi rõ họ tên)*
Phạm Quốc Thuận

Viện Kỹ thuật Hutech

PHIẾU GIAO NHIỆM VỤ THỰC HIỆN ĐỒ ÁN TỐT NGHIỆP

(Mỗi sinh viên một phiếu, GVHD ghi rõ tên đề tài và nhiệm vụ của từng sinh viên. GVHD chuyển cho SV để nộp về VP Viện.)

1. Sinh viên thực hiện đề tài

Họ tên : Trần Đăng Khoa MSSV : 2186300054 Lớp: 21DRTA1
Điện thoại : 0848889098 Email : tdkhoa18062003@gmail.com
Ngành : Robot và trí tuệ nhân tạo

2. Mã đề tài : 01**3. Tên đề tài:**

Ứng dụng robot di động Omni trong hệ thống phân phối thuốc tự động tại bệnh viện

4. Nhiệm vụ thực hiện đề tài:

- Tổng quan
 - Nghiên cứu hệ thống điều khiển robot từ xa.
 - Tổng quan về các giao thức truyền thông (ROS, MQTT, WebSocket...).
- Xây dựng hệ thống điều khiển & giao tiếp
 - Lựa chọn nền tảng điều khiển (ROS trên Raspberry Pi, Jetson Nano...).
 - Xây dựng hệ thống giao tiếp giữa robot và máy chủ trung tâm.
 - Thiết kế giao diện quản lý (ứng dụng trên máy tính, điện thoại, hoặc web).
 - Tích hợp hệ thống quản lý danh sách đơn thuốc và lộ trình giao thuốc.
- Kiểm tra và tối ưu hệ thống
 - Đánh giá độ trễ và độ tin cậy của hệ thống giao tiếp.
 - Kiểm tra khả năng quản lý nhiều robot cùng lúc (nếu có).
 - Tinh chỉnh phần mềm để đảm bảo hệ thống hoạt động ổn định.

TP. HCM, ngày ... tháng ... năm 20...

Sinh viên thực hiện
(Ký và ghi rõ họ tên)

Giảng viên hướng dẫn
(Ký và ghi rõ họ tên)


Trần Đăng Khoa
Phạm Quốc Thiên

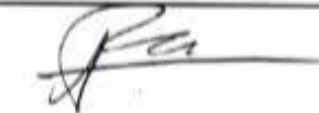
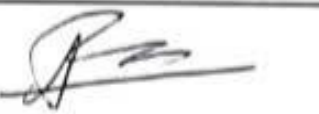
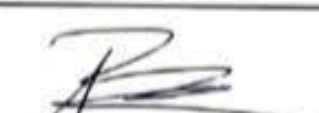
VIỆN KỸ THUẬT HUTECH
**PHIẾU THEO DÕI TIẾN ĐỘ
THỰC HIỆN ĐỒ ÁN/ KHÓA LUẬN TỐT NGHIỆP**

(Do giảng viên hướng dẫn ghi và giao cho sinh viên nộp chung với ĐA/KLTN sau khi hoàn tất đề tài)

1. Tên đề tài: Ứng dụng robot di động Omni trong hệ thống phân phối thuốc tự động tại bệnh viện
2. Mã đề tài: 1
3. Giảng viên hướng dẫn: TS. Phạm Quốc Thiện
4. Sinh viên/ nhóm sinh viên thực hiện đề tài (sĩ số trong nhóm 3):

(1) Nguyễn Đình Khiêm	MSSV: 2186300559	Lớp: 21DRTA1
Điện thoại: 0398089733	Email: dinhkhiemk3@gmail.com	
(2) Trần Đăng Khoa	MSSV: 2186300054	Lớp: 21DRTA1
Điện thoại: 0848889098	Email: tdkhoa18062003@gmail.com	
(3) Phạm Văn Huy	MSSV: 2186300537	Lớp: 21DRTA1
Điện thoại: 0768611506	Email: huy156003@gmail.com	

Ngành : Robot & Trí tuệ nhận tạo
 Chuyên ngành : Robot thông minh

Tuần lễ	Ngày	Nội dung	Nhận xét của GVHD (Ký tên)
1	24/3/2025 – 30/3/2025	Tìm hiểu các bài báo, bài nghiên cứu trong nước và ngoài nước. cứu liên quan đến đề tài.	
2	31/3/2025 – 6/4/2025	Nghiên cứu hệ thống robot điều khiển từ xa và lựa chọn dạng robot phù hợp.	
3	7/4/2025 – 13/4/2025	Thiết kế bản vẽ kỹ thuật robot (các thành phần cơ khí, nguồn, điều khiển, chấp hành ...).	
4	14/4/2025 – 20/4/2025	Gia công và lắp ráp phần cứng robot.	
5	21/4/2025 – 27/4/2025	Xây dựng bộ điều khiển robot sử dụng kinematic model của robot omni.	
6	28/4/2025 – 4/5/2025	Tìm hiểu và sử dụng về hệ điều hành ROS các thư viện hỗ trợ quản lý và điều khiển robot.	



Tuần lễ	Ngày	Nội dung	Nhận xét của GVHD (Ký tên)
7	5/5/2025 – 11/5/2025	Nghiên cứu phương pháp định vị robot trong môi trường trong nhà.	
8 - Kiểm tra ngày: 12/5/2025		Đánh giá công việc hoàn thành:60.....% Được tiếp tục: ☒ Không tiếp tục: ☐	
9	19/5/2025 – 25/5/2025	Tìm hiểu về thuật toán SLAM và thư viện hỗ trợ lidar trên ROS.	
10	26/5/2025 – 1/6/2025	Cấu hình cảm biến Rplidar và xây dựng bản đồ bệnh viện.	
11	2/6/2025 – 8/6/2025	Đọc giá trị encoder bánh xe và cảm biến IMU để tính toán odometry để định vị robot trên bản đồ.	
12	9/6/2025 – 15/6/2025	Định vị và nhận diện các vị trí làm việc cấp thuốc và giao thuốc.	
13	16/6/2025 – 22/6/2025	Phát triển ứng dụng di động để điều khiển robot từ xa và giám sát hoạt động robot trên bản đồ.	
14	23/6/2025 – 29/6/2025	Thực nghiệm và kiểm tra độ chính xác.	
15	30/6/2025 – 6/7/2025	Viết và hoàn thiện bài báo cáo	

Giảng viên hướng dẫn phụ (nếu có)
(Ký và ghi rõ họ tên)

TP. HCM, ngày 7 tháng 7 năm 2025.
Giảng viên hướng dẫn chính
(Ký và ghi rõ họ tên)

Phạm Quốc Thiện

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ TP. HCM
VIỆN KỸ THUẬT HUTECH

PHIẾU BÁO CÁO
KẾT QUẢ CHỈNH SỬA ĐỒ ÁN TỐT NGHIỆP SAU KHI BẢO VỆ
Học kỳ .2... năm học 2024 - 2025

Họ và tên sinh viên:

(1) Nguyễn Đình Khiêm	MSSV: 2186300559	Lớp: 21DRTA1
(2) Phạm Văn Huy	MSSV: 2186300537	Lớp: 21DRTA1
(3) Trần Đăng Khoa	MSSV: 2186300054	Lớp: 21DRTA1

Mã đề tài: 01

Tên đề tài: Ứng dụng robot di động Omni trong hệ thống phân phối thuốc tự động tại bệnh viện

Nội dung chỉnh sửa báo cáo Đồ án tốt nghiệp theo góp ý của hội đồng:

STT	Nội dung góp ý của hội đồng	Kết quả thực hiện chỉnh sửa đồ án tốt nghiệp
1	Chưa mô tả rõ thuật toán động học điều khiển, đặc biệt là vector chuyển động điều khiển góc và tốc độ bánh xe.	Chỉnh sửa bổ xung mục 4.2.2.1. Quy trình điều khiển động cơ khi đã có tín hiệu điều khiển, trang 49 → 51
2	Chưa đề cập đến các tiêu chuẩn an toàn (ISO 13482, ISO 13849) trong môi trường có người vận hành.	Chỉnh sửa bổ xung mục 4.1.5. Yêu cầu và các tiêu chuẩn an toàn trang 44 → 45
3	Thiếu mô tả về xử lý sự cố, dừng khẩn cấp hoặc cảnh báo nguy hiểm	Chỉnh sửa bổ xung mục 4.1.6. Hệ thống cảnh báo và xử lý sự cố trang 45 → 46

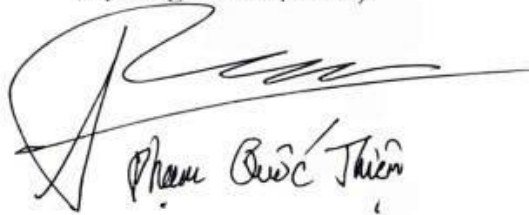
Xác nhận của giảng viên hướng dẫn (GVHD):

1. Giảng viên đã kiểm tra kết quả thực hiện chỉnh sửa đồ án tốt nghiệp của sinh viên sau khi bảo vệ.
2. File báo cáo đồ án tốt nghiệp của nhóm sinh viên thực hiện đã chỉnh sửa theo đúng góp ý của HĐ (GVHD đã kiểm tra file mềm báo cáo "định dạng .PDF" trùng khớp

với nội dung trong cuốn báo cáo Đồ án tốt nghiệp và đã ký xác nhận trên trang BÌA CHÍNH của bản báo cáo giấy)

3. Phiếu theo dõi tiến độ thực hiện đồ án tốt nghiệp của nhóm sinh viên (GVHD đã kiểm tra và ký xác nhận tính chính xác của các thông tin trong phiếu)

Giảng viên hướng dẫn
(Ký & ghi rõ họ tên)


Phạm Quốc Thiện

Tp. Hồ Chí Minh, ngày ... tháng ... năm 20...

Nhóm sinh viên thực hiện
(Các thành viên Ký & ghi rõ họ tên)


Phạm Văn Huy


Trần Đăng Khoa


Nguyễn Đình Khiêm

LỜI CAM ĐOAN

Chúng tôi xin cam đoan rằng đề tài đồ án tốt nghiệp này là công trình nghiên cứu của riêng chúng tôi do chính chúng tôi thực hiện dưới sự hướng dẫn của giảng viên hướng dẫn TS. Phạm Quốc Thiện

Chúng tôi xin cam kết toàn bộ nội dung trong bài báo cáo là sản phẩm của quá trình làm học tập, làm việc và nghiên cứu nghiêm túc của chúng tôi suốt thời gian thực hiện đồ án. Các số liệu tính toán và kết quả trong bài báo cáo là trung thực và chưa từng được công bố ở bất kì công trình nghiên cứu nào khác.

Tôi xin cam kết rằng mọi sự giúp đỡ cho việc hoàn thành báo cáo này đều đã được cảm ơn và mọi tài liệu tham khảo, hình ảnh, đều có ghi rõ nguồn gốc và trích dẫn rõ ràng.

Tp. Hồ chí minh, ngày 15 tháng 7 năm 2025

(Kí và ghi rõ họ tên)

LỜI CẢM ƠN

Đầu tiên, chúng tôi xin gửi lời cảm ơn trân thành đến giảng viên hướng dẫn TS. Phạm Quốc Thiện và theo sát và hướng dẫn tận tình cho chúng tôi trong suốt thời gian thực hiện đến lúc hoàn thành đề tài đồ án tốt nghiệp. Ngoài ra thầy còn đảm nhiệm vai trò giảng viên chủ nhiệm truyền đạt những kiến thức quý giá suốt thời gian học tập trên giảng đường đại học. Những kiến thức đó đã làm nền tảng vững chắc để cho chúng tôi có thể thực hiện đồ án tốt nghiệp này. Hơn nữa Thầy còn đưa ra những định hướng rõ ràng cho chúng tôi có những lựa chọn phù hợp. Nhờ vậy chúng tôi đã vượt qua giai đoạn khó khăn và thách thức để hoàn thành được đồ án này.

Chúng tôi gửi lời cảm ơn đến Công Ty TNHH TM & SX Thiên Phước đã tạo điều kiện thuận lợi cho chúng tôi thực tập và tiếp cận với môi trường làm việc thực tế, góp phần không nhỏ vào quá trình thực hiện đồ án này. Đặc biệt, cảm ơn anh Trần Viết Nghĩa – người hướng dẫn trực tiếp tại công ty, đã hướng dẫn, chia sẻ kinh nghiệm thực tiễn quý báu và tạo điều kiện sử dụng máy móc hiện đại hỗ trợ cho việc hoàn thiện mô hình robot cho đề tài đồ án của chúng tôi.

Cuối cùng, chúng tôi xin gửi lời cảm ơn sâu sắc đến Trường Đại học Công nghệ TP. Hồ Chí Minh (HUTECH) và các thầy cô thuộc Viện Kỹ thuật, những người đã tận tâm giảng dạy, hỗ trợ và tạo điều kiện thuận lợi cho chúng tôi trong suốt những năm tháng học tập và nghiên cứu tại trường.

Xin chân thành cảm ơn!

TÓM TẮT

Đề tài này ứng dụng robot di động với mô hình 3 bánh xe omni, được bố trí cách đều nhau 120° (dạng tam giác đều – Triangular), vào hệ thống phân phối thuốc tự động cho bệnh nhân trong bệnh viện. Nhờ tính linh hoạt cao của bánh xe omni, robot có thể di chuyển dễ dàng trong môi trường có diện tích hạn chế hoặc nhiều vật cản như bệnh viện.

Robot sử dụng công nghệ LiDAR để xây dựng bản đồ môi trường, từ đó có khả năng tự định vị và di chuyển chính xác đến các vị trí được chỉ định. Ngoài ra, hệ thống còn tích hợp LiDAR để phát hiện và tránh vật cản, đảm bảo robot di chuyển an toàn trong môi trường phức tạp. Robot sử dụng thuật toán AMCL để tự định vị trên bản đồ đã xây dựng, kết hợp với khả năng di chuyển linh hoạt giúp thực hiện việc phân phối thuốc chính xác và hiệu quả.

Robot hoạt động trên nền tảng hệ điều hành mã nguồn mở ROS (Robot Operating System), đóng vai trò trung tâm trong việc quản lý, kết nối và điều phối các thành phần của hệ thống robot, giúp các bộ phận giao tiếp và hoạt động nhịp nhàng. ROS đồng thời cung cấp nhiều công cụ và thư viện mạnh mẽ, hỗ trợ lập trình điều hướng, xử lý cảm biến và vận hành tự động mà không cần điều khiển thủ công.

Robot được lập trình sẵn hành trình để vận hành tự động. Quá trình giám sát và quản lý được thực hiện thông qua phần mềm giao diện người dùng, được phát triển bằng framework React Native. Người quản lý có thể theo dõi trạng thái robot và gửi lệnh điều khiển thông qua giao diện này.

Việc tự động hóa quá trình vận chuyển thuốc không chỉ giúp giảm tải cho nhân viên y tế, hạn chế tiếp xúc gần trong các tình huống dịch bệnh mà còn nâng cao hiệu quả và độ chính xác trong việc phân phối thuốc đến đúng bệnh nhân.

Từ khóa: robot di động, bánh xe omni, LiDAR, Camera 3D, ROS, hệ thống phân phối thuốc tự động, React Native.

ABSTRACT

This project applies a mobile robot with a three-omni-wheel triangular configuration, where the wheels are evenly spaced at 120° , into an automatic medicine delivery system for hospital patients. Thanks to the high maneuverability provided by omni wheels, the robot can navigate efficiently in constrained environments with many obstacles, such as hospital corridors and patient rooms.

The robot uses LiDAR technology to map the environment, enabling it to localize itself and navigate accurately to designated locations. Additionally, the system integrates LiDAR for obstacle detection and avoidance, ensuring safe movement in complex environments. By applying the AMCL algorithm for localization on the pre-built map and leveraging its high mobility, the robot can deliver medication accurately and efficiently.

The robot is powered by the open-source Robot Operating System (ROS), which serves as the core framework for managing, connecting, and coordinating all robot components, allowing seamless communication and synchronized operations. ROS also provides powerful tools and libraries for sensor processing, path planning, and autonomous control, reducing the need for manual intervention.

The robot's route is pre-programmed for autonomous operation. Monitoring and control are performed through a user interface application developed with the React Native framework, allowing administrators to track robot status and issue control commands remotely.

Automating the medicine delivery process helps reduce the workload for medical staff, minimizes close contact during infectious outbreaks, and enhances the accuracy and efficiency of medication distribution within hospitals.

Keywords: mobile robot, omni wheels, LiDAR, 3D camera, ROS, automated medication distribution system, React Native.

MỤC LỤC

LỜI CAM ĐOAN	i
LỜI CẢM ƠN.....	ii
TÓM TẮT.....	iii
ABSTRACT.....	iv
MỤC LỤC	v
DANH MỤC HÌNH ẢNH	viii
DANH MỤC BẢNG	xi
DANH MỤC CÁC TỪ VIẾT TẮT	xii
Chương 1. GIỚI THIỆU ĐỀ TÀI.....	1
1.1 Tính cấp thiết của đề tài.....	1
1.2 Mục tiêu nghiên cứu	1
1.3 Đối tượng nghiên cứu	2
1.4 Phương pháp nghiên cứu	2
1.5 Kết quả đạt được	2
1.6 Cấu trúc đề tài	3
Chương 2. TỔNG QUAN NGHIÊN CỨU.....	4
2.1 Bối cảnh nghiên cứu và các xu hướng ứng dụng.....	4
2.2 Các nghiên cứu và đề tài liên quan	4
2.2.1 Một số bài nghiên cứu trong nước	4
2.2.2 Một số bài nghiên cứu nước ngoài.....	6
Chương 3. CƠ SỞ LÝ THUYẾT	11
3.1 Động học robot di động	11
3.1.1 Động học thuận và nghịch robot di động.....	11
3.1.2 Ràng buộc nonholonomic và holonomic	13
3.1.3 Động học robot di động 3 bánh xe omni	13
3.2 Hệ điều hành Robot (ROS).....	18
3.2.1 giới thiệu	18
3.2.2 Sử dụng ROS	18
3.3 Công nghệ LiDAR.....	23
3.3.1 Định nghĩa.....	23
3.3.2 Nguyên tắc hoạt động của LiDAR.....	24
3.3.3 Các loại lidar.....	25
3.4 Thuật toán Dijkstra	26

3.4.1 Nguyên lí hoạt động [8].....	26
3.4.2 Các bước hoạt động của thuật toán Dijkstra	26
3.4.1 Ưu điểm và nhược điểm.....	28
3.5 Thuật toán TED local planner.....	29
3.5.1 Nguyên lí hoạt động.....	29
3.5.2 Cấu trúc TED local planner trong ROS	30
3.5.3 Các ràng buộc tối ưu trong TED.....	30
Chương 4. THIẾT KẾ PHẦN CỨNG VÀ PHẦN MỀM.....	31
4.1 Thiết kế phần cứng.....	31
4.1.1 Sơ lược về phần mềm thiết kế kĩ thuật	31
4.1.2 Kích thước và hình dáng tổng thể của robot.....	31
4.1.3 Các thành phần cơ khí robot	32
4.1.4 Các thành phần điện tử của robot.....	36
4.1.5 Yêu cầu và các tiêu chuẩn an toàn.....	44
4.1.6 Hệ thống cảnh báo và xử lý sự cố.....	45
4.2 Thiết kế phần mềm	46
4.2.1 Phương pháp đề xuất	46
4.2.2 Tầng nhúng (Embedded Layer)	47
4.2.2.1 Quy trình điều khiển động cơ khi đã có tính hiệu điều khiển.....	49
4.2.3 Tầng ROS layer	51
4.2.3.1 Cấu hình hệ thống.....	51
4.2.3.2 Xây dựng bản đồ với Hector SLAM.....	55
4.2.3.3 Navigation với Move Base	58
4.2.4 Mobile Application.....	62
Chương 5. TRIỂN KHAI MÔ HÌNH VÀ KẾT QUẢ	64
5.1 Mô hình robot hoàn thiện.....	64
5.2 Kết quả xây dựng bản đồ với Hector SLAM.....	64
5.3 Kết quả bám đường của thuật toán Ted Local Planner.....	66
5.4 Mobile app	71
Chương 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	73
6.1 Kết luận.....	73
6.1.1 Đã đạt được.....	73
6.1.2 Chưa đạt được.....	73
6.2 Hướng phát triển	74
TÀI LIỆU THAM KHẢO	76

Phụ lục. CODE HỆ THỐNG	77
------------------------------	----

DANH MỤC HÌNH ẢNH

Hình 2. 1 Hệ thống robot y tế Vibot	5
Hình 2. 2 Robot hỗ trợ trong bệnh viện	6
Hình 2. 3 Robot Carver-Cap và các thành phần chính.....	7
Hình 2. 4 Hình 1. Sơ đồ hệ thống trên CARVER và phía dịch vụ đám mây.....	7
Hình 2. 5 Mẫu thử nghiệm ứng dụng web nền tảng Carver.....	8
Hình 2. 6 Bản đồ trong GUI hiển thị đường dẫn và điểm đích của điều hướng.	8
Hình 2. 7 Mô hình robot y tế MedBuddy.....	9
Hình 2. 8 Sơ đồ hoạt động của MedBuddy	10
Hình 3. 1 Mô hình động học của robot trong hệ tọa độ toàn cục.....	11
Hình 3. 2 Mối quan hệ giữa động học thuận và động học nghịch	12
Hình 3. 3 Mô hình động học vi phân	13
Hình 3. 4 Cấu trúc robot di động 3 bánh xe omni.....	14
Hình 3. 5 Khả năng cơ động của robot 3 bánh xe omni.....	14
Hình 3. 6 Mô hình robot di động 3 bánh xe omni trong các hệ tọa độ	15
Hình 3. 7 Mô hình mô tả động học của robot	16
Hình 3. 8 Mô hình mô tả động học của robot	17
Hình 3. 9 Quy trình 5 bước phát triển robot với ROS	18
Hình 3. 10 Sơ đồ giao tiếp ros master và các nodes	19
Hình 3. 11 Mô hình giao tiếp giữa các node trong qua topics	20
Hình 3. 12 Mô hình giao tiếp giữa các node trong ROS thông qua Service.....	21
Hình 3. 13 Mối liên hệ giữa các node trong một ROS package	22
Hình 3. 14 Cấu trúc của workspace trong ROS	22
Hình 3. 15 Tổng quan công nghệ LiDAR ứng dụng trong hệ thống ADAS	23
Hình 3. 16 Nguyên tắc tính toán khoảng cách bằng cách sử dụng LiDAR	24
Hình 3. 17 Hoạt động Airborne LiDAR và bản đồ tái tạo	25
Hình 3. 18 Hoạt động Terrestrial LiDAR và bản đồ tái tạo.....	25
Hình 3. 19 Đồ thị minh họa đồ thị đơn giản với các đỉnh và trọng số [8]	27
Hình 3. 20 Mô hình Time-Elastic Band trong lập kế hoạch quỹ đạo [9].....	29

Hình 3. 21 Sơ đồ tích hợp TEB vào ROS Navigation Stack [10].....	30
Hình 4. 1 Phần mềm thiết kế 3D SolidWork 2023	31
Hình 4. 2 Bản vẽ thiết kế 3D tổng thể của robot.....	32
Hình 4. 3 Bản vẽ 3D để robot di động 3 bánh omni	33
Hình 4. 4 Bản vẽ 3D để robot tầng 2 di động 3 bánh omni	33
Hình 4. 5 Bản vẽ 3D để trên cùng robot di động 3 bánh omni	34
Hình 4. 6 Nhôm định hình.....	34
Hình 4. 7 Bánh xe đa hướng omni	35
Hình 4. 8 Động cơ bước vòng kín 2 pha YAKO	35
Hình 4. 9 Sơ đồ mạch điện điều khiển động cơ.....	36
Hình 4. 10 Sơ đồ mạch điều khiển đọc encoder động cơ.....	37
Hình 4. 11 Sơ đồ mạch điện điều khiển lớp trên của robot.....	38
Hình 4. 12 Máy tính nhúng Jetson nano	38
Hình 4. 13 Driver động cơ bước encoder vòng kín ESD2505M	39
Hình 4. 14 Rplidar A1M8	40
Hình 4. 15 Pin dự phòng	40
Hình 4. 16 Nguồn động lực.....	41
Hình 4. 17 Cảm biến gia tốc BMX190	41
Hình 4. 18 Vi điều khiển ESP32	42
Hình 4. 19 Vi điều khiển Arduino uno r3	43
Hình 4. 20 IC SN75175.....	43
Hình 4. 21 Mạch TXS0108	44
Hình 4. 22 Kiến trúc hệ thống.....	46
Hình 4. 23 Lưu đồ giải thuật cho MCU 1	48
Hình 4. 24 Lưu đồ giải thuật cho MCU2	49
Hình 4. 25 Kiến trúc hệ thống Hector SLAM.....	55
Hình 4. 26 Sơ đồ transform cho việc vẽ map.....	57
Hình 4. 27 Kiến trúc hệ thống điều hướng move_base trong ROS	58
Hình 4. 28 Sơ đồ transform cho navigation	59

Hình 5. 1 Mô hình robot di động omni giao thuốc trong bệnh viện	64
Hình 5. 2 Bản đồ 2D không gian làm việc của robot.....	65
Hình 5. 3 Quỹ đạo dự kiến và quỹ đạo thực tế của robot với thông số lần 1.....	67
Hình 5. 4 Quỹ đạo dự kiến và quỹ đạo thực tế của robot với thông số lần 2.....	68
Hình 5. 5 Quỹ đạo dự kiến và quỹ đạo thực tế của robot với thông số tối ưu	70
Hình 5. 6 Màn hình đăng nhập và kết nối robot	71
Hình 5. 7 Màn hình setting, thông tin tài khoản	71
Hình 5. 8 Màn hình Dashboard	72
Hình 5. 9 Thông số kỹ thuật và các tính năng của robot	72

DANH MỤC BẢNG

Bảng 4. 1 Cấu hình tham số RPLidar.....	52
Bảng 4. 2 Cấu hình tham số cho ekf_imu_filter_madgwick_node.....	52
Bảng 4. 3 Cấu hình tham số cho laser_scan_matcher.....	53
Bảng 4. 4 Cấu hình tham số cho bộ lọc ekf	54
Bảng 4. 5 Cấu hình tham số cho hector_mapping	56
Bảng 4. 6 Cấu hình cho amcl	59
Bảng 4. 7 Cấu hình tham số cho teb_local_planner.....	60
Bảng 4. 8 Cấu hình tham số recovery behaviors.....	62
Bảng 5. 1 Tham số Ted local Planner ban đầu.....	66
Bảng 5. 2 Tối ưu thông số Ted local Planner lần 2.....	67
Bảng 5. 3 Tối ưu thông số Ted local Planner cuối cùng	69

DANH MỤC CÁC TỪ VIẾT TẮT

Viết tắt	Ý nghĩa
IMU	Inertial Measurement Unit
TCP	Transmission Control Protocol
IP	Internet Protocol
MCU	Microcontroller Unit
ROS	Robot Operating System
UART	Universal Asynchronous Receiver Transmitter
SLAM	Simultaneous Localization and Mapping
PWM	Pulse Width Modulation
ADAS	Advanced Driver Assistance Systems
ToF	Time of Flight
UAV	Unmanned Aerial Vehicle
CW/CWW	Clockwise / Counter-Clockwise
HDIM	Human–Device Interaction Model
USB	Universal Serial Bus
I2C	Inter-Integrated Circuit
CPU	Central Processing Unit
OTA	Over-The-Air
URDF	Unified Robot Description Format
TF	Transform
EKF	Extended Kalman Filter
YAML	YAML Ain’t Markup Language
PGM	Portable Gray Map
AMCL	Adaptive Monte Carlo Localization
TEB	Time Elastic Band
UI	User interface

Chương 1. GIỚI THIỆU ĐỀ TÀI

1.1 Tính cấp thiết của đề tài

Trong bối cảnh ngành y tế đang ngày càng đẩy mạnh ứng dụng công nghệ nhằm nâng cao chất lượng dịch vụ và giảm tải cho nhân lực y tế, việc tự động hóa các quy trình nội viện trở nên cần thiết hơn bao giờ hết. Đặc biệt, hoạt động phân phối thuốc tại các bệnh viện, vốn đòi hỏi sự chính xác cao, nhanh chóng và liên tục, vẫn đang được thực hiện chủ yếu bằng phương pháp thủ công với sự tham gia của nhân viên y tế hoặc dược sĩ. Điều này không chỉ tốn nhiều thời gian và công sức, mà còn tiềm ẩn nguy cơ sai sót và lây nhiễm chéo trong môi trường bệnh viện.

Trước thực trạng đó, sự phát triển mạnh mẽ của khoa học công nghệ, đặc biệt trong các lĩnh vực tự động hóa và robot, đang mở ra những hướng đi mới nhằm nâng cao hiệu quả hoạt động của bệnh viện. Các hệ thống robot ngày nay không chỉ được ứng dụng trong công nghiệp mà còn được triển khai rộng rãi trong lĩnh vực y tế, nhờ khả năng thực hiện các tác vụ lặp lại, chính xác và hạn chế tiếp xúc trực tiếp. Việc tích hợp robot vào quy trình phân phối thuốc giúp giảm tải cho nhân viên, tiết kiệm thời gian, nâng cao độ chính xác và đảm bảo an toàn, góp phần hiện đại hóa bệnh viện và hướng tới xây dựng mô hình bệnh viện thông minh.

Với đặc thù môi trường trong bệnh viện thường có diện tích hạn chế, hành lang hẹp, nhiều vật cản và người qua lại, robot cần có khả năng di chuyển linh hoạt và thích ứng tốt với các điều kiện thực tế.

Từ những lý do trên, đề tài **"Ứng dụng robot di động omni trong hệ thống phân phối thuốc tự động tại bệnh viện"** là một hướng nghiên cứu mang tính cấp thiết, có ý nghĩa thực tiễn cao và phù hợp với xu thế phát triển của y tế thông minh hiện nay.

1.2 Mục tiêu nghiên cứu

Đề tài nghiên cứu nhằm mục đích hướng đến mô hình tự động hóa trong một lĩnh vực quy trình phân phối thuốc giúp giảm tải cho nhân lực y tế và giảm thiểu nguy cơ sai sót và truyền nhiễm trong môi trường bệnh viện

1.3 Đối tượng nghiên cứu

Robot di động với khả năng di chuyển linh hoạt trong môi trường bệnh viện, thực hiện nhiệm vụ phân phối thuốc tự động trong bệnh viện. Hệ thống có khả năng định vị, dẫn đường và giao thuốc chính xác theo lộ trình được thiết lập, sử dụng hệ điều hành mở nguồn mở ROS để quản lý và cảm biến LiDAR để xây dựng bản đồ môi trường và điều hướng.

1.4 Phương pháp nghiên cứu

Dựa vào phạm vi, đối tượng và mục tiêu nghiên cứu câu đề tài, đề án của chúng tôi sẽ được nghiên cứu theo phương pháp như sau

- Tham khảo các kết quả của các bài nghiên cứu liên quan trước đó để kế thừa và phát triển.
- Nghiên cứu các tài liệu có thông tin liên quan đến ứng dụng robot di động trong lĩnh vực y tế.
- Nghiên cứu khái niệm và lý thuyết về phương pháp liên điều khiển, nguyên lý hoạt động của các công nghệ thường sử dụng trong robot di động.
- Khảo sát các loại cơ cấu robot và các giải pháp điều khiển từ xa hoặc tự động đã được ứng dụng rộng rãi trong công nghiệp và lĩnh vực giải trí.
- Tìm hiểu điều kiện môi trường làm việc, nhiệm vụ cụ thể và khả năng đáp ứng của các loại công nghệ, để đề ra các phương pháp thiết kế phù hợp.

Bên cạnh đó, Nhóm tham khảo các tiêu chuẩn an toàn quốc tế để định hướng thiết kế robot phù hợp với môi trường bệnh viện. Cụ thể, **ISO 13482** (Robots and robotic devices – Safety requirements for personal care robots) **ISO 13849** (Safety of machinery – Safety-related parts of control systems).

1.5 Kết quả đạt được

Tính toán, thiết kế và chế tạo khung robot

Đọc encoder động cơ và cảm biến IMU

Lập trình điều khiển động cơ cho robot di chuyển bằng các giải pháp động

Sử dụng ROS kết hợp với công nghệ cảm biến LiDAR để lập bản đồ không gian làm việc xung quanh robot, xác định vị trí và lập kế hoạch đường đi cho robot.

1.6 Cấu trúc đề tài

Đề tài đồ án tốt nghiệp “Ứng dụng robot di động omni trong hệ thống phân phối thuốc tự động tại bệnh viện” bao gồm 6 chương:

Chương 1 - Giới thiệu: Tổng quan về đề tài, trình bày bối cảnh thực tế, lý do chọn đề tài, mục tiêu nghiên cứu, phương pháp thực hiện và bố cục tổng thể của đồ án. Cuối cùng nêu ra các kết quả đã đạt được trong đồ án này.

Chương 2 - Tổng quan nghiên cứu: Trình bày bức tranh tổng quát hơn về tình hình chung và xu hướng công nghệ. Tổng hợp các nghiên cứu tiêu biểu trong nước cũng như nước ngoài, giải pháp liên quan đến robot di động, hệ thống phân phối thuốc tự động và các công nghệ hiện có, từ đó rút ra nhận xét, đánh giá và định hướng cho đề tài.

Chương 3 - Cơ sở lý thuyết: Trình bày các nội dung lý thuyết quan trọng làm nền tảng cho việc thiết kế và triển khai hệ thống, phân tích chuyên sâu về các kiến trúc và công nghệ chủ chốt được sử dụng trong đề tài, bao gồm: Động học robot di động cụ thể model 3 bánh omni, các thuật toán điều hướng (như Djistra, TEB), nguyên lý của cảm biến LiDAR, kiến trúc các thư viện hỗ trợ mạnh mẽ và quản lý của hệ điều hành robot ROS.

Chương 4 - Qui trình thiết kế phần cứng và phần mềm: Trình bày chi tiết các bước thiết kế và xây dựng hệ thống, bao phần cứng các thành phần cấu tạo nên khung robot. Các thành phần điện điện tử, cấu trúc điều khiển lớp dưới và lớp trên của robot và phần mềm (lập trình điều khiển, xử lý bản đồ, điều hướng bằng ROS, giao tiếp giữa các thành phần). Phần này là trọng tâm của quá trình hiện thực hóa mô hình.

Chương 5 - Triển khai mô hình và kết quả: Trình bày việc triển khai mô hình robot hoàn thiện và các kết quả đạt được từ quá trình thử nghiệm, bao gồm kết quả xây dựng bản đồ với Hector SLAM, kết quả bám đường của thuật toán TEB Local Planner, phát triển áp di động.

Chương 6 - Kết luận và hướng phát triển: Tổng kết những thành tựu và hạn chế của đề tài, đồng thời đề xuất các hướng phát triển tiếp theo cho nghiên cứu.

Chương 2. TỔNG QUAN NGHIÊN CỨU

2.1 Bối cảnh nghiên cứu và các xu hướng ứng dụng

Trong bối cảnh chuyển đổi số ngành y tế đang diễn ra mạnh mẽ, việc tích hợp các công nghệ hiện đại như Internet of Things (IoT), trí tuệ nhân tạo (AI) và robot tự hành đang ngày càng trở nên phổ biến. Các giải pháp này không chỉ giúp tăng hiệu quả vận hành mà còn mở ra hướng phát triển cho các mô hình bệnh viện thông minh. Điển hình, tại Nhật Bản và Hàn Quốc, nhiều bệnh viện đã đưa robot vào hỗ trợ công việc phân phối thuốc, giao mẫu xét nghiệm và vận chuyển vật tư.

Bên cạnh đó, đại dịch COVID-19 là một minh chứng rõ nét cho thấy nhu cầu cấp bách trong việc giảm tiếp xúc giữa nhân viên y tế và bệnh nhân, đồng thời duy trì hiệu quả vận chuyển nội viện. Những kinh nghiệm này đã thúc đẩy các quốc gia đầu tư vào hệ thống robot tự hành, như robot dẫn đường, robot giao thuốc và robot hỗ trợ thông tin y tế.

Những xu hướng này cho thấy ứng dụng robot di động không còn là lựa chọn thử nghiệm, mà đã trở thành một phần trong chiến lược phát triển hệ thống y tế hiện đại, hướng tới mô hình vận hành thông minh, an toàn và tự động hóa cao.

2.2 Các nghiên cứu và đề tài liên quan

2.2.1 Một số bài nghiên cứu trong nước

Hệ thống robot y tế "Vibot" (Việt Nam, 2020–2021)

Bài nghiên cứu này chúng tôi tìm hiểu và phân tích về “Hệ thống robot y tế vận chuyển trong khu cách ly bệnh truyền nhiễm có nguy cơ cao, đặt tên là Vibot” [1]. Được triển khai thực hiện bởi Học viện Kỹ thuật Quân sự (HVKTQS) trong bối cảnh toàn thế giới cũng như cả nước đang phải gồng mình phòng, chống đại dịch Covid-19.

Bài báo tập trung vào việc giảm tiếp xúc giữa nhân viên y tế và bệnh nhân trong khu cách ly, góp phần phòng chống lây nhiễm trong môi trường bệnh viện bằng cách tự động hóa quy trình vận chuyển thuốc men, thức ăn, vật tư y tế đến phòng bệnh.



Hình 2. 1 Hệ thống robot y tế Vibot

Robot được phát triển với khả năng Robot di chuyển bám đường dán sẵn trên sàn hành lang bằng (magnetic line following). Dùng thẻ Thẻ RFID/ID để xác định các trạm dừng (điểm giao thuốc/thức ăn). Tích hợp các cảm biến Phát hiện vật cản để dừng khẩn cấp hoặc tránh va chạm. Giao tiếp từ xa với hệ thống truyền dẫn không dây giữa các robot và trung tâm giám sát, điều khiển.

Hệ thống robot hỗ trợ trong bệnh viện (Việt Nam, 2024)

Bài nghiên cứu này chúng tôi tìm hiểu và phân tích về “Hệ thống robot hỗ trợ trong bệnh viện” [2] do TS. Đặng Xuân Ba cùng nhóm giảng viên, sinh viên Trường Đại học Sư phạm Kỹ thuật TP.HCM phát triển và giới thiệu tại hội thảo do Trung tâm Thông tin – Thống kê KH&CN TP.HCM tổ chức ngày 1/11/2024.

Bối cảnh dự án nhằm hỗ trợ bệnh viện giảm tải cho nhân viên y tế, nâng cao trải nghiệm và an toàn cho bệnh nhân trong khâu hướng dẫn, lễ tân và giao tiếp thông tin.

Bài báo tập trung vào hai chức năng chính:

- Dẫn đường tự động trong không gian bệnh viện đã được quét và lưu bản đồ.
- Hỗ trợ thông tin và giao tiếp với bệnh nhân qua giọng nói, nhận diện khuôn mặt và hiển thị thông báo.



Hình 2. 2 Robot hỗ trợ trong bệnh viện

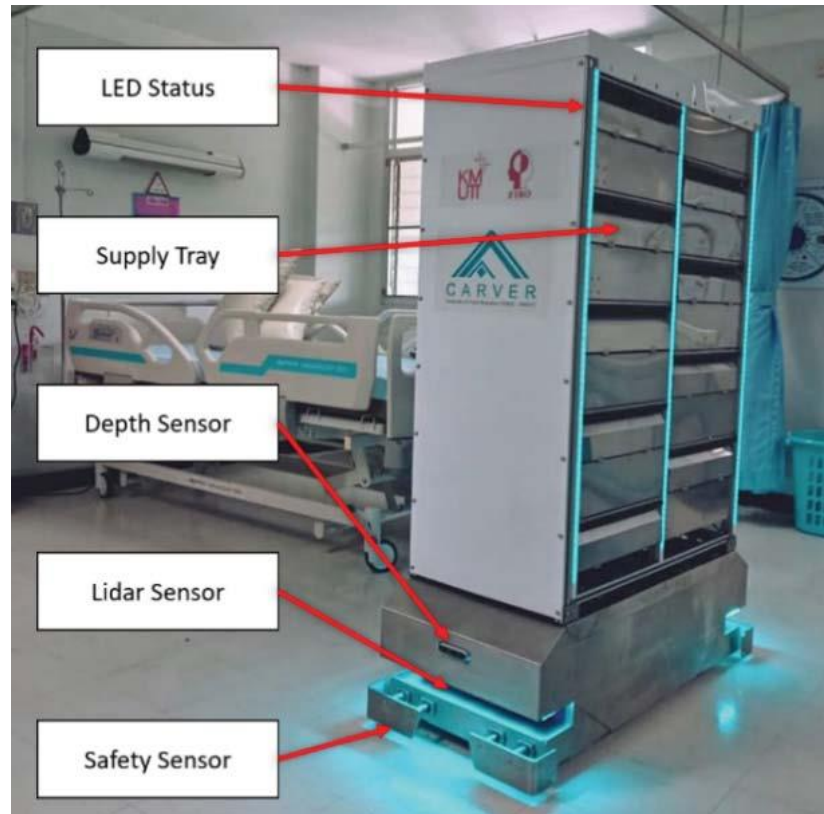
Robot sử dụng LiDAR và ROS để tự quét bản đồ, lập kế hoạch đường đi và tránh vật cản, đồng thời phát âm thanh nhường đường khi gặp người; tích hợp AI tương tác giọng nói, nhận diện khuôn mặt, cơ cấu 4 bánh di chuyển trên địa hình bằng phẳng.

2.1.2 Một số bài nghiên cứu nước ngoài

Hệ thống robot y tế “CARVER” ứng dụng nền tảng IoT (Thái Lan, 2021)

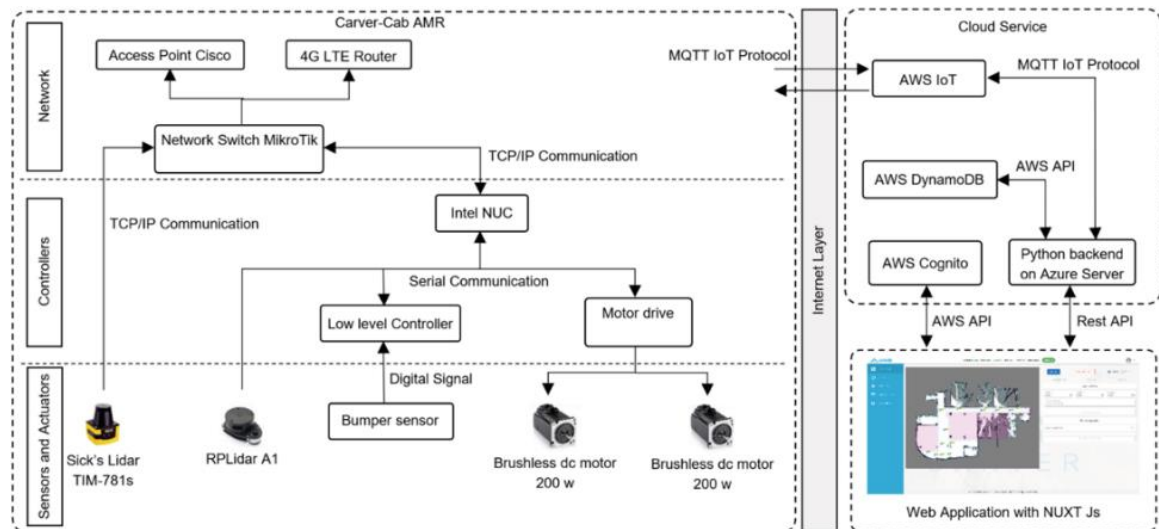
Bài nghiên cứu tiếp theo đã chúng tôi tìm hiểu và phân tích về “A Framework of IoT Platform for Autonomous Mobile Robot in Hospital Logistics Applications” [3]. được thực hiện bởi nhóm tác giả thuộc Viện Robot hiện trường (Institute of Field Robotics) – Đại học Công nghệ King Mongkut Thonburi, Thái Lan.

Nghiên cứu tập trung phát triển hệ thống robot tự hành (Autonomous Mobile Robot – AMR) tích hợp nền tảng Internet vạn vật (IoT) để phục vụ công tác phân phối thuốc và vật tư y tế tại các bệnh viện, đặc biệt là trong bối cảnh dịch COVID-19 bùng phát.



Hình 2. 3 Robot Carver-Cap và các thành phần chính

Hệ thống được đặt tên là CARVER – bao gồm robot AMR, nền tảng điều khiển qua web, cơ chế xác thực cá nhân và mô hình quản lý dữ liệu điện toán đám mây.

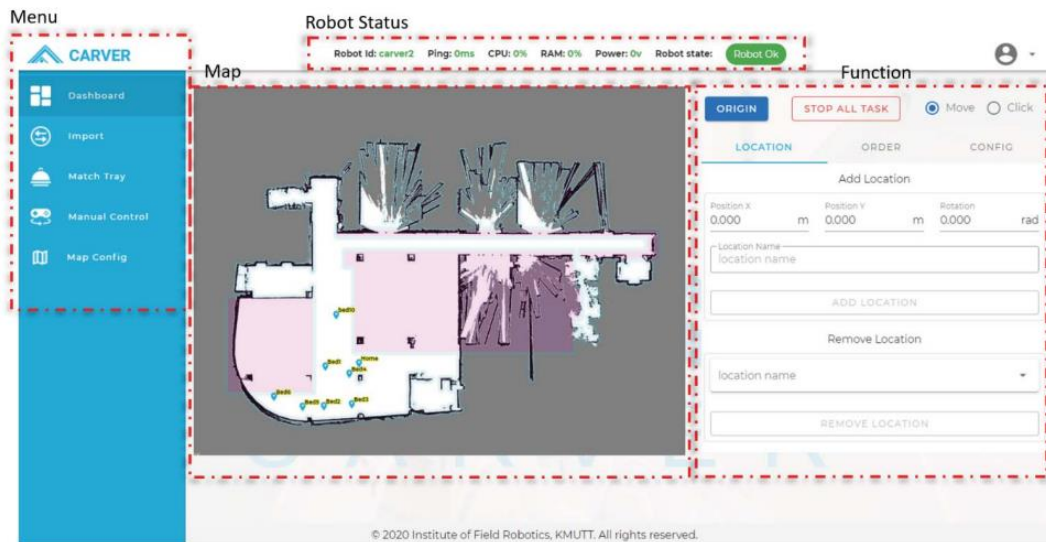


Hình 2. 4 Hình 1. Sơ đồ hệ thống trên CARVER và phía dịch vụ đám mây

Robot được phát triển với khả năng vận hành ở hai chế độ: tự động và điều khiển thủ công từ xa. Lịch trình vận chuyển được nhập bằng tệp CSV từ hệ thống quản lý

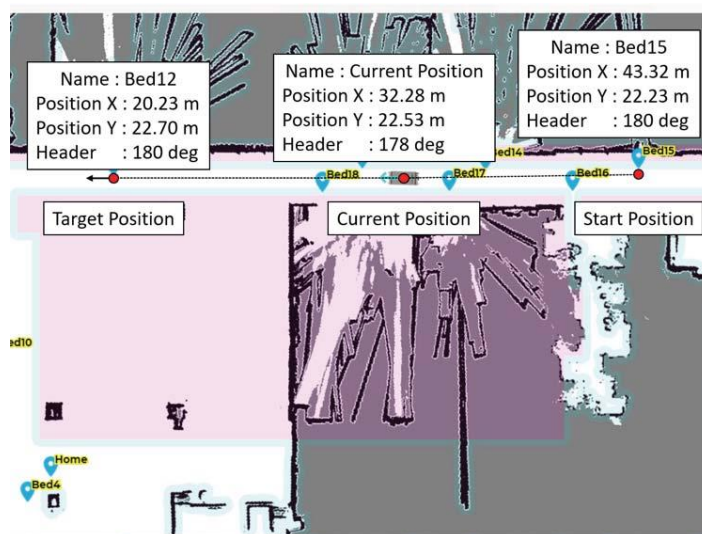
bệnh viện (HIS), sau đó truyền đến robot qua nền tảng web sử dụng MQTT protocol. Mỗi robot CARVER được trang bị cảm biến LiDAR công nghiệp, bộ điều khiển ROS, máy tính công suất cao Intel NUC, kết nối Wi-Fi và 4G dự phòng.

Thử nghiệm trong môi trường mô phỏng bệnh viện, Kết quả, Nhân viên y tế có thể giám sát robot CARVER theo thời gian thực qua giao diện điều khiển.



Hình 2. 5 Mẫu thử nghiệm ứng dụng web nền tảng Carver

Giao diện bảng điều khiển hiển thị vị trí bắt đầu, vị trí hiện tại và điểm đến của robot, đồng thời liên tục cập nhật tọa độ (X, Y), hướng quay và trạng thái hoạt động. Các vị trí giường bệnh được định nghĩa sẵn trên bản đồ và hiển thị rõ ràng theo từng nhãn, giúp người vận hành dễ theo dõi và kiểm soát quá trình giao thuốc.

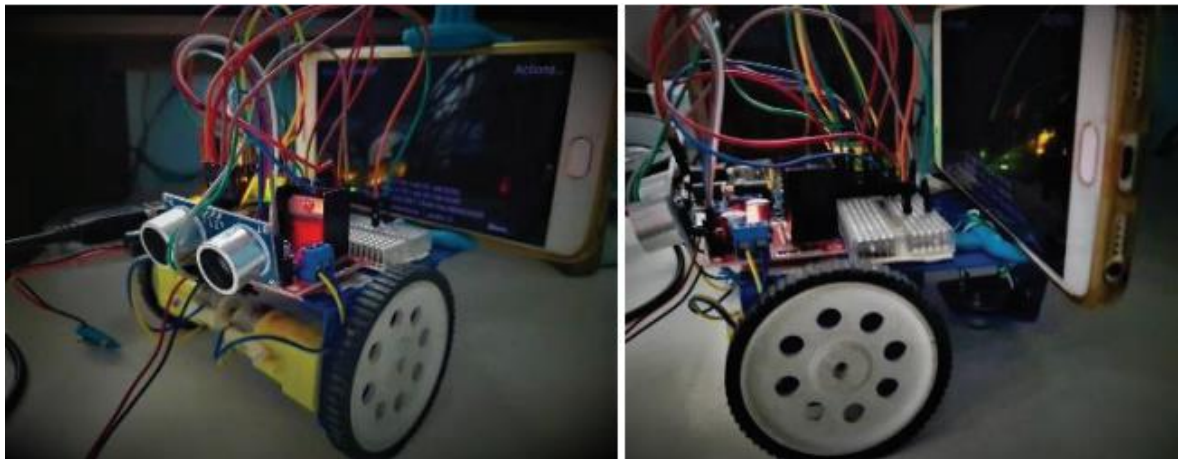


Hình 2. 6 Bản đồ trong GUI hiển thị đường dẫn và điểm đích của điều hướng.

Hệ thống robot y tế "MedBuddy" (Ấn Độ, 2021)

Bài nghiên cứu này chúng tôi tìm hiểu và phân tích về “MedBuddy: The Medicine Delivery Robot” [4] do nhóm tác giả Akshet Patel, Pranav Sharma và Princy Randhawa thuộc Đại học Manipal Jaipur, Ấn Độ thực hiện và công bố tại Hội nghị quốc tế IEEE ICRITO năm 2021. Bài nghiên cứu được thực hiện trong bối cảnh dịch COVID-19 bùng phát, nhằm giảm tiếp xúc trực tiếp giữa nhân viên y tế và bệnh nhân, qua đó hạn chế nguy cơ lây nhiễm chéo.

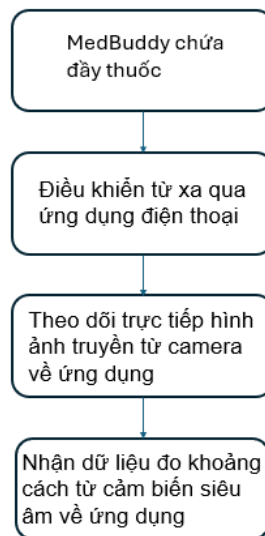
Nội dung tập trung vào thiết kế robot vận chuyển thuốc đơn giản, Robot được thiết kế theo mô hình xe hai bánh truyền động, điều khiển từ xa bằng Bluetooth qua ứng dụng điện thoại, cho phép giao thuốc an toàn ở khoảng cách lên đến 15 m.



Hình 2. 7 Mô hình robot y tế MedBuddy

Quy trình vận hành mô phỏng giao thuốc thực tế của robot MedBuddy. Được mô tả theo sơ đồ khối và lưu đồ hoạt động (Flowchart) sau:

Flowchart of working of MedBuddy



Hình 2. 8 Sơ đồ hoạt động của MedBuddy

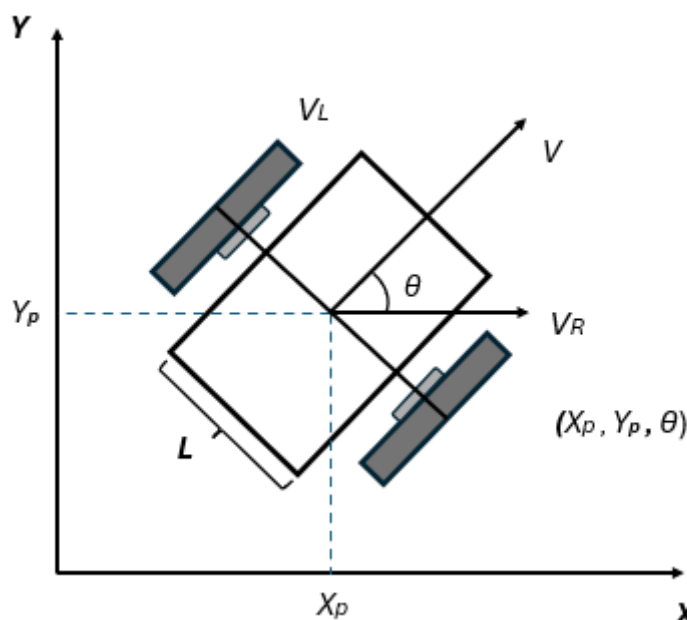
Kết quả robot hoạt động ổn định và phối hợp hiệu quả, Các thành phần chính của hệ thống như module Bluetooth, mạch điều khiển động cơ L298N, cảm biến siêu âm HC-SR04 và Arduino Uno đều được kiểm thử thành công. Đảm bảo robot vận hành đúng chức năng đã thiết kế. Hệ thống đạt được mục tiêu vận chuyển thuốc từ xa một cách an toàn, đơn giản và chi phí thấp trong môi trường cách ly.

Chương 3. CƠ SỞ LÝ THUYẾT

3.1 Động học robot di động

Động học robot di động tập trung vào việc nghiên cứu cách robot thay đổi vị trí và hướng trong không gian làm việc của chúng. Nó liên quan đến việc phân tích mối quan hệ giữa các yếu tố hình học của robot (như kích thước, vị trí bánh xe) và các hạn chế trong chuyển động của chúng. Các phương trình động học được xây dựng dựa trên cấu trúc cơ học của robot.

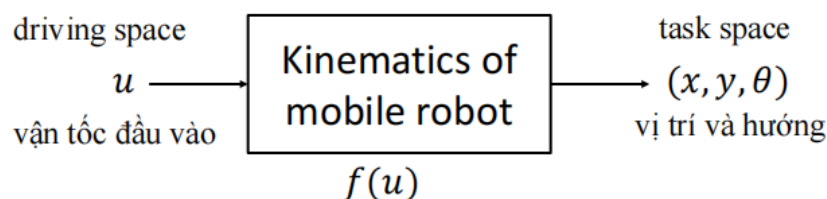
Robot di động, chúng có thể sử dụng một, hai, ba hoặc nhiều bánh xe, với hoặc không có giới hạn trong chuyển động (chẳng hạn như bánh xe bị khóa theo một số hướng).



Hình 3. 1 Mô hình động học của robot trong hệ tọa độ toàn cục

3.1.1 Động học thuận và nghịch robot di động

Động học robot di động là bài toán tìm mối quan hệ giữa không gian truyền động và không gian nhiệm vụ.



Xét đến 1 robot di động có tọa độ tổng quát là $q_1, q_2, \dots, q_n$ và các biến đầu vào $u_1, u_2, \dots, u_m$ trong không gian nhiệm vụ. Xác định được các vector:

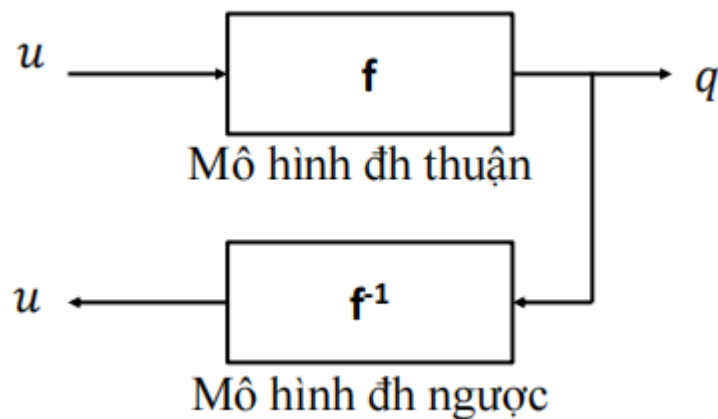
$$q = \begin{bmatrix} q_1 \\ q_2 \\ \dots \\ q_n \end{bmatrix} \quad u = \begin{bmatrix} u_1 \\ u_2 \\ \dots \\ u_m \end{bmatrix}$$

Không gian chuyển động Là không gian của các biến điều khiển (u) nơi đưa các tiến hiệu điều khiển, Ví dụ: góc bánh xe, tốc độ quay bánh xe, hoặc các tham số điều khiển khác để tác động đến robot.

Không gian nhiệm vụ: Là không gian mà robot thực hiện công việc, được mô tả bởi tọa độ q , ví dụ như vị trí hoặc hướng của một điểm trong không gian làm việc. Đây là không gian liên quan đến kết quả chuyển động tổng thể của robot.

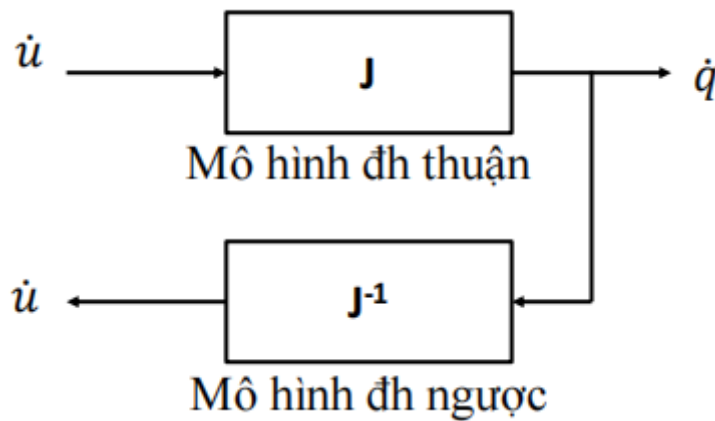
Động học thuận (Direct Kinematic): là quá trình xác định tọa độ trong không gian tác vụ $q_1, q_2, \dots, q_n$ dựa trên các thông số điều khiển $u_1, u_2, \dots, u_m$

Động học nghịch (Inverse Kinematic): là quá trình tính toán các thông số điều khiển $u_1, u_2, \dots, u_m$. Để đạt được vị trí và hướng mong muốn trong không gian tác vụ $q_1, q_2, \dots, q_n$.



Hình 3. 2 Mối quan hệ giữa động học thuận và động học nghịch

Mô hình động học vi phân:



Hình 3. 3 Mô hình động học vi phân

3.1.2 Ràng buộc nonholonomic và holonomic

Ràng buộc holonomic là các ràng buộc có thể được biểu diễn dưới dạng các phương trình đại số hoặc phương trình động học liên quan đến vị trí và thời gian. Những ràng buộc này hoàn toàn giới hạn chuyển động của hệ thống robot trong không gian làm việc.

Đặc điểm:

Là ràng buộc toàn phần: hoàn toàn giới hạn chuyển động của hệ thống.

Thường được biểu diễn bằng các phương trình đại số dạng:

$$f(q_1, q_2, \dots, q_n, t) = 0$$

Ràng buộc nonholonomic là các ràng buộc liên quan đến vận tốc hoặc đạo hàm thời gian của các biến trạng thái, không thể được tích phân để biểu diễn trực tiếp dưới dạng ràng buộc vị trí. Những ràng buộc này thường xuất hiện trong robot di động, do tính chất chuyển động phụ thuộc vào cấu trúc bánh xe hoặc động cơ.

Tính chất:

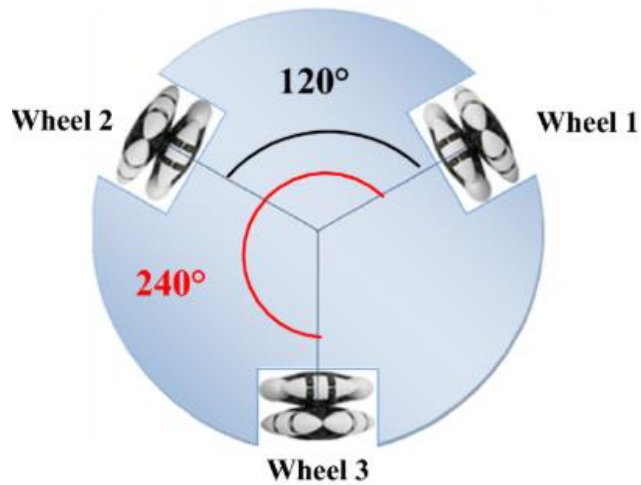
Ràng buộc nonholonomic thường liên quan đến tốc độ hoặc hướng chuyển động, chứ không phải vị trí.

$$f(q_1, q_2, \dots, q_n, \dot{q}_1, \dot{q}_2, \dots, \dot{q}_n, t) = 0$$

3.1.3 Động học robot di động 3 bánh xe omni

Robot di động 3 bánh xe omni (Omnidirectional Mobile Robot - 3OMR) là loại robot sử dụng 3 bánh xe omni được bố trí cách đều nhau 120° quanh tâm robot. Đây

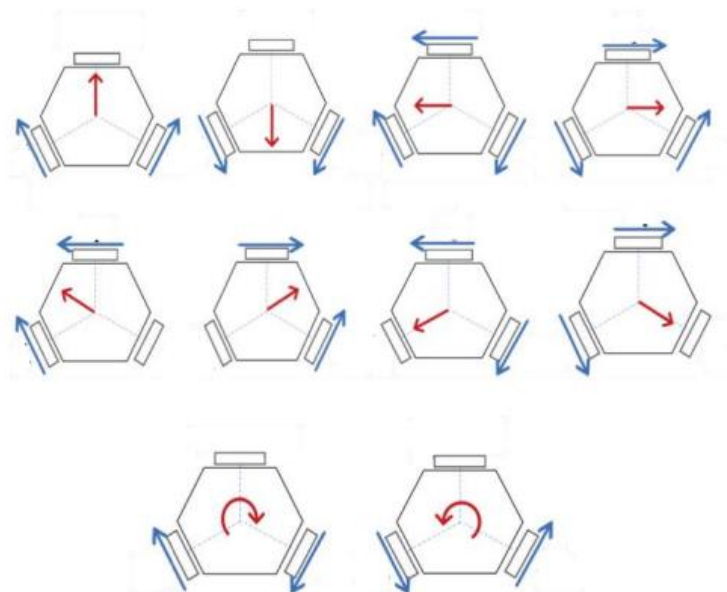
là một cấu trúc rất phổ biến trong các ứng dụng cần tính linh hoạt cao, như trong môi trường chật hẹp hoặc trong các hệ thống vận chuyển tự động.



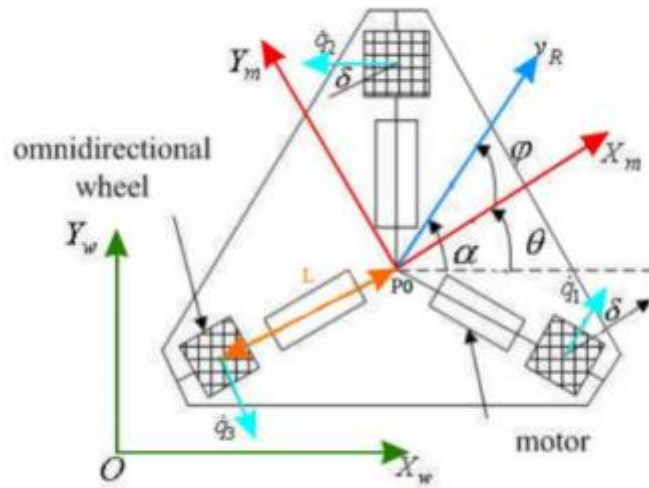
Hình 3. 4 Cấu trúc robot di động 3 bánh xe omni

Khả năng cơ động của robot omni

Robot di động sử dụng ba bánh xe omni có khả năng di chuyển linh hoạt theo mọi hướng trên mặt phẳng. Nhờ cấu trúc đặc biệt của bánh xe omni – gồm các con lăn nhỏ cho phép trượt ngang – robot có thể xoay tại chỗ và di chuyển theo bất kỳ hướng nào mà không cần quay đầu. Đây là ưu điểm nổi bật so với các loại robot di động khác như robot vi sai (differential drive) hay robot Mecanum.



Hình 3. 5 Khả năng cơ động của robot 3 bánh xe omni



Hình 3. 6 Mô hình robot di động 3 bánh xe omni trong các hệ tọa độ

Trong đó:

- (O, X_w, Y_w) biểu diễn hệ tọa độ toàn cục
- (X_m, Y_m) biểu diễn hệ tọa độ cục bộ gắn liền với robot
- ϕ_1, ϕ_2, ϕ_3 là các góc quay của các bánh xe
- $\dot{q}_1, \dot{q}_2, \dot{q}_3$ là các vận tốc quay của bánh xe
- δ là góc giữa mặt phẳng bánh xe và phương của trục quay
- α là Góc giữa phương của trục X_m (khung thân robot) và trục của bánh xe.
- θ là Góc quay của robot so với hệ tọa độ toàn cục (X_w, Y_w)
- L Khoảng cách từ tâm robot (P_0) đến vị trí bánh xe.

Phương trình động học [5] của robot được biểu diễn như sau

$$\begin{bmatrix} \dot{x}_R \\ \dot{y}_R \\ \dot{\theta} \end{bmatrix} = \begin{bmatrix} \frac{2}{3} \cos(\theta + \delta) & -\frac{2}{3} \cos(\theta - \delta) & \frac{2}{3} \sin(\theta) \\ \frac{2}{3} \sin(\theta + \delta) & -\frac{2}{3} \sin(\theta - \delta) & \frac{2}{3} \cos(\theta) \\ \frac{1}{3L} & \frac{1}{3L} & \frac{1}{3L} \end{bmatrix} \begin{bmatrix} \dot{q}_1 \\ \dot{q}_2 \\ \dot{q}_3 \end{bmatrix}$$

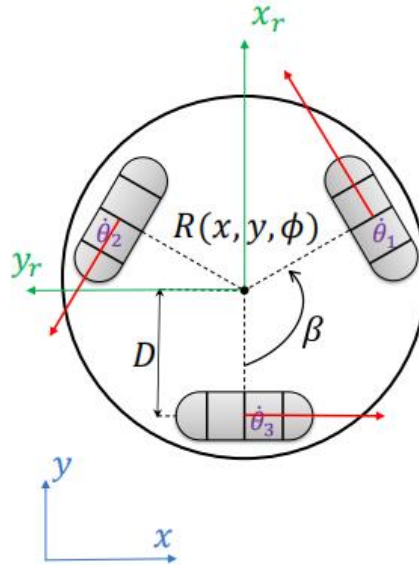
với (x_R, y_R) là tọa độ của robot, θ là hướng của robot, δ là góc giữa mặt phẳng bánh xe và phương của trục quay trong trường hợp này là 30. $(\dot{x}_R, \dot{y}_R)$ và $\dot{\theta}$ lần lượt là vận tốc tuyến tính và vận tốc góc của robot:

$$\dot{x}_R = \begin{bmatrix} \frac{2}{3} \cos(\theta + \delta) & -\frac{2}{3} \cos(\theta - \delta) & \frac{2}{3} \sin(\theta) \end{bmatrix} \cdot \begin{bmatrix} \dot{q}_1 \\ \dot{q}_2 \\ \dot{q}_3 \end{bmatrix}$$

$$\dot{y}_R = \begin{bmatrix} \frac{2}{3} \sin(\theta + \delta) & -\frac{2}{3} \sin(\theta - \delta) & \frac{2}{3} \cos(\theta) \end{bmatrix} \cdot \begin{bmatrix} \dot{q}_1 \\ \dot{q}_2 \\ \dot{q}_3 \end{bmatrix}$$

$$\dot{x}_R = \begin{bmatrix} \frac{1}{3L} & \frac{2}{3L} & \frac{2}{3L} \end{bmatrix} \cdot \begin{bmatrix} \dot{q}_1 \\ \dot{q}_2 \\ \dot{q}_3 \end{bmatrix}$$

Mô hình động học thuận



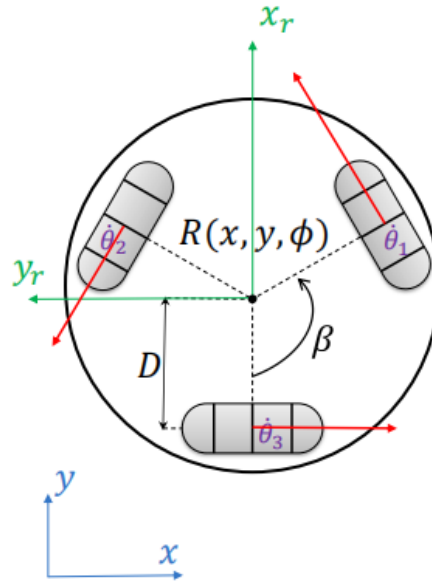
Hình 3. 7 Mô hình mô tả động học của robot

➔ Phương trình động học thuận

$$\begin{bmatrix} \dot{x}_r \\ \dot{y}_r \\ \dot{\phi}_r \end{bmatrix} = R \begin{bmatrix} \frac{\sqrt{3}}{3} & \frac{-\sqrt{3}}{3} & 0 \\ \frac{1}{3} & \frac{1}{3} & -\frac{2}{3} \\ \frac{1}{3L} & \frac{1}{3L} & \frac{1}{3L} \end{bmatrix} \begin{bmatrix} \dot{\theta}_1 \\ \dot{\theta}_2 \\ \dot{\theta}_3 \end{bmatrix}$$

$$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\phi} \end{bmatrix} = \begin{bmatrix} \cos\phi & -\sin\phi & 0 \\ \sin\phi & \cos\phi & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \dot{x}_r \\ \dot{y}_r \\ \dot{\phi}_r \end{bmatrix}$$

Mô hình động học nghịch



Hình 3. 8 Mô hình mô tả động học của robot

→ Phương trình động học nghịch

$$\begin{bmatrix} \dot{\theta}_1 \\ \dot{\theta}_2 \\ \dot{\theta}_3 \end{bmatrix} = \frac{1}{R} \begin{bmatrix} \frac{1}{2} & \frac{\sqrt{3}}{2} & L \\ -\frac{1}{2} & \frac{\sqrt{3}}{2} & L \\ 0 & -1 & L \end{bmatrix} \begin{bmatrix} \dot{x}_r \\ \dot{y}_r \\ \dot{\phi}_r \end{bmatrix}$$

$$\begin{bmatrix} \dot{x}_r \\ \dot{y}_r \\ \dot{\phi}_r \end{bmatrix} = \begin{bmatrix} \cos\phi & \sin\phi & 0 \\ -\sin\phi & \cos\phi & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\phi} \end{bmatrix}$$

3.2 Hệ điều hành Robot (ROS)

3.2.1 giới thiệu

ROS (Robot Operating System) là một nền tảng phần mềm mã nguồn mở dành cho lập trình và phát triển ứng dụng robot. Hệ thống này cung cấp nhiều dịch vụ tương tự như một hệ điều hành truyền thống, bao gồm: trừu tượng hóa phần cứng, giao tiếp giữa các tiến trình, điều khiển thiết bị cấp thấp, quản lý gói, và triển khai các chức năng thường dùng trong robot. Ngoài ra, ROS còn cung cấp tập hợp công cụ và thư viện hỗ trợ quá trình lập trình, biên dịch và thực thi mã trên một hoặc nhiều máy tính khác nhau.

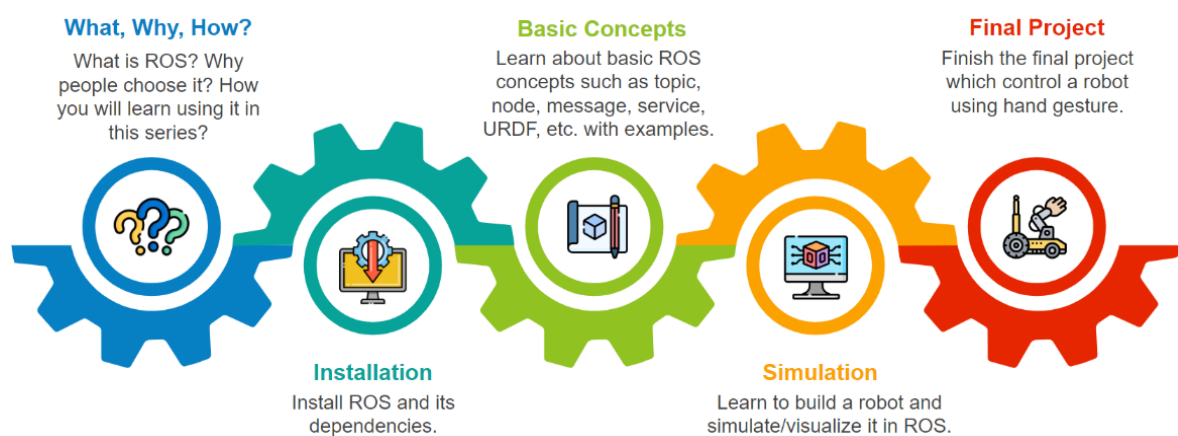
ROS hiện chỉ chạy trên nền tảng Unix. Phần mềm cho ROS chủ yếu được thử nghiệm trên hệ thống Ubuntu và Mac OS X, mặc dù cộng đồng ROS đã đóng góp hỗ trợ cho Fedora, Gentoo, Arch Linux và các nền tảng Linux khác. [6]

Tại sao lại chọn ROS

Trên trang chủ của ROS có dòng chữ "Don't reinvent the wheel. Create something new and do it faster and better by building on ROS!" nôm na nghĩa là không cần phải mất công làm lại những cái có sẵn, ROS giúp bạn dựng lên những thứ mới nhanh và hiệu quả hơn.

3.2.2 Sử dụng ROS

Để có thể hiểu và bắt đầu sử dụng Robot Operating System – ROS, một trong những nền tảng phổ biến nhất hiện nay cho phát triển robot. Ta có thể tóm gọn nó qua 5 bước chính ở hình dưới đây [6].



Hình 3. 9 Quy trình 5 bước phát triển robot với ROS

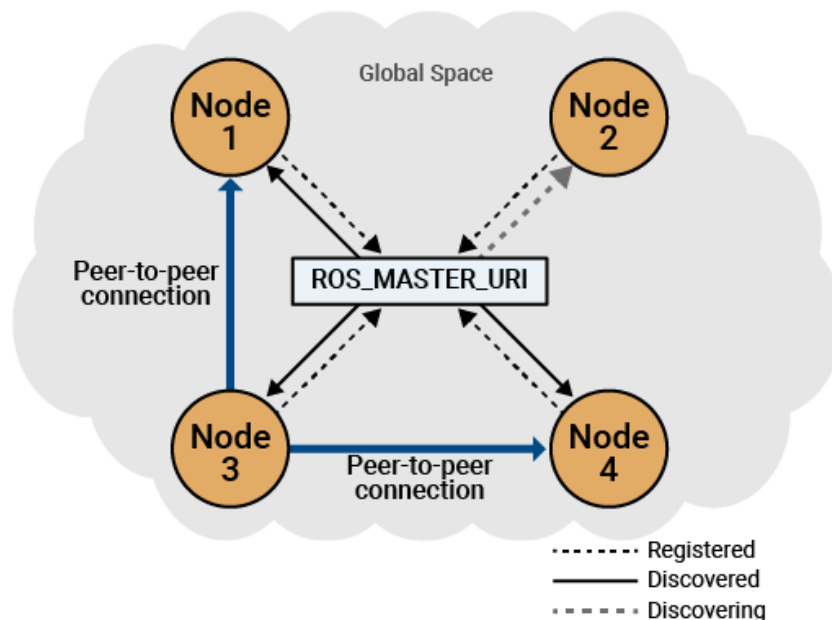
Qua phần giới thiệu thì phần nào ta cũng hiểu được ROS – Robot operation system là gì, tại sao chúng ta lại chọn ROS cho hệ thống robot và lợi ích của nó mang lại.

Bước quan trọng thứ 2 là cài đặt được ROS – Robot operation system cùng các phụ thuộc (dependencies) cần thiết. Quá trình này bao gồm tải phiên bản ROS phù hợp, cấu hình môi trường và đảm bảo các thư viện hỗ trợ như Python hoặc C++ được tích hợp.

Sau khi đã cài đặt thành công ROS và các phụ thuộc của nó. Bước tiếp theo cần nắm vững các khái niệm cơ bản của nó để biết cách sử dụng. 1 Số khái niệm quan trọng cần nắm đó là:

Ros master

ROS dựa vào một nút trung tâm gọi là ROS Master để phát hiện các node (nút) và thiết lập kết nối giữa chúng. ROS Master cung cấp dịch vụ đăng ký tên và tra cứu tên cho các node khác trong mạng, giúp chúng tìm thấy nhau và giao tiếp với nhau. Nó cho phép các node phát hiện nhau và trao đổi thông điệp (message) với nhau. ROS_MASTER_URI là một biến môi trường chỉ định địa chỉ của ROS Master. URI này cho phép các node tìm thấy và kết nối với ROS Master.



Hình 3. 10 Sơ đồ giao tiếp ros master và các nodes

Nodes

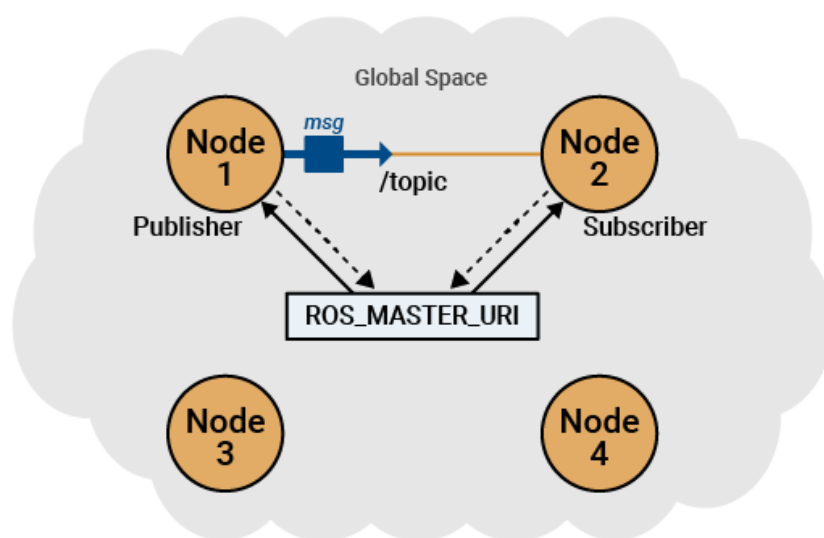
Nodes là các tiến trình độc lập trong ROS, thực hiện các phép tính và giao tiếp với nhau qua việc gửi, nhận thông điệp. Khi khởi động, node sẽ đăng ký với ROS Master, cung cấp tên, các topic, dịch vụ và tham số liên quan. Một hệ thống robot thường có nhiều node, mỗi node đảm nhiệm một chức năng riêng như xử lý cảm biến, điều khiển động cơ hay ra quyết định. Nodes được thiết kế mô-đun, có thể phát triển và chạy độc lập, giúp hệ thống linh hoạt và dễ mở rộng.

Topics

Các kênh giao tiếp riêng lẻ mà các node sử dụng để liên lạc với nhau được gọi là topic. Một topic được sử dụng cho giao tiếp một chiều, nhiều-nhiều. Mỗi topic có một kiểu thông điệp (message type) xác định cấu trúc dữ liệu được truyền đi. Một topic có thể có nhiều node cùng publish, và nhiều node cùng subscribe. ROS Master hỗ trợ thiết lập kết nối ban đầu giữa các publisher và subscriber.

- Publisher - Là các node tạo ra dữ liệu. Một node có thể publish một loại thông điệp cụ thể lên một topic và cho phép các node khác truy cập dữ liệu đó.
- Subscriber - Là các node nhận dữ liệu. Một node có thể subscribe để nhận các thông điệp được publish trên một topic.

Topics được sử dụng để publish và subscribe các dữ liệu được cập nhật liên tục, bao gồm dữ liệu cảm biến và trạng thái của robot.



Hình 3. 11 Mô hình giao tiếp giữa các node trong qua topics

Messages

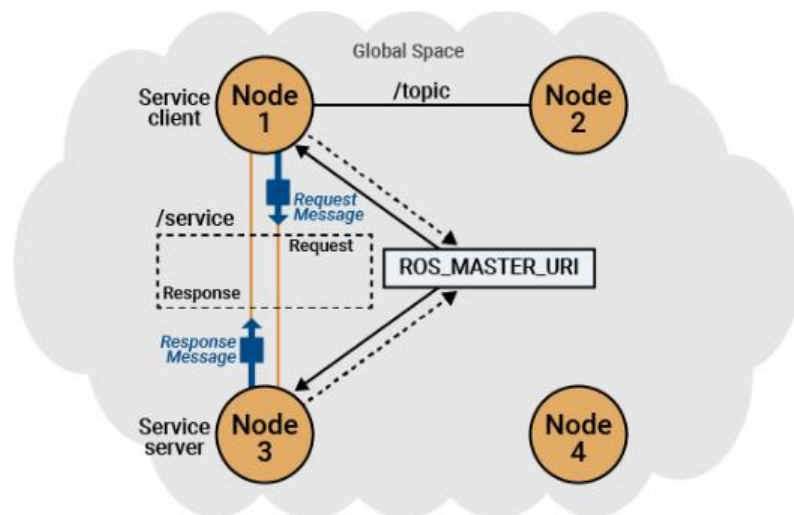
Messages là đơn vị dữ liệu trao đổi giữa các node, được định nghĩa trong file .msg với các trường dữ liệu và hằng số. Khi node publish lên topic, nó gửi một thể hiện của kiểu message đó. Các node subscriber nhận và xử lý thông điệp này. Messages có thể chứa dữ liệu như cảm biến, lệnh điều khiển, hình ảnh, v.v.

Services

Services là cách các node giao tiếp theo kiểu đồng bộ, sử dụng mô hình yêu cầu – phản hồi (request–response). Khác với topics truyền dữ liệu liên tục, services dùng cho các tác vụ riêng lẻ, cần phản hồi ngay, như ra lệnh robot di chuyển đến một vị trí. Thông điệp service được định nghĩa trong file .srv, gồm hai phần, 1 request và 1 response:

- Client - Là node gửi yêu cầu (request) thực hiện một tác vụ .
- Server - Là node tiếp nhận yêu cầu và thực hiện phép tính cần thiết và gửi lại (response)

Services thường dùng cho các lệnh nhanh, như truy vấn trạng thái, tính toán, chuyển đổi dữ liệu, v.v.

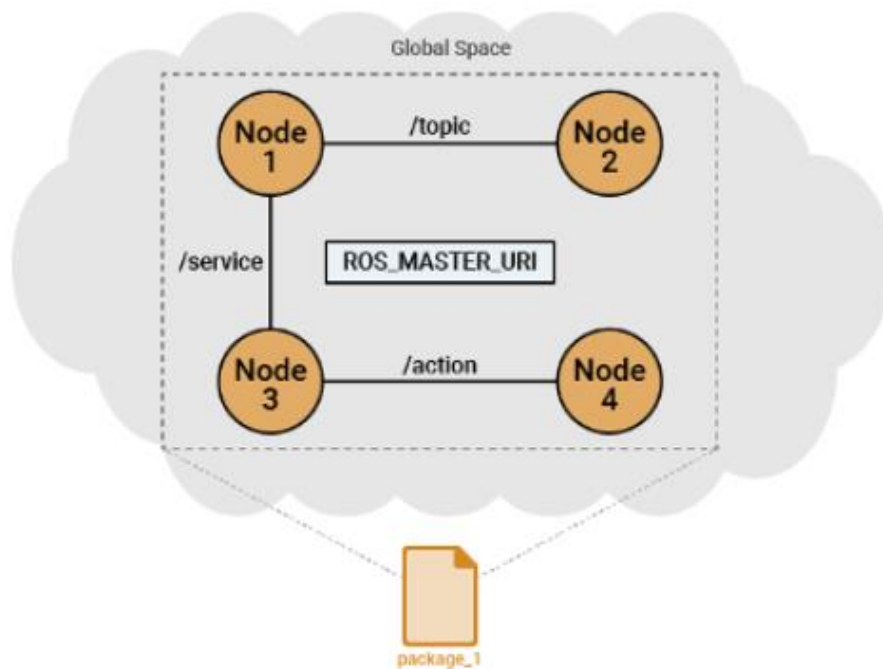


Hình 3. 12 Mô hình giao tiếp giữa các node trong ROS thông qua Service

Packages

Packages là đơn vị tổ chức phần mềm chính trong ROS, chứa mã nguồn, tập lệnh biên dịch và thông tin mô tả. Mỗi package có thể gồm các node, message,

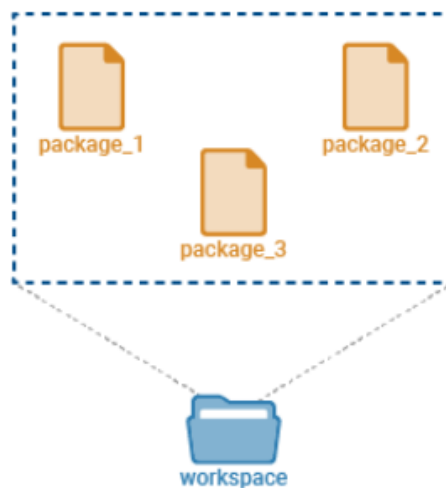
service, action, thư viện và file cấu hình. Packages được thiết kế để dễ tái sử dụng, chia sẻ và được quản lý bằng công cụ như catkin hoặc rosbuild.



Hình 3. 13 Mối liên hệ giữa các node trong một ROS package

Workspace

Workspace là thư mục dùng để chứa và tổ chức các package trong ROS. Một workspace có thể bao gồm nhiều package, mỗi package nằm trong thư mục riêng, theo cấu trúc rõ ràng và hiệu quả.



Hình 3. 14 Cấu trúc của workspace trong ROS

Khi nắm vững các khái niệm cơ bản bây giờ có thể ứng dụng nó để phát triển và quản lý robot trên ROS. Xây dựng mô hình robot và mô phỏng nó trong môi trường ROS và sử dụng các công cụ như Gazebo, Rviz... Trực quan hóa chuyển động, cảm biến và logic điều khiển.

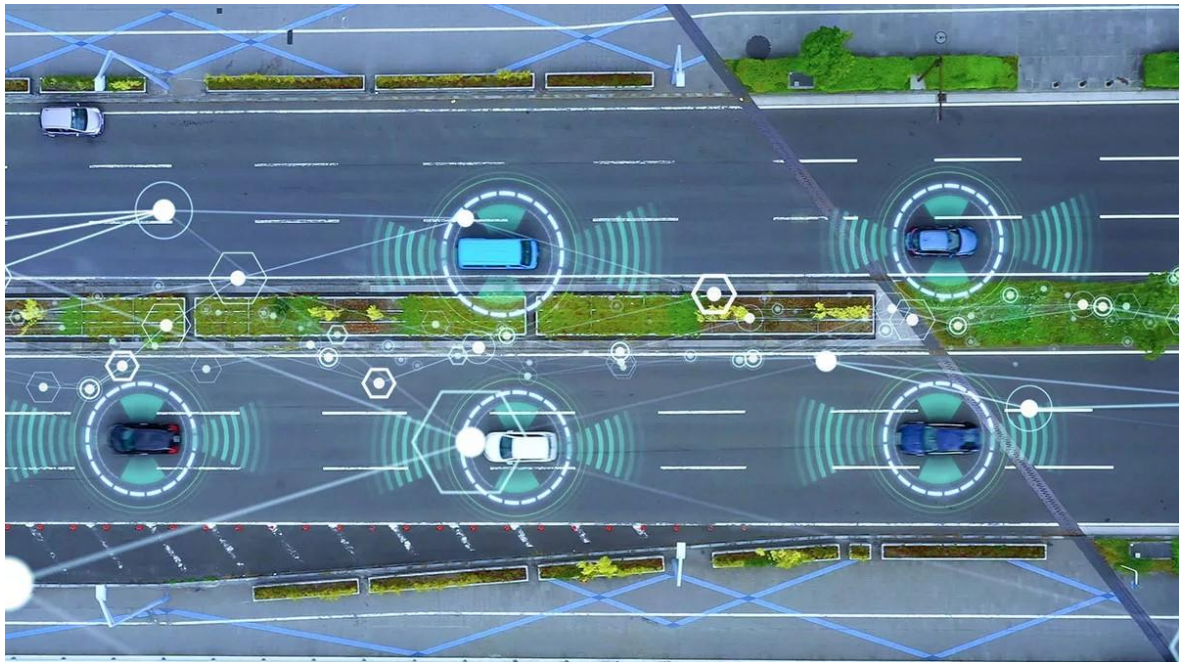
Cuối cùng ứng dụng toàn bộ kiến thức đã học để thực hiện một bài toán thực tế phát triển robot robot.

3.3 Công nghệ LiDAR

3.3.1 Định nghĩa

LiDAR (viết tắt của Light Detection and Ranging) là một công nghệ cảm biến đo khoảng cách bằng cách chiếu tia laser đến một đối tượng và đo thời gian phản xạ của tia sáng trở lại cảm biến. Dựa vào tốc độ ánh sáng và thời gian phản xạ, hệ thống sẽ tính được khoảng cách đến vật thể một cách chính xác.

LiDAR là công nghệ cảm biến quang học được đánh giá cao trong việc đo khoảng cách, đặc biệt trong các hệ thống tự hành. Nhiều hãng đang phát triển các giải pháp LiDAR nhỏ gọn, tiết kiệm chi phí. Công nghệ này được xem là yếu tố then chốt trong lĩnh vực xe tự lái và đã bắt đầu được ứng dụng trong các hệ thống hỗ trợ lái nâng cao (ADAS). [7]

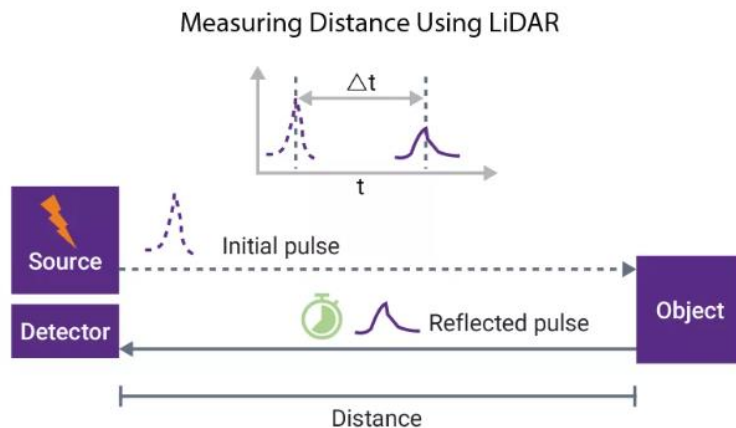


Hình 3. 15 Tổng quan công nghệ LiDAR ứng dụng trong hệ thống ADAS

3.3.2 Nguyên tắc hoạt động của LiDAR

LiDAR hoạt động dựa trên nguyên lý phát xạ tia laser, các chùm tia laser được phát đi để quét không gian xung quanh và phản xạ về của tia này từ vật thể trở lại để tính khoảng cách. [7]

SYNOPSYS®



Hình 3. 16 Nguyên tắc tính toán khoảng cách bằng cách sử dụng LiDAR

Trên hình là phương pháp Time-of-Flight (ToF) sử dụng để đo khoảng cách trong lidar. Cụ thể như sau:

- Bộ phát (Source) gửi một xung laser ban đầu (Initial pulse) về phía vật thể (Object).
- Tia laser va chạm vào vật thể và bị phản xạ ngược trở lại.
- Bộ thu (Detector) nhận xung phản xạ (Reflected pulse).
- Đo thời gian Δt : thời gian trễ giữa lúc phát và lúc thu.
- Tính khoảng cách đến vật thể theo công thức:

$$\text{distance} = \frac{c \times \Delta t}{2}$$

Trong đó: c là tốc độ ánh sáng ($\sim 300.000.000$ m/s)

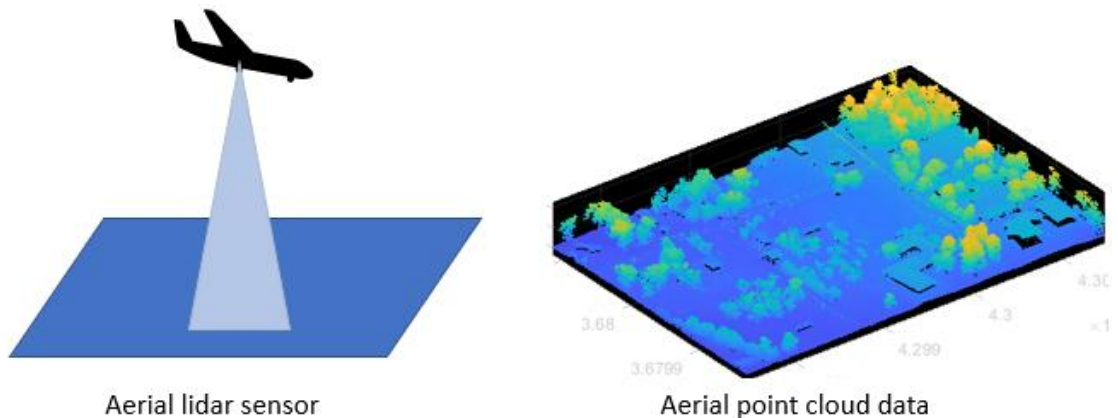
Δt : thời gian từ lúc phát đến lúc thu.

Chia đôi vì tia đi và về.

3.3.3 Các loại lidar

Airborne LiDAR (Lidar trên không): là hệ thống cảm biến LiDAR gắn trên các phương tiện bay như máy bay, trực thăng hoặc UAV (drone). Chức năng quét mặt đất từ trên cao để tạo bản đồ địa hình, rừng, hạ tầng... Loại LiDAR này lại được chia 2 nhánh nhỏ đó là:

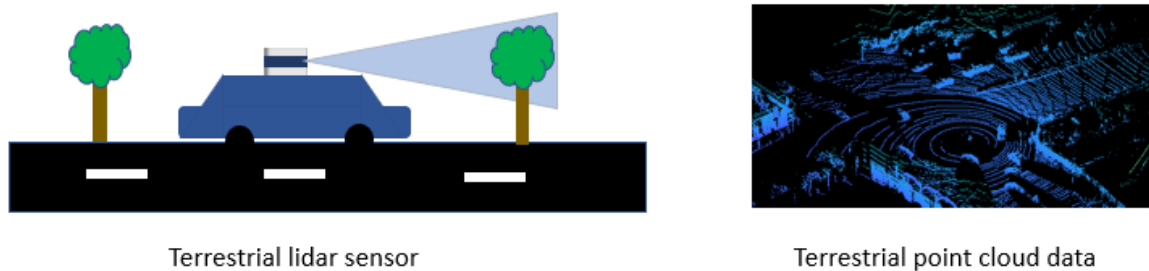
- Topographic LiDAR: dùng ánh sáng hồng ngoại → đo địa hình mặt đất
- Bathymetric LiDAR: dùng ánh sáng xanh → đo đáy nước (biển, sông)



Hình 3. 17 Hoạt động Airborne LiDAR và bản đồ tái tạo

Terrestrial LiDAR (lidar mặt đất): là cảm biến quét laser được đặt trên mặt đất, dùng để thu thập dữ liệu về bề mặt Trái Đất hoặc môi trường xung quanh cảm biến. lidar này cũng chia làm 2 loại nhỏ đó là:

- Static Terrestrial LiDAR (Cố định): Cảm biến được gắn cố định tại một vị trí, quét dữ liệu từ đó.
- Mobile Terrestrial LiDAR (Di động): Cảm biến được gắn trên các phương tiện di chuyển như ô tô, robot.



Hình 3. 18 Hoạt động Terrestrial LiDAR và bản đồ tái tạo

3.4 Thuật toán Dijkstra

Thuật toán Dijkstra là một thuật toán tìm đường ngắn nhất trong đồ thị có trọng số không âm. Được phát triển bởi nhà khoa học máy tính người Hà Lan Edsger W. Dijkstra vào năm 1956, đây là một trong những thuật toán nền tảng và được sử dụng rộng rãi trong các bài toán tìm đường như: hệ thống GPS, mạng máy tính, điều hướng robot, và tối ưu hóa lộ trình trong logistics.

3.4.1 Nguyên lý hoạt động [8]

Dijkstra là thuật toán tìm kiếm theo chi phí thấp nhất, hoạt động bằng cách mở rộng dần các đỉnh gần nguồn nhất, theo nguyên tắc tham lam (greedy):

- Đặt khoảng cách ban đầu cho tất cả các đỉnh: 0 cho đỉnh nguồn và vô cực cho tất cả các đỉnh còn lại.
- Chọn đỉnh chưa được thăm có khoảng cách ngắn nhất từ điểm bắt đầu làm đỉnh hiện tại. Vì vậy, thuật toán sẽ luôn bắt đầu với đỉnh hiện tại là đỉnh nguồn.
- Đối với mỗi đỉnh lân cận chưa được ghé thăm của đỉnh hiện tại, hãy tính khoảng cách từ đỉnh nguồn và cập nhật khoảng cách nếu khoảng cách mới được tính toán thấp hơn.
- Bây giờ chúng ta đã hoàn thành đỉnh hiện tại, vì vậy chúng ta đánh dấu nó là đã ghé thăm. Một đỉnh đã ghé thăm sẽ không được kiểm tra lại.
- Quay lại bước 2 để chọn đỉnh hiện tại mới và tiếp tục lặp lại các bước này cho đến khi thăm hết tất cả các đỉnh.
- Cuối cùng, chúng ta còn lại đường đi ngắn nhất từ đỉnh nguồn đến mọi đỉnh khác trong đồ thị.

3.4.2 Các bước hoạt động của thuật toán Dijkstra

Bước 1: Khởi tạo:

- Gán cho tất cả các đỉnh giá trị khoảng cách là ∞ , riêng đỉnh xuất phát có khoảng cách là 0.
- Tạo một tập hợp chứa tất cả các đỉnh chưa được xét.

Bước 2: Lặp lại cho đến khi tập hợp rỗng:

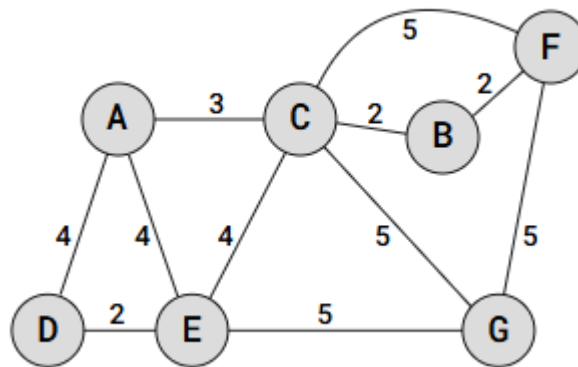
- Chọn đỉnh có khoảng cách nhỏ nhất trong tập hợp chưa xét.
- Xét tất cả các đỉnh kề chưa xét của nó.
- Nếu tìm được đường đi ngắn hơn, cập nhật khoảng cách của đỉnh kề đó.

Bước 3: Kết thúc:

- Khi tất cả các đỉnh đã được xét, thuật toán trả về khoảng cách ngắn nhất từ điểm bắt đầu đến các đỉnh còn lại.

Ví dụ minh họa

Giả sử chúng ta có một đồ thị đơn giản với các đỉnh và trọng số như sau:



Hình 3. 19 Đồ thị minh họa đồ thị đơn giản với các đỉnh và trọng số [8]

Mục tiêu: Tìm đường đi ngắn nhất từ đỉnh A đến tất cả các đỉnh khác

Khởi tạo:

- Khoảng cách: $A = 0, B = \infty, C = \infty, D = \infty, E = \infty, F = \infty, G = \infty$
- Đỉnh trước đó: $A = \text{Không có}, B = \text{Không có}, C = \text{Không có}, D = \text{Không có}, E = \text{Không có}, F = \text{Không có}, G = \text{Không có}$
- Đã thăm: $\{\}$
- Hàng đợi ưu tiên: $[(0, A)]$

Xét các nút tính khoảng cách tới các nút liền kề

- Đánh dấu A là đã ghé thăm (thêm A vào visited).
- Cập nhật khoảng cách của các hàng xóm của A:

- **A -> C:** Khoảng cách hiện tại của A (0) + trọng số cạnh AC (3) = 3. Vì $3 < \infty$ (khoảng cách hiện tại của C), cập nhật $\text{distances}[C] = 3$, $\text{predecessors}[C] = A$.
- **A -> D:** Khoảng cách hiện tại của A (0) + trọng số cạnh AD (4) = 4. Vì $4 < \infty$ (khoảng cách hiện tại của D), cập nhật $\text{distances}[D] = 4$, $\text{predecessors}[D] = A$.
- **A -> E:** Khoảng cách hiện tại của A (0) + trọng số cạnh AE (4) = 4. Vì $4 < \infty$ (khoảng cách hiện tại của E), cập nhật $\text{distances}[E] = 4$, $\text{predecessors}[E] = A$.
- Cập nhật lại bảng khoảng cách: Khoảng cách: A = 0, B = ∞ , C = 3, D = 4, E = 4, F = ∞ , G = ∞
- Lặp lại tương tự chu trình này với các nút chưa ghé thăm cho đến khi tất cả các nút được ghé thăm.

Kết thúc

Khi đã lặp qua tất cả các nút ta có được bảng khoảng cách ngắn nhất từ A đến các đỉnh: A: 0, B: 5 (từ A), C: 3 (từ A), D: 4 (từ A), E: 4 (từ A), F: 7 (từ A), G: 8 (từ A).

3.4.1 Ưu điểm và nhược điểm

Ưu điểm:

Chính xác tuyệt đối: Tìm được đường đi ngắn nhất từ một đỉnh đến tất cả các đỉnh khác trong đồ thị (với trọng số không âm).

Ổn định và đáng tin cậy: Thuật toán không bị phụ thuộc vào hàm ước lượng như A^* , do đó kết quả luôn chắc chắn.

Nhược điểm:

Không xử lý được trọng số âm: Thuật toán không hoạt động nếu đồ thị có cạnh mang trọng số âm.

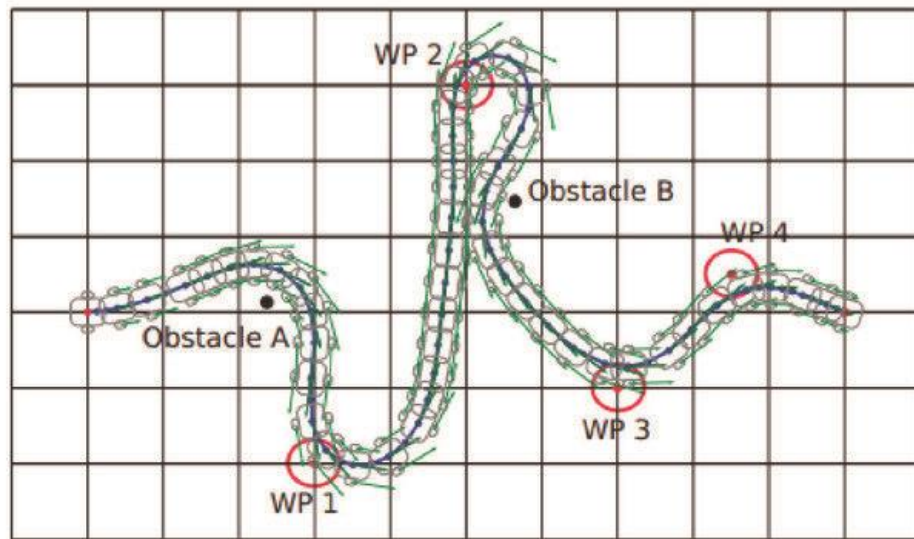
Không định hướng ưu tiên: Không sử dụng thông tin dự đoán (heuristic), nên có thể mở rộng nhiều nút không cần thiết, làm chậm quá trình tìm kiếm hơn so với A^* .

3.5 Thuật toán TED local planner

Thuật toán TEB Local Planner (Time Elastic Band Local Planner) là thuật toán lập kế hoạch chuyển động cục bộ (local planning) rất phổ biến trong ROS (Robot Operating System). Giúp robot di chuyển theo quỹ đạo ngắn gọn, an toàn và hợp lý từ vị trí hiện tại đến mục tiêu, trong khi tránh chướng ngại vật một cách linh hoạt trong thời gian thực.

3.5.1 Nguyên lý hoạt động

TEB xem đường đi như một chuỗi các điểm (poses) được nối lại bằng các đoạn thẳng, gọi là "dải đàn hồi thời gian" (elastic band). [9]



Hình 3. 20 Mô hình Time-Elastic Band trong lập kế hoạch quỹ đạo [9]

Các điểm trong dải này có thể được biến dạng không gian và thời gian để tối ưu:

- Không gian: Tránh vật cản, duy trì quỹ đạo trơn tru.
- Thời gian: Tối ưu vận tốc và tăng khả năng phản ứng.

TEB sử dụng tối ưu hóa phi tuyến để cập nhật các điểm theo các ràng buộc như:

- Tránh va chạm
- Giới hạn tốc độ
- Ràng buộc động học của robot

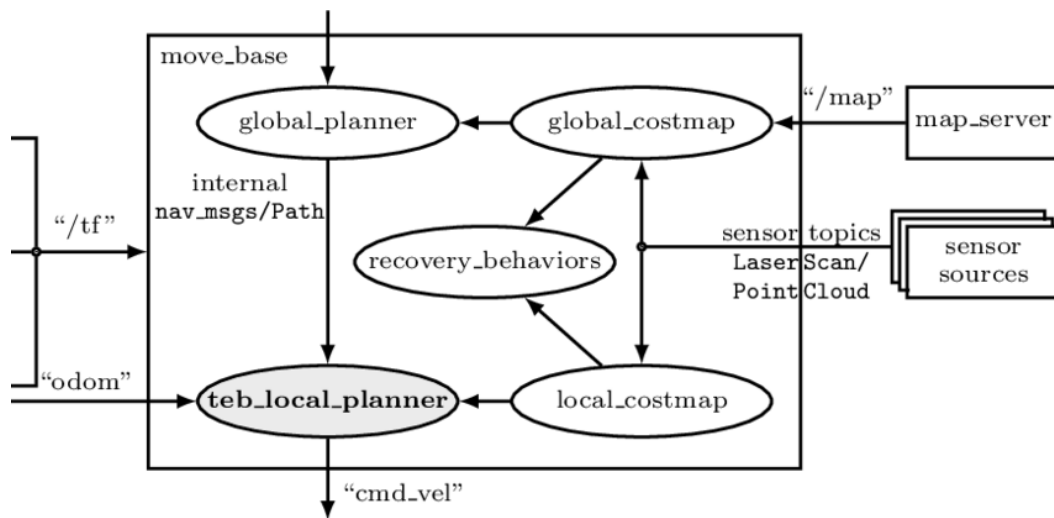
3.5.2 Cấu trúc TED local planner trong ROS

TEB thường nằm trong thành phần **local planner** của ROS Navigation Stack.

Cụ thể:

- Global Planner (ví dụ A*) tạo đường đi tổng thể.
- TEB Local Planner tối ưu hóa đoạn đường cục bộ gần vị trí robot.
- Kết quả là một tập các vận tốc điều khiển (linear, angular velocity) được gửi tới robot.

TEB sử dụng gói ROS `teb_local_planner` và có thể cấu hình bằng file `.yaml`.



Hình 3. 21 Sơ đồ tích hợp TEB vào ROS Navigation Stack [10]

3.5.3 Các ràng buộc tối ưu trong TED

TEB thực hiện tối ưu hóa theo nhiều tiêu chí:

- Tránh va chạm (chướng ngại vật động & tĩnh)
- Ràng buộc vận tốc (tốc độ tối đa, gia tốc, góc quay)
- Ràng buộc động học (ví dụ robot không thể đi lùi hoặc quay tức thời)
- Tối ưu độ cong quỹ đạo để tránh robot rung giật
- Tối ưu thời gian di chuyển

Chương 4. THIẾT KẾ PHẦN CỨNG VÀ PHẦN MỀM

4.1 Thiết kế phần cứng

Ý tưởng và mục tiêu thiết kế robot di động có khả năng di chuyển và thực hiện nhiệm vụ giao thuốc trong bệnh viện. Robot có khả năng di chuyển trong môi trường đặc thù với địa hình bằng phẳng, không gian hạn chế nhiều vật cản và người qua lại.

4.1.1 Sơ lược về phần mềm thiết kế kỹ thuật

SolidWork 2023 là phần mềm được chúng tôi lựa chọn để thiết kế mô hình hóa robot di động cho đề án này.

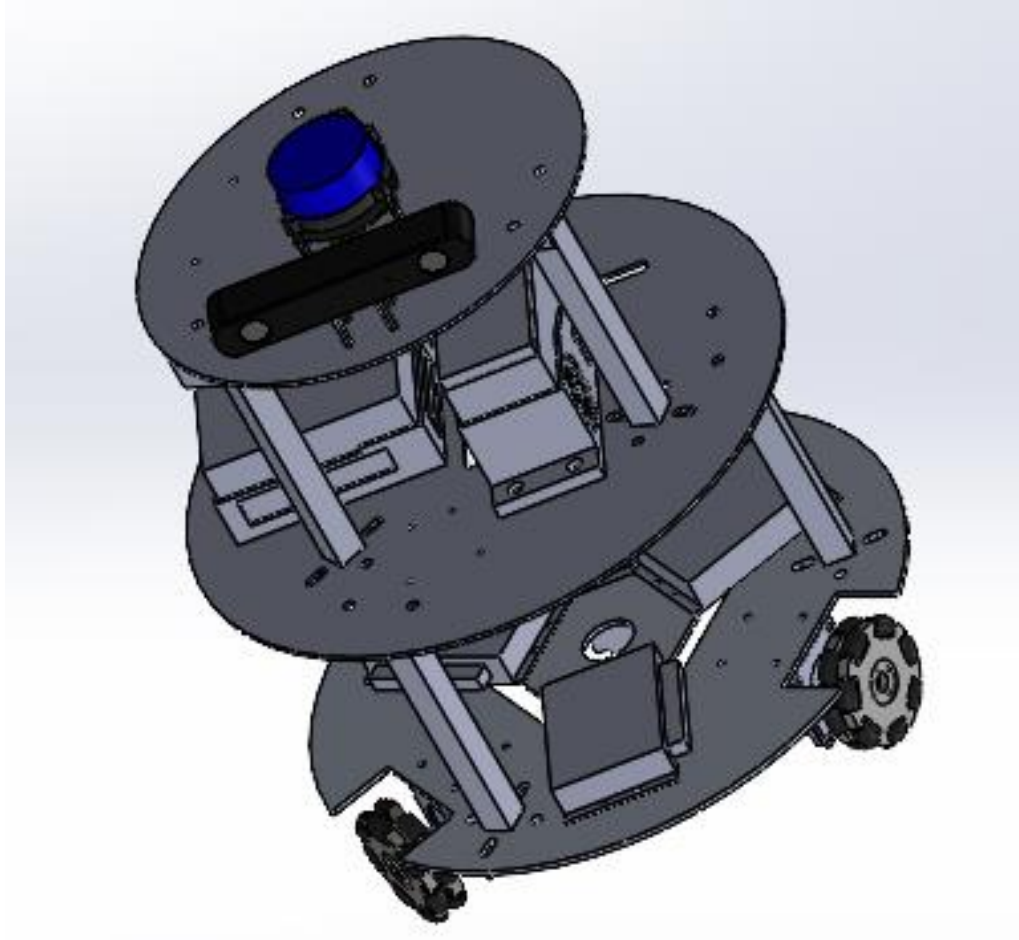


Hình 4. 1 Phần mềm thiết kế 3D SolidWork 2023

SolidWork là 1 phần mềm thiết kế 3D chuyên dụng chạy trên hệ điều hành Windows. Được sử dụng rộng rãi trong lĩnh vực thiết kế cơ khí nhờ khả năng mô phỏng trực quan, dễ sử dụng và hỗ trợ thiết kế lắp ráp hiệu quả.. Giống như các phần mềm thiết kế khác SolidWork cung cấp cho người dùng những tính năng tuyệt vời nhất để thiết kế các chi tiết của khối 3D, lắp ráp các chi tiết lại với nhau để tạo thành sản phẩm hoàn chỉnh và xuất bản vẽ kỹ thuật 2D phục vụ chế tạo.

4.1.2 Kích thước và hình dáng tổng thể của robot

Khung cơ cấu robot dạng hình trụ tròn với 2 tầng đối xứng, được ghép lại từ 3 miếng để tròn kết nối với nhau bằng các trụ nhôm.



Hình 4. 2 Bản vẽ thiết kế 3D tổng thể của robot

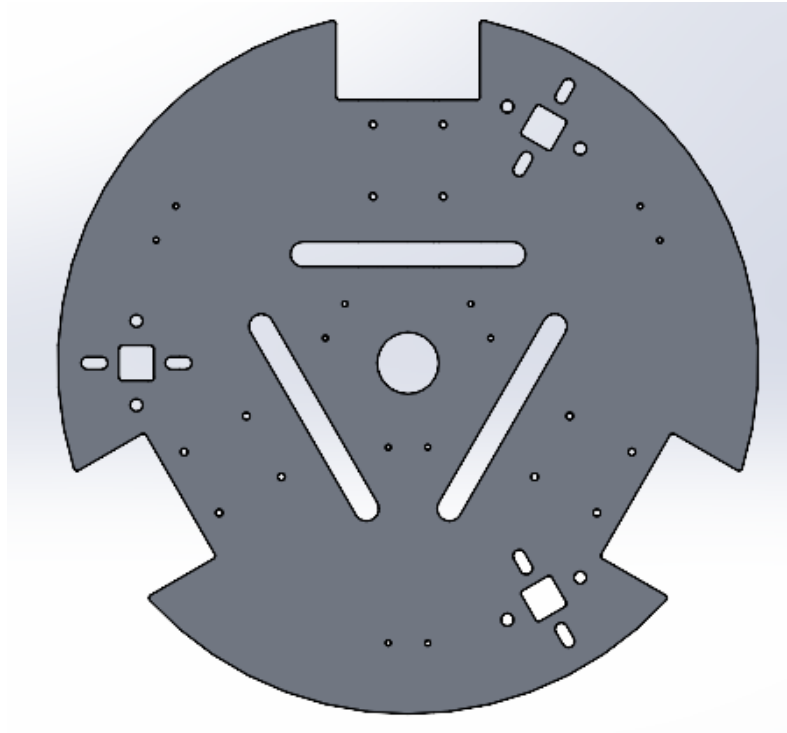
Kích thước: đường kính của robot là 50cm, chiều cao tổng thể ~ 56cm

Vật liệu khung: được cấu thành từ 2 vật liệu chính là nhôm và mica đảm bảo robot không cong kênh, nặng nề nhưng chắc chắn và chịu được những tác động cho phép.

4.1.3 Các thành phần cơ khí robot

❖ Đế robot di động 3 bánh omni (base plate)

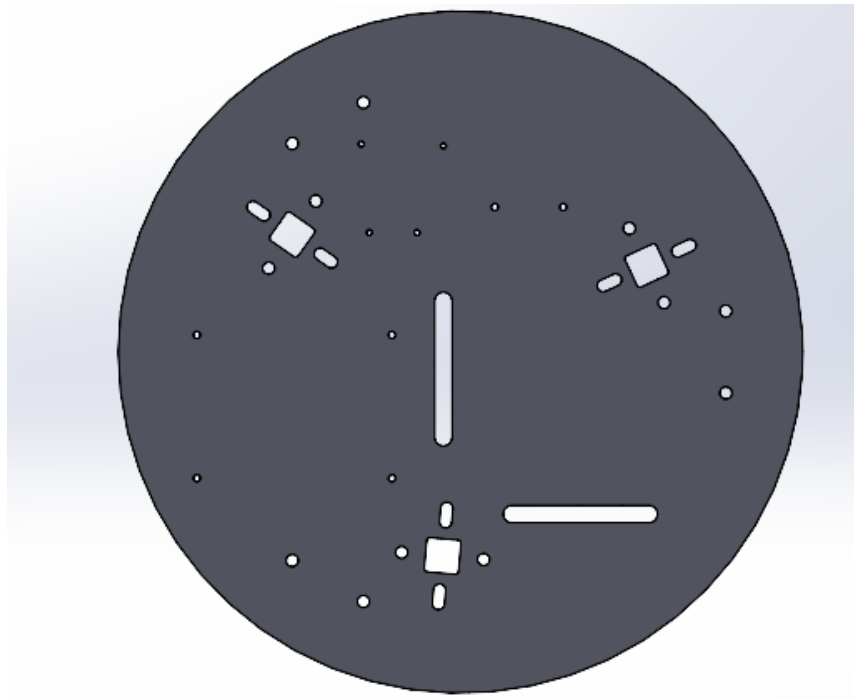
- Vật liệu: nhôm
- Hình dạng và độ dày: tròn đường kính 50cmx3mm



Hình 4. 3 Bản vẽ 3D đế robot di động 3 bánh omni

❖ **Đế tầng 2 robot di động 3 bánh omni (middle plate)**

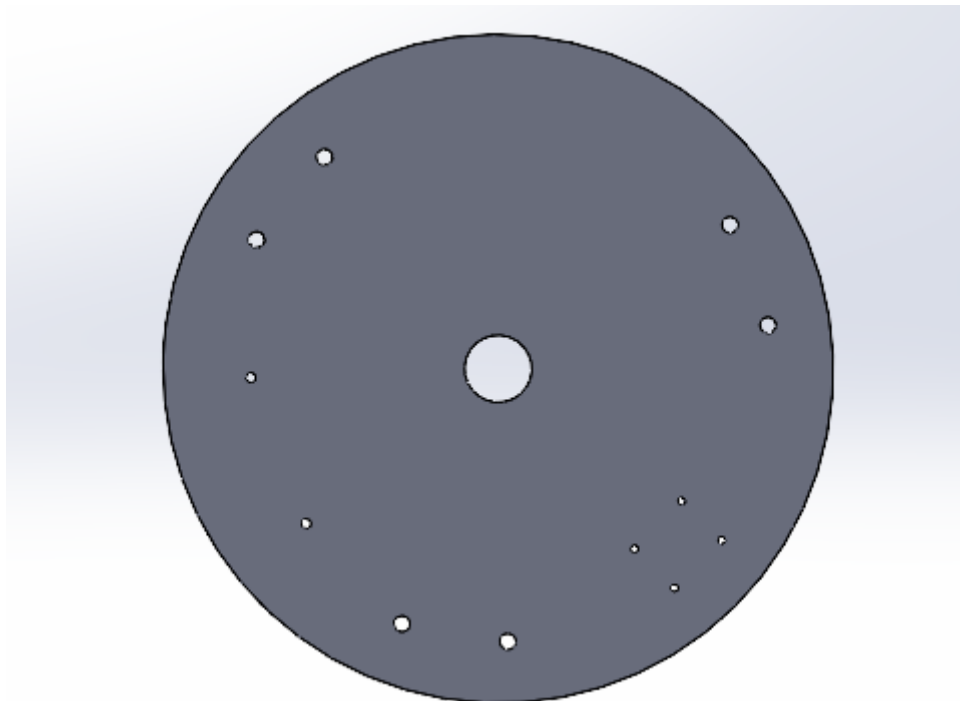
- Vật liệu: mica trong suốt
- Hình dạng và độ dày : tròn đường kính 50cmx4mm



Hình 4. 4 Bản vẽ 3D đế robot tầng 2 di động 3 bánh omni

❖ **Đế trên cùng (top plate)**

- Vật liệu: nhôm
- Hình dạng và độ dày: tròn đường kính 30cmx4mm



Hình 4. 5 Bản vẽ 3D đế trên cùng robot di động 3 bánh omni

❖ **Trụ đỡ các đế**

- Vật liệu nhôm: Nhôm
- Kích thước: hình trụ 2x2cm, dài 20cm



Hình 4. 6 Nhôm định hình

❖ **Bánh xe đa hướng omni**

- Vật liệu: nhôm
- Kích thước: đường kính 82mm, độ dày 21mm
- 7 con lăn phụ (rollers) mỗi bên



Hình 4. 7 Bánh xe đa hướng omni

❖ **Động cơ bước vòng kín 2 pha 57mm YAKO**



Hình 4. 8 Động cơ bước vòng kín 2 pha YAKO

Thông số kỹ thuật:

- Độ chính xác góc bước: 5% (mỗi bước có sai số tối đa 5%)
- Góc bước: $1,8^\circ$
- Có tích hợp encoder

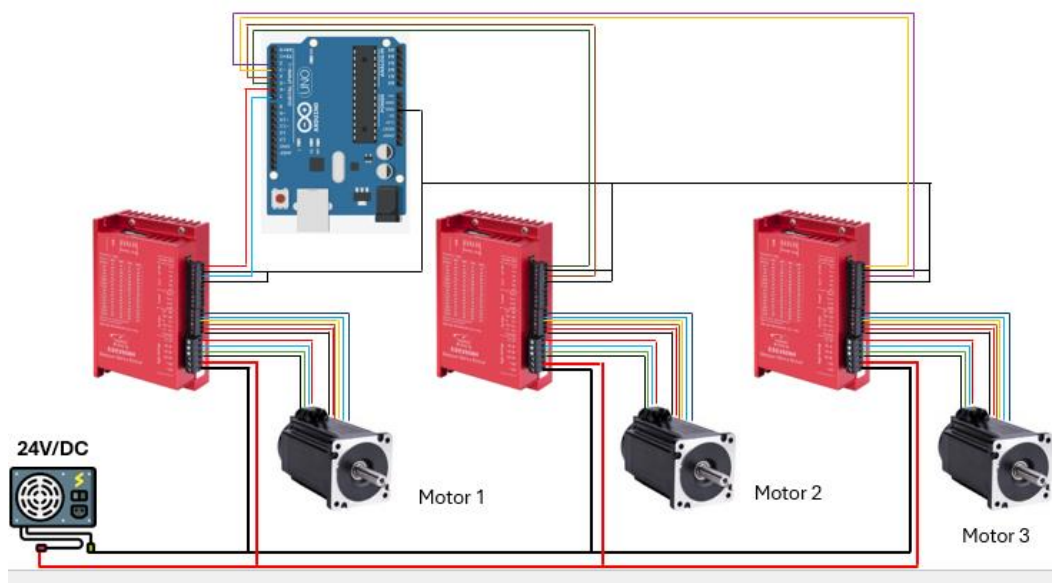
- Điện áp hoạt động 24-50VDC
- Dòng điện định mức: 3 - 5A / pha
- Nhiệt độ hoạt động bình thường: -20°C đến +50°C

4.1.4 Các thành phần điện tử của robot

Hệ thống điện – điện tử của robot được thiết kế theo mô hình điều khiển hai lớp phân tầng: lớp điều khiển dưới (low-level) và lớp điều khiển trên (high-level), nhằm tối ưu hóa khả năng xử lý, dễ dàng mở rộng chức năng và giảm tải cho vi điều khiển.

Lớp điều khiển dưới đóng vai trò điều khiển thời gian thực các phần cứng gồm các 2 thành phần điều khiển chính:

vi điều khiển Arduino Uno, 3 driver ESD2505M điều khiển 3 động cơ bước 2 pha. Nguồn di động 24V/DC là nguồn chính cho 3 động cơ và driver. Nguồn phụ pin dự phòng 5V cấp cho adruino uno điều khiển.



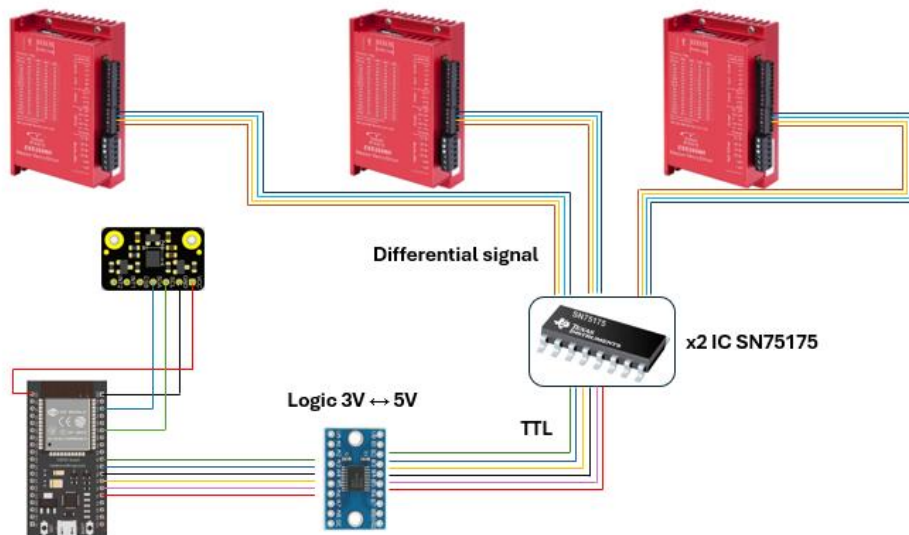
Hình 4. 9 Sơ đồ mạch điện điều khiển động cơ

Giải thích sơ đồ

- Nối 2 pha của từng động cơ và từng driver.
- Cấp nguồn 24V/DC cho 3 driver mắc song song.
- Nối dây encoder từng động cơ vào cổng điều khiển encoder ở từng driver.

- Kết nối chân điều khiển xung và hướng của driver vào arduino để điều khiển.

Thành phần điều khiển thứ hai vì điều khiển Esp32 đọc encoder của 3 động cơ. Qua IC SN75157 để chuyển tín hiệu vi sai thành tín hiệu TTL và qua mạch TXS0108 để chuyển mức logic 5V từ driver về 3V3 cho esp32. Và đọc cảm biến gia tốc



Hình 4. 10 Sơ đồ mạch điều khiển đọc encoder động cơ

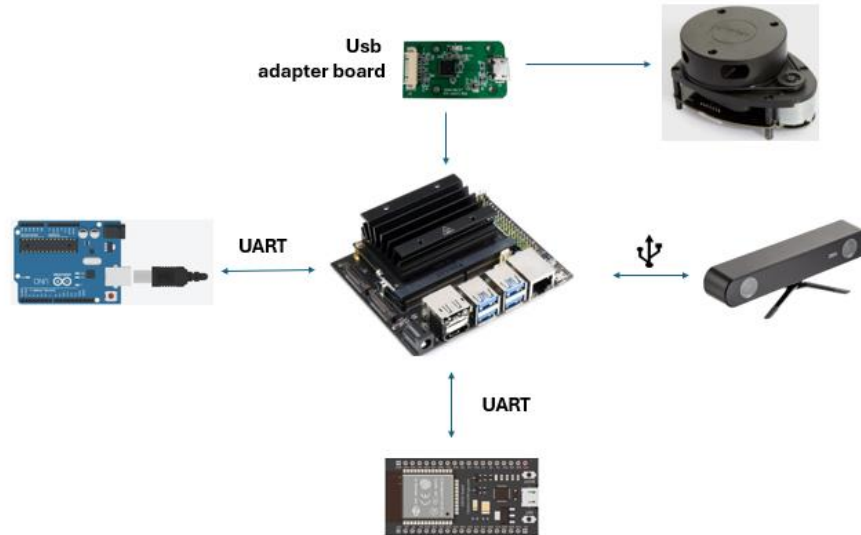
Giải thích sơ đồ

- Nối dây từ 2 pha A+, B+, A-, B- từ 3 driver sang 2 IC SN75157 (1 IC đọc được 2 driver) để chuyển tín hiệu vi sai → TTL.
- Tín hiệu chuyển thành TTL sẽ tiếp tục nối qua module chuyển mức logic từ 5V → 3,3 V.
- Tín hiệu TTL chuyển sang 3,3V rồi sẽ được nối trực tiếp vào esp32 để đọc tín hiệu encoder.
- Cuối cùng là đọc cảm biến IMU bằng cách kết nối với ESP32 qua I2C (SCL, SDA) và cấp nguồn bằng VCC, GND.

Lớp điều khiển trên sử dụng nền tảng Jetson Nano để xử lý các tác vụ tính toán phức tạp. Bao gồm:

- Phân tích dữ liệu từ cảm biến
- Nhận tín hiệu đọc encoder động từ esp32 về xử lý.

- Giao tiếp với arduino điều khiển động cơ, tạo ra các node quản lí
- Chạy hệ điều hành ROS cung cấp thư viện mạnh mẽ cho lidar để quét không gian xung quanh, tạo bản đồ môi trường và hỗ trợ điều hướng



Hình 4. 11 Sơ đồ mạch điện điều khiển lớp trên của robot

Các thành phần điện, điện tử của robot

❖ NVIDIA Jetson nano develop kit



Hình 4. 12 Máy tính nhúng Jetson nano

Thông số kĩ thuật chính:

- CPU: Quad-core ARM Cortex-A57

- GPU: NVIDIA Maxwell với 128 nhân CUDA → giúp xử lý các tác vụ AI, học sâu nhanh và hiệu quả
- RAM: 4GB LPDDR
- Lưu trữ: sử dụng thẻ microSD (hoặc có thể mở rộng qua USB/SATA)
- Cổng kết nối: HDMI, USB 3.0, GPIO, Camera CSI, Ethernet, v.v.
- Hệ điều hành: Ubuntu-based Linux (JetPack SDK của NVIDIA)

❖ **ESD2505M stepper servo driver**



Hình 4. 13 Driver động cơ bước encoder vòng kín ESD2505M

Thông số kỹ thuật chính:

- Thiết kế để điều khiển chính xác động cơ bước hai pha
- Điện áp hoạt động: 24-50V/DC
- Tích hợp điều khiển Encoder vòng kín
- Xung điều khiển: PULSE/DIRECTION & CW/CCW
- 16 mức xung khác nhau để lựa chọn, lên tới 40,000 xung/vòng

❖ Rplidar A1M8



Hình 4. 14 Rplidar A1M8

Thông số kỹ thuật chính:

- Phạm vi đo: 0,15 – 12 m (điều kiện lí tưởng)
- Tần số quét: 5 – 10 Hz (tùy chỉnh)
- Tốc độ lấy mẫu: lên đến 8000 mẫu/giây (thông thường là 4000)
- Dòng điện hoạt động: 100mA
- Điện áp hoạt động: 5V/DC
- Giao tiếp: UART hoặc USB (thường dùng usb adapter board đi kèm)

❖ Pin dự phòng



Hình 4. 15 Pin dự phòng

Thông số kỹ thuật

- Nguồn cấp tối đa: 5V/DC
- Công suất: 20W

❖ **Nguồn động lực**

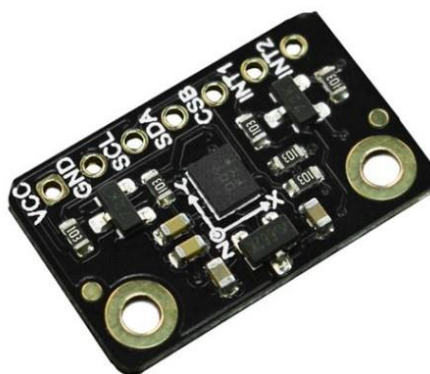


Hình 4. 16 Nguồn động lực

Thông số kỹ thuật

- Nguồn cấp tối đa: 24V/DC
- Dòng cấp tối đa: 10A
- Loại pin: lithium

❖ **Cảm biến gia tốc BMX190-9 axis**



Hình 4. 17 Cảm biến gia tốc BMX190

Cảm biến gia tốc BMX160 9-axis Sensor (Breakout) sử dụng cảm biến từ Bosch Sensortec có khả năng đo các thông số gia tốc kế, con quay hồi chuyển, từ trường với độ chính xác cao, mức tiêu thụ điện năng của cảm biến dưới 1.5mA.

Thông số kỹ thuật:

- Điện áp hoạt động: 3.3V đến 5.5V
- Giao tiếp: I2C (chuẩn 7-bit địa chỉ 0x68 hoặc 0x69)
- Đầu nối: 2.54mm Single Row 7 chân

❖ Vi điều khiển ESP32

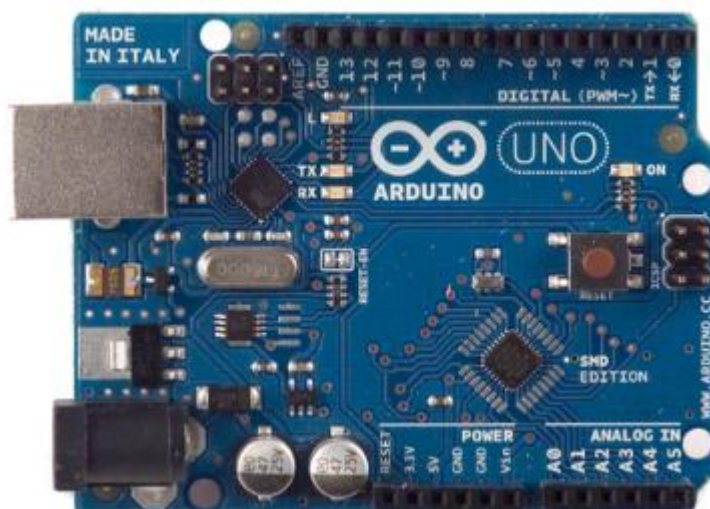


Hình 4. 18 Vi điều khiển ESP32

Thông số kỹ thuật:

- CPU: Tensilica Xtensa LX6 Dual-core (2 nhân)
- Flash: 4MB
- Điện áp nguồn (USB): 5V DC
- Đầu vào/Đầu ra điện áp: 3.3V DC
- Giao tiếp: USART, SPI, I2C, CAN, USB 2.0 (thiết bị)
- Timer: 2 bộ định thời
- ADC: 12bit SAR loại ADC, 18 kênh
- GPIO: 24 chân vào/ra

❖ Vi điều khiển Arduino uno r3



Hình 4. 19 Vi điều khiển Arduino uno r3

Thông số kỹ thuật:

- Vi điều khiển: ATmega328P
- Tốc độ: 16 MHz
- Điện áp hoạt động: 5V
- Nguồn vào: 7–12V (qua jack DC)
- Digital I/O: 14 chân (6 chân PWM)
- Analog input: 6 chân (A0–A5)
- Giao tiếp: UART, SPI, I2C

❖ IC SN75175



Hình 4. 20 IC SN75175

SN75175 là một bộ thu truyền vi sai thường được sử dụng trong các hệ thống truyền thông nối tiếp RS-485 hoặc RS-422 – các chuẩn giao tiếp phổ biến trong công nghiệp và nhúng.

Thông số kỹ thuật

- Chuẩn giao tiếp: RS-485, RS-422
- Số kênh nhận: 4 kênh độc lập
- Điện áp hoạt động: 4.75-5.25V
- Đầu ra: TTL/CMOS-compatible

❖ **Mạch Chuyển Đổi Mức Logic TXS0108**



Hình 4. 21 Mạch TXS0108

Thông số kỹ thuật:

- Số kênh: 8 kênh
- Chuyển đổi 2 chiều : tự động phát hiện chiều dữ liệu
- Điện áp hoạt động: VCCA: 1.2V – 3.6V, VCCB: 1.65V – 5.5V
- Tốc độ truyền: 110 Mbps

4.1.5 Yêu cầu và các tiêu chuẩn an toàn

Môi trường bệnh viện có đặc thù hành lang hẹp, mật độ người qua lại cao và nhiều thiết bị y tế cần bảo vệ, do đó việc đảm bảo an toàn khi robot vận hành là yếu tố bắt buộc. Trong quá trình nghiên cứu, nhóm tham khảo các tiêu chuẩn an toàn quốc tế sau:

ISO 13482: Quy định yêu cầu an toàn cho robot dịch vụ cá nhân, trong đó bao gồm:

- Giới hạn tốc độ và gia tốc khi di chuyển gần con người.
- Giới hạn lực tiếp xúc và áp suất để tránh gây chấn thương.
- Xác định vùng nguy hiểm và duy trì khoảng cách an toàn tối thiểu.

- Yêu cầu về khả năng phát hiện con người và vật cản để điều chỉnh quỹ đạo kịp thời.

ISO 13849: Quy định về thiết kế các bộ phận liên quan đến an toàn của hệ thống điều khiển, bao gồm:

- Thiết kế mạch dừng khẩn cấp với mức độ tin cậy cao.
- Đảm bảo hệ thống điều khiển duy trì chức năng an toàn ngay cả khi xảy ra lỗi phần cứng.
- Giám sát trạng thái hệ thống và phản ứng kịp thời khi phát hiện sự cố.

Hiện tại, các tiêu chuẩn này được áp dụng ở mức định hướng thiết kế và sẽ được triển khai đầy đủ trong các giai đoạn phát triển tiếp theo để đảm bảo robot vận hành an toàn trong môi trường bệnh viện.

4.1.6 Hệ thống cảnh báo và xử lý sự cố

Trong môi trường bệnh viện, nơi có mật độ người qua lại cao và không gian di chuyển hạn chế, việc đảm bảo an toàn cho con người và thiết bị xung quanh là yếu tố bắt buộc khi thiết kế robot. Dựa trên yêu cầu của tiêu chuẩn **ISO 13482** và **ISO 13849**, nhóm đã xây dựng phương án hệ thống xử lý sự cố và cảnh báo nguy hiểm như sau.

– Xử lý sự cố

- Khi hệ thống phát hiện lỗi vận hành, cảm biến báo nguy hoặc mất tín hiệu điều khiển, robot sẽ tự động chuyển sang trạng thái an toàn.
- Mọi hoạt động cơ khí sẽ dừng lại, đồng thời hệ thống điều khiển ghi lại mã lỗi để phục vụ quá trình kiểm tra và khắc phục.
- Chỉ khi lỗi được xử lý và trạng thái an toàn được xác nhận, robot mới được phép khởi động lại.

– Dừng khẩn cấp (Emergency Stop)

- Nút dừng khẩn cấp sẽ được bố trí ở vị trí dễ thấy và dễ thao tác trên thân robot.
- Khi kích hoạt, nguồn cấp cho động cơ bị ngắt ngay lập tức, dừng mọi chuyển động của robot để tránh gây nguy hiểm cho người và thiết bị.

- Hệ thống yêu cầu thao tác thủ công để khởi động lại nhằm đảm bảo không tự hoạt động ngoài ý muốn.

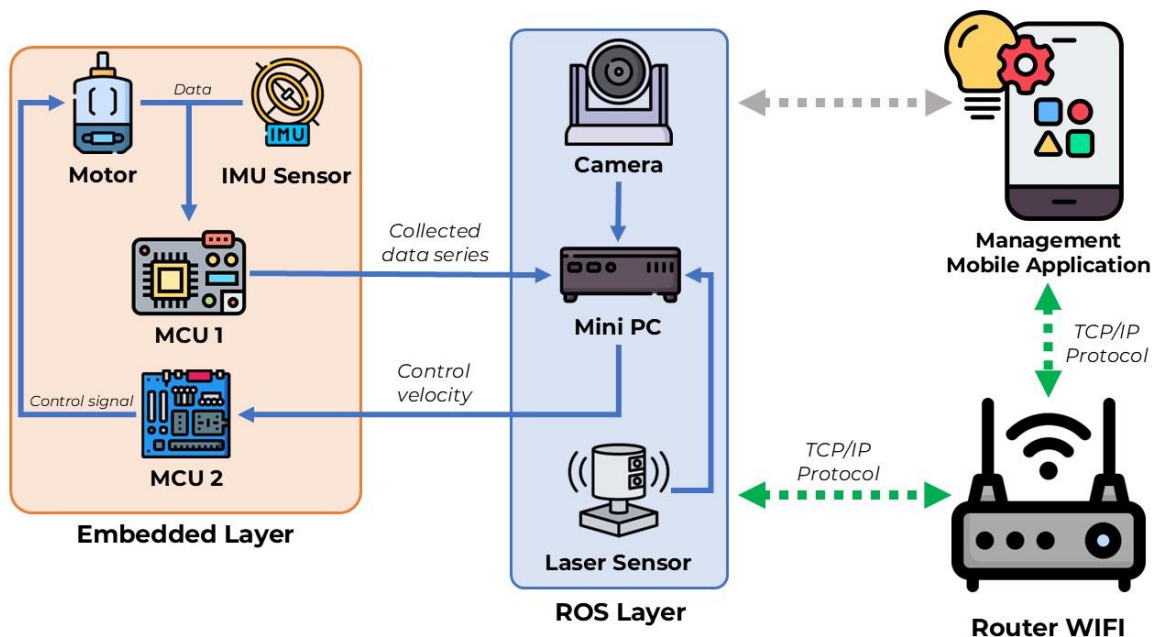
– Cảnh báo nguy hiểm

- Âm thanh và đèn LED: Thông báo trực tiếp tại hiện trường khi robot chuẩn bị di chuyển, đổi hướng đột ngột hoặc phát hiện vật cản nguy hiểm.
- Cảnh báo qua ứng dụng di động: Hệ thống sẽ gửi thông báo thời gian thực (push notification) đến ứng dụng giám sát trên điện thoại hoặc máy tính bảng của nhân viên vận hành.

Các cơ chế trên sẽ được triển khai trong các giai đoạn phát triển tiếp theo của dự án. Việc tuân thủ và áp dụng tiêu chuẩn **ISO 13482** và **ISO 13849** sẽ đảm bảo robot đáp ứng đầy đủ yêu cầu an toàn, nâng cao độ tin cậy và khả năng ứng dụng thực tế trong môi trường bệnh viện.

4.2 Thiết kế phần mềm

4.2.1 Phương pháp đề xuất



Hình 4. 22 Kiến trúc hệ thống

Hệ thống phần mềm được tổ chức theo kiến trúc phân tầng rõ ràng, gồm Tầng Nhúng (Embedded Layer), Tầng ROS (ROS Layer) và Tầng Giao tiếp Mạng (Network Layer), nhằm tối ưu việc thu thập dữ liệu, xử lý và điều khiển từ xa.

Tầng Nhúng (sử dụng 2 vi điều khiển độc lập, phân chia nhiệm vụ giúp giảm tải để bảo trì và mở rộng phần cứng:

- MCU 1 thu thập dữ liệu từ cảm biến IMU (gia tốc, con quay, từ kế) và encoder của 3 động cơ, đóng gói dưới dạng nhị phân và truyền lên Mini PC qua UART tốc độ cao.
- MCU 2 điều khiển 3 động cơ bước vòng kín bằng tín hiệu Step/Dir, nhận lệnh điều khiển tốc độ góc từ Mini PC.

Tầng ROS chạy trên Mini PC với hệ điều hành ROS Melodic:

- Giải mã dữ liệu từ MCU 1, tích hợp tín hiệu IMU và encoder để tính toán vận tốc bánh xe, tọa độ và hướng di chuyển.
- Xử lý dữ liệu từ camera và cảm biến laser để nhận diện vật cản, lập bản đồ, điều hướng và lập kế hoạch chuyển động (SLAM).
- Kết quả điều khiển (vận tốc góc từng bánh) được gửi xuống MCU 2 để điều khiển động cơ chính xác và ổn định.

Tầng Giao tiếp Mạng sử dụng ROS Bridge để kết nối Mini PC với ứng dụng di động qua giao thức TCP/IP thông qua WiFi Router:

- Cho phép người dùng giám sát vị trí, vận tốc, bản đồ; điều khiển robot; thay đổi tham số và nhận video hoặc dữ liệu laser thời gian thực.
- Hệ thống hỗ trợ mở rộng IoT, điều khiển qua web và cập nhật phần mềm từ xa (OTA) cho cả MCU và các node ROS.

➔ Nhờ kiến trúc phân tầng rõ ràng và các chuẩn giao tiếp linh hoạt (UART, ROS message dạng JSON/CBOR), hệ thống đảm bảo hiệu suất cao, khả năng mở rộng tốt và thuận tiện trong vận hành, nâng cấp.

4.2.2 Tầng nhúng (Embedded Layer)

Tầng nhúng sử dụng hai vi điều khiển độc lập với nhiệm vụ tách biệt và hỗ trợ lẫn nhau để đảm bảo thu thập dữ liệu và điều khiển động cơ hiệu quả. MCU 1 thực hiện hai luồng xử lý song song:

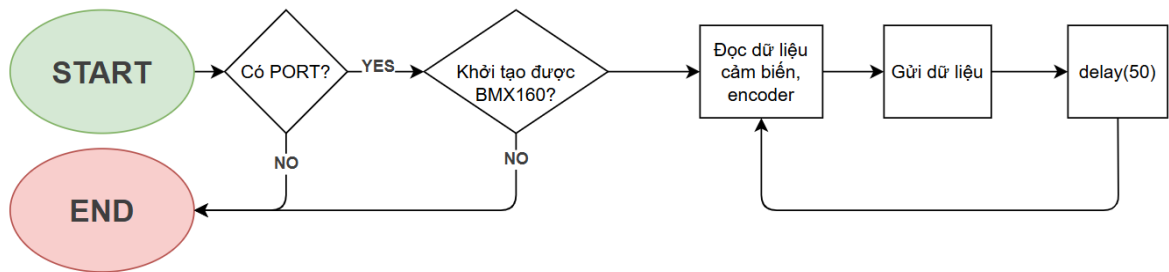
- Giao tiếp **I2C** để đọc liên tục giá trị từ cảm biến IMU: gia tốc, con quay và từ kế.

- Luồng **ngắt (interrupt)** để ghi nhận chính xác xung encoder từ ba động cơ bước vòng kín, đảm bảo không mất dữ liệu ngay cả khi đang xử lý I2C.

Sau khi thu thập đầy đủ, MCU 1 đóng gói dữ liệu theo định dạng chuẩn:

$$DATA_g_x_g_y_g_z_a_x_a_y_a_z_m_x_m_y_m_z_cnt_1_cnt_2_cnt_3$$

Trong đó g_x, g_y, g_z là góc quay quanh các trục; a_x, a_y, a_z là gia tốc theo các phương; cnt_1, cnt_2, cnt_3 là số xung encoder của ba động cơ. Chuỗi dữ liệu được gửi liên tục lên Mini PC qua UART tốc độ cao, đảm bảo độ trễ chỉ vài mili giây, giúp tầng ROS luôn nhận được trạng thái cảm biến cập nhật. Mã được trình bày tại Phụ lục 1.

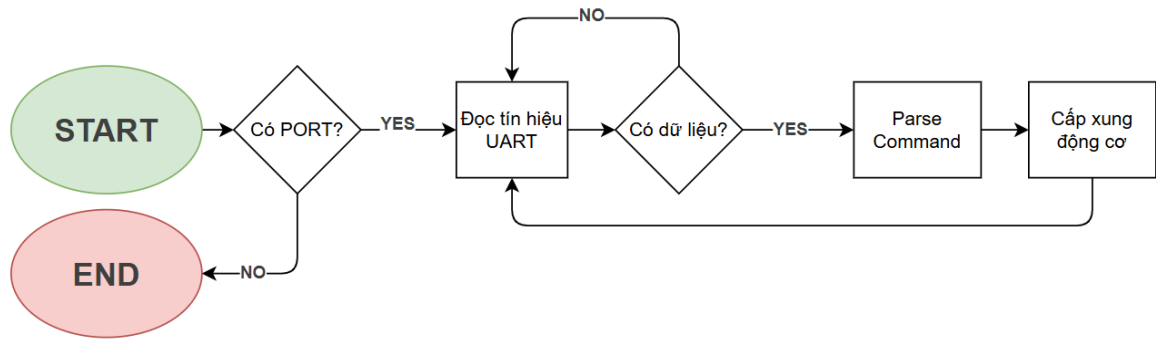


Hình 4. 23 Lưu đồ giải thuật cho MCU 1

Trên MCU 2, chúng tôi triển khai cơ chế điều khiển động cơ bước vòng kín dựa trên vận tốc góc do tầng ROS tính toán. MCU 2 nhận chuỗi lệnh dạng :GO_ ω_1 _ ω_2 _ ω_3 : từ Mini PC qua UART, trong đó $\omega_1, \omega_2, \omega_3$ là vận tốc góc mong muốn (rad/s).

Sau khi phân tích chuỗi, MCU 2 chuyển các giá trị này thành tần số xung theo công thức: $f = (\omega / 2\pi) \times \text{số bước/vòng}$

Sau đó, MCU phát xung Step/Dir để điều khiển driver động cơ. Hệ thống được tối ưu để duy trì điều khiển vòng kín tốc độ, thích ứng với thay đổi tải, giúp động cơ vận hành ổn định, hạn chế rung và sai số. Mã nguồn chi tiết ở Phụ lục 2.



Hình 4. 24 Lưu đồ giải thuật cho MCU2

4.2.2.1 Quy trình điều khiển động cơ khi đã có tính hiệu điều khiển

Trong hệ thống robot omni ba bánh, bài toán điều khiển chuyển động được giải quyết thông qua việc tính toán mối quan hệ giữa vận tốc mong muốn của robot và vận tốc góc của từng bánh xe. Đầu vào điều khiển ở mức cao của robot thường là một vector chuyển động gồm ba thành phần: vận tốc theo trục X (ký hiệu V_x) vận tốc theo trục Y (ký hiệu V_y) và vận tốc quay quanh trục Z (ký hiệu ω_z). Trong hệ thống sử dụng ROS, các giá trị này được truyền thông qua topic /cmd_vel với kiểu dữ liệu geometry_msgs/Twist, nơi linear.x ứng với V_x , linear.y ứng với V_y và angular.z ứng với ω_z .

Để chuyển từ vector chuyển động $[V_x, V_y, \omega_z]$ sang vận tốc góc của từng bánh xe, hệ thống sử dụng **thuật toán động học ngược**. Với cấu hình robot ba bánh omni, mỗi bánh được đặt lệch nhau 120° quanh tâm robot và có góc định vị cụ thể: bánh 1 tại 60° , bánh 2 tại 180° , và bánh 3 tại 300° . Các góc này được lưu trữ dưới dạng radian để thuận tiện cho tính toán. Công thức tổng quát của động học ngược được biểu diễn dưới dạng ma trận:

$$\begin{bmatrix} \omega_1 \\ \omega_2 \\ \omega_3 \end{bmatrix} = \frac{1}{R} \begin{bmatrix} \sin(\sigma_1) & -\cos(\sigma_1) & -L \\ \sin(\sigma_2) & -\cos(\sigma_2) & -L \\ \sin(\sigma_3) & -\cos(\sigma_3) & -L \end{bmatrix} \begin{bmatrix} V_x \\ V_y \\ \omega_z \end{bmatrix}$$

Trong đó:

- R là bán kính bánh xe (m).
- L là khoảng cách từ tâm robot đến tâm bánh xe (m).

- σ_i là góc đặt bánh thứ i (độ).
- ω_i là vận tốc góc của bánh thứ i (rad/s)

Việc nhân ma trận này với vector $[V_x, V_y, \omega_z]$ sẽ cho ra $[\omega_1, \omega_2, \omega_3]$ — đây chính là vận tốc góc cần đặt cho từng bánh xe để robot đạt được chuyển động mong muốn.

Trong phần triển khai thực tế, node ROS `cmd_vel_to_arduino` chịu trách nhiệm thực hiện phép tính động học ngược này. Sau khi khởi tạo, node đăng ký (subscribe) vào topic `/cmd_vel` và mỗi khi nhận được một thông điệp Twist, node sẽ trích xuất $[V_x, V_y, \omega_z]$ từ thông điệp này. Sau đó, các giá trị này được đưa vào hàm `inverse_kinematics()`, nơi ma trận động học ngược J_{inv} được xây dựng từ các tham số góc bánh xe, bán kính bánh và khoảng cách L . Việc nhân J_{inv} với vector vận tốc cơ thể $v_{body} = [V_x, V_y, \omega_z]$ sẽ cho ra mảng chứa ba giá trị $[\omega_1, \omega_2, \omega_3]$.

Tại phía Arduino, chương trình nhận dữ liệu chuỗi từ ROS node, phân tích cú pháp để tách ra ba giá trị vận tốc góc và điều khiển từng động cơ thông qua mạch điều khiển động cơ (motor driver). Việc điều khiển động cơ có thể được thực hiện ở dạng điều khiển vòng hở (open-loop) bằng cách chuyển đổi trực tiếp vận tốc góc sang giá trị PWM.

Cụ thể, Sau khi tính toán Mini PC định dạng lệnh điều khiển gồm ba giá trị vận tốc này và truyền chúng tới khối điều khiển động cơ nội bộ. Trong firmware, mỗi giá trị vận tốc góc được chuyển đổi sang tần số xung PUL cần thiết, dựa trên thông số số xung mỗi vòng quay động cơ (PULSES_PER_REV). Đồng thời, dấu của vận tốc góc quyết định tín hiệu DIR — HIGH hoặc LOW — để điều khiển chiều quay. Với tốc độ dương, DIR được đặt để động cơ quay thuận; với tốc độ âm, DIR được đảo để quay ngược. Tiếp theo, bộ đếm thời gian (Timer1) của Arduino được cấu hình để tạo ngắt định kỳ với chu kỳ micro giây chính xác (TICK_INTERVAL). Trong mỗi lần ngắt, chương trình so sánh bộ đếm nội bộ của từng động cơ với giá trị `toggleCount` tính từ vận tốc yêu cầu: nếu đủ thời gian trôi qua, tín hiệu PUL được đảo trạng thái để tạo một xung mới cho động cơ. Vì động cơ bước di chuyển một bước với mỗi xung PUL, tần số xung này sẽ quyết định tốc độ quay của bánh.

Cả ba bánh được điều khiển song song bằng kỹ thuật điều khiển bit trực tiếp lên thanh ghi PORTD, cho phép thay đổi đồng thời trạng thái các chân PUL với độ trễ tối thiểu. Điều này giúp đảm bảo rằng ba bánh có thể nhận xung chính xác và đồng bộ, đặc biệt quan trọng khi robot cần di chuyển thẳng hoặc quay mượt mà. Khi không có yêu cầu vận tốc cho một bánh, chương trình gán giá trị cực lớn cho toggleCount của bánh đó để nó không nhận xung và dừng hẳn. Ngược lại, nếu nhận lệnh tốc độ mới, các giá trị toggleCount và DIR được cập nhật ngay lập tức, cho phép robot thay đổi hướng hoặc tốc độ gần như tức thời.

Toàn bộ chu trình này — từ nhận vector vận tốc, chuyển đổi qua công thức động học, xác định chiều quay, tính toán tần số xung, và phát xung qua PUL/DIR — diễn ra hoàn toàn trong MCU mà không cần bộ xử lý trung gian khác. Điều này đảm bảo phản hồi nhanh, giảm độ trễ, đồng thời cho phép hệ thống hoạt động ổn định và chính xác trong điều kiện thực tế.

4.2.3 Tầng ROS layer

Hệ thống được xây dựng trên nền tảng ROS Melodic, gồm 3 thành phần chính khi robot vận hành.

4.2.3.1 Cấu hình hệ thống

Chúng tôi sử dụng file robot.launch làm điểm khởi đầu để khởi tạo toàn bộ hệ thống ROS, đảm nhận vai trò trung tâm để khởi tạo và cấu hình toàn bộ các thành phần phần mềm cần thiết cho robot omni 3 bánh hoạt động và di chuyển. File này tự động nạp các thành phần quan trọng như:

Mô hình **URDF** để khai báo hệ thống TF tĩnh: base_link→laser, base_link→imu_frame, base_link→camera. URDF mô tả cấu trúc cơ khí, giúp xác định chính xác vị trí và hướng cảm biến trong không gian 3D.

Các node cảm biến:

- rplidar_ros thu thập dữ liệu quét từ LiDAR, xuất ra topic /scan cung cấp thông tin khoảng cách vật thể xung quanh.

- zed-wrapper đọc hình ảnh và độ sâu từ camera ZED, xuất các topic như /zed/left/image_rect_color, /zed/depth/depth_registered và dữ liệu nội suy góc nhìn. Đối với camera, chúng tôi sử dụng các tham số cấu hình mặc định.

Ngoài ra, các node xử lý dữ liệu thô, bộ lọc và ghép nối cảm biến (như EKF) cũng được khởi chạy để đảm bảo tích hợp và xử lý dữ liệu đồng bộ từ nhiều nguồn.

Bảng 4. 1 Cấu hình tham số RPLidar

Tham số	Giá trị
serial_port	/dev/serial/by-id/usb-Silicon_Labs_CP2102_USB_to_UART_Bridge_Controller_0001-if00-port0
serial_baudrate	115200
frame_id	laser
Inverted	false
angle_compensate	true

Đối với IMU chúng tôi triển khai một **node riêng cho IMU** để publish các topic:

- /imu/data_raw: chứa dữ liệu gia tốc và góc quay.
- /imu/mag: dữ liệu từ kế (magnetometer).
- /odom_raw: thông tin odometry thô từ encoder hoặc SLAM sơ bộ.

Các dữ liệu này là đầu vào cho quá trình lọc và hợp nhất tiếp theo.

Để xử lý nhiễu và trôi (drift), chúng tôi sử dụng node ekf_imu_filter_madgwick_node, áp dụng bộ lọc Madgwick để hiệu chỉnh dữ liệu IMU và xuất ra topic /imu/data ổn định hơn, phục vụ tính toán hướng và tư thế của robot.

Bảng 4. 2 Cấu hình tham số cho ekf_imu_filter_madgwick_node

Tham số	Giá trị
use_mag	true
gain	0.1
zeta	0.0

fixed_frame	odom
publish_tf	false
use_magnetic_field_msg	true

Để kết hợp dữ liệu LiDAR và camera, sử dụng node `laser_scan_matcher`, Node này đăng ký subscribe vào topic ảnh hoặc điểm sâu, thực hiện đối sánh các đám mây điểm để sinh ra topic `/pose_stamp`, cung cấp thông tin ước tính pose (thể) tạm thời của robot dựa trên tương quan giữa hình ảnh và quét laser.

Bảng 4. 3 Cấu hình tham số cho `laser_scan_matcher`

Tham số	Giá trị
fixed_frame	odom
base_frame	base_link
use_odom	TRUE
use_imu	TRUE
use_vel	FALSE
use_cloud_input	FALSE
kf_dist_linear	0.05
kf_dist_angular	0.087
publish_tf	TRUE
publish_pose	FALSE
publish_pose_stamped	TRUE
max_iterations	10
max_correspondence_dist	0.3
max_linear_correction	0.3
max_angular_correction_deg	30
epsilon_xy	1.00E-04
epsilon_theta	1.00E-04
outliers_maxPerc	0.9
sigma	0.01

use_corr_tricks	1
restart	0
restart_threshold_mean_error	0.01
restart_dt	1
restart_dtheta	0.1
clustering_threshold	0.25
orientation_neighbourhood	10
use_point_to_line_distance	1
do_alpha_test	0
do_alpha_test_thresholdDeg	20
outliers_adaptive_order	0.7
outliers_adaptive_mul	2
do_visibility_test	0
outliers_remove_doubles	1
do_compute_covariance	0

Cuối cùng, Node `ekf_localization_node` đóng vai trò trung tâm trong việc hợp nhất dữ liệu cảm biến. Nó subscribe vào các topic `/odom_raw` (odometry thô), `/pose_stamp` (ước lượng pose tạm thời), `/imu/data` (IMU đã lọc),

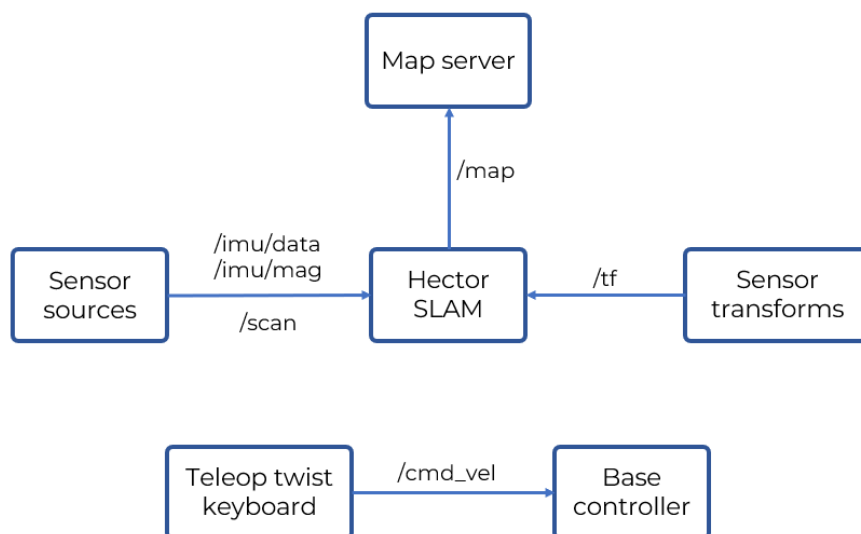
Và áp dụng Bộ lọc Kalman Mở Rộng (EKF) để tạo ra output `/odom` có độ chính xác cao. Node này cũng phát TF từ `odom` → `base_link`, đảm bảo hệ ROS có thể xác định chính xác vị trí và hướng hiện tại của robot phục vụ điều khiển.

Bảng 4. 4 Cấu hình tham số cho bộ lọc ekf

Tham số	Giá trị
frequency	30
sensor_timeout	0.1
two_d_mode	TRUE
transform_time_offset	0
transform_timeout	0

print_diagnostics	TRUE
debug	FALSE
map_frame	map
odom_frame	odom
base_link_frame	base_link
world_frame	odom
publish_tf	TRUE

4.2.3.2 Xây dựng bản đồ với Hector SLAM



Hình 4. 25 Kiến trúc hệ thống Hector SLAM

Chúng tôi triển khai gói Hector SLAM để tạo bản đồ 2D thời gian thực bằng cách xử lý dữ liệu từ cảm biến LiDAR, không cần sử dụng odometry từ bánh xe. File launch `hector_mapping.launch` khởi tạo các thành phần chính, gồm Node `hector_mapping`, TF tính giữa LiDAR và `base_link`, Topic `/map` để xuất bản đồ.

Thuật toán Hector SLAM sử dụng phương pháp **Gauss–Newton** để ước lượng pose hiện tại. Dữ liệu từ topic `/scan` được đưa vào bộ so khớp (matcher), so sánh liên tiếp các bản quét laser để tính toán sự dịch chuyển của robot. Vì không phụ thuộc vào encoder, hệ thống vẫn đảm bảo độ chính xác cao trong môi trường có thể gây trượt bánh.

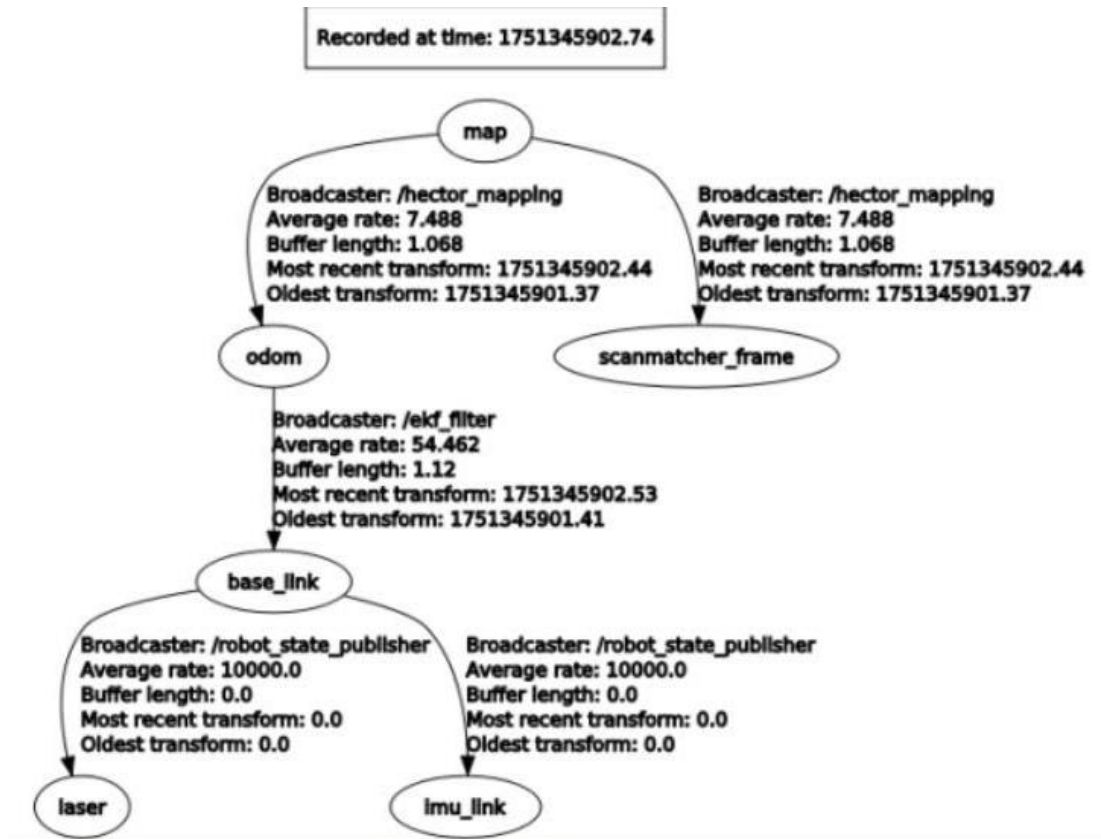
Các đám mây điểm từ LiDAR được tích hợp liên tục vào bản đồ dạng occupancy grid, với độ phân giải và kích thước lưới được cấu hình sẵn.

Bảng 4. 5 Cấu hình tham số cho hector_mappig

Tham số	Giá trị
map_frame	map
base_frame	base_link
odom_frame	odom
use_tf_scan_transformation	TRUE
use_tf_pose_start_estimate	FALSE
pub_map_odom_transform	true
map_resolution	0.05
map_size	52
map_start_x	0.5
map_start_y	0.5
map_multi_res_levels	2
update_factor_free	0.2
update_factor_occupied	0.85
map_update_distance_thresh	0.4
map_update_angle_thresh	0.6
advertise_map_service	TRUE
scan_subscriber_queue_size	10
scan_topic	/scan
tf_map_scanmatch_transform_frame_name	scanmatcher_frame
pub_map_scanmatch_transform	TRUE

Chúng tôi cấu hình các tham số như map_resolution, map_size, update_factor_free, và update_factor_occupied để đảm bảo cân bằng giữa độ chi tiết bản đồ và hiệu năng xử lý. Cụ thể, map_resolution = 0.05m/pixel giúp hiển thị rõ đường viền vật thể; update_factor_free và update_factor_occupied xác định độ tin cậy khi nhận diện vùng trống và vật cản.

Chuỗi TF được thiết lập với liên kết tĩnh base_link → laser, và các liên kết map → odom → base_link được tự động sinh bởi node hector_mapping. Nhờ đó, các node điều hướng hoặc hiển thị bản đồ có thể dễ dàng xác định chính xác vị trí của robot trong bản đồ toàn cục.

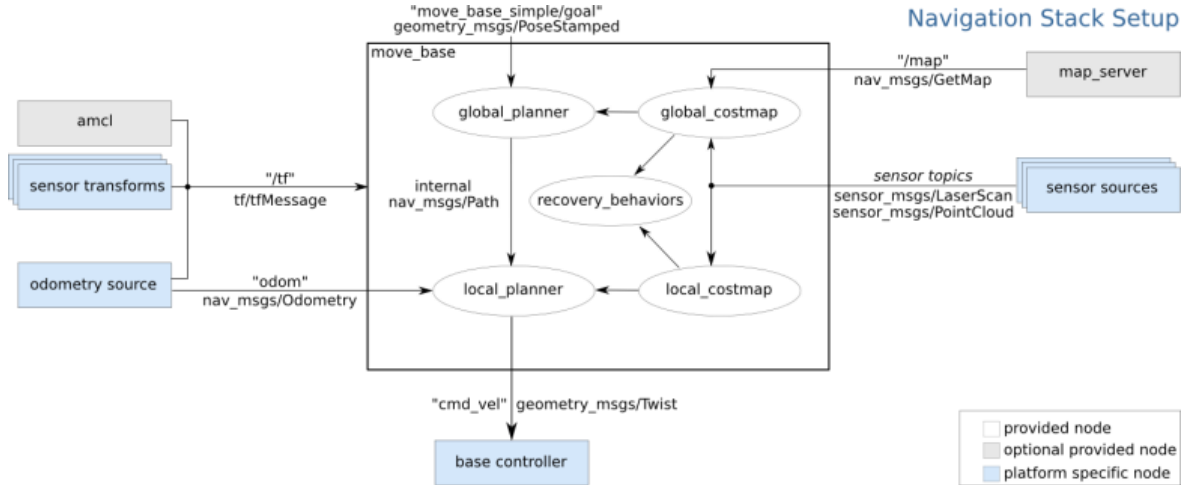


Hình 4. 26 Sơ đồ transform cho việc vẽ map

Sau khi bản đồ được cập nhật, chúng tôi sử dụng node map_saver để lưu bản đồ dưới dạng file YAML và PGM, giúp dễ dàng khôi phục khi khởi động lại robot. Đồng thời, topic /initial_pose cũng được cấu hình để nhận vị trí khởi tạo từ người dùng qua RViz, hỗ trợ quá trình định vị ban đầu trong bản đồ.

Nhờ triển khai Hector SLAM, hệ thống có thể vẽ và cập nhật bản đồ theo thời gian thực trong quá trình robot di chuyển, cung cấp thông tin bản đồ chính xác.

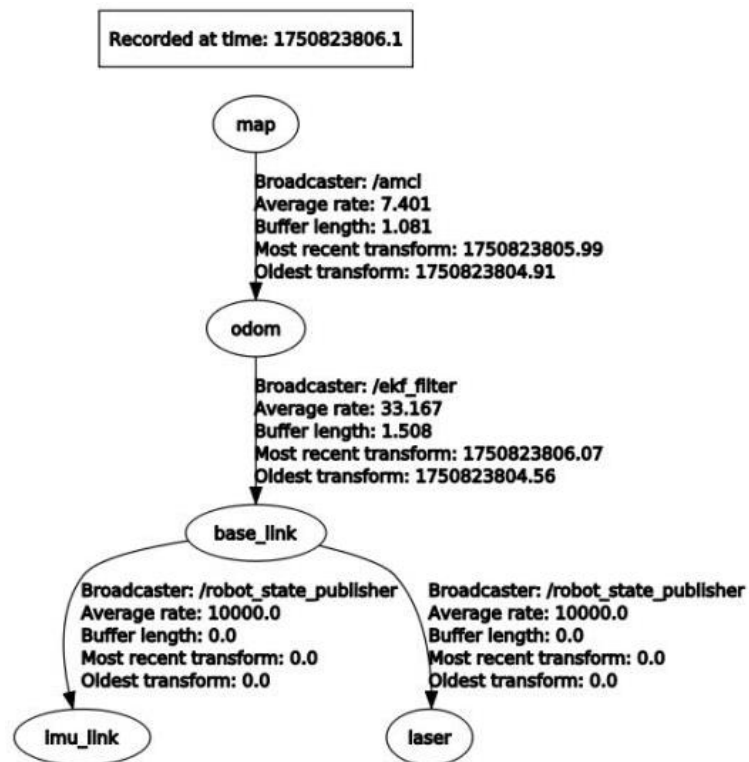
4.2.3.3 Navigation với Move Base



Hình 4. 27 Kiến trúc hệ thống điều hướng move_base trong ROS

Trong phần này, cấu hình hệ thống điều hướng sử dụng gói move_base để lập kế hoạch đường đi và điều khiển robot omni 3 bánh di chuyển đến mục tiêu. File launch “navigation.launch” gồm các node chính map_server (phục vụ bản đồ tĩnh), amcl (xác định vị trí robot), move_base (điều hướng và điều khiển robot), cùng các plugin hỗ trợ: global_costmap, local_costmap, global_planner, local_planner và recovery_behaviors.

Node map_server đọc bản đồ tĩnh từ file YAML và publish lên topic /map, cung cấp dữ liệu không gian cho quá trình lập kế hoạch di chuyển. Node amcl (Adaptive Monte Carlo Localization) sử dụng dữ liệu từ LiDAR (/scan) và TF map → odom để thực hiện lọc hạt (particle filter), liên tục cập nhật vị trí robot trên bản đồ qua topic /amcl_pose và phát sinh transform map → odom.



Hình 4. 28 Sơ đồ transform cho navigation

Bảng 4. 6 Cấu hình cho amcl

Tham số	Giá trị
odom_model_type	omni
global_frame_id	map
odom_alpha1	0.01
odom_alpha2	0.01
odom_alpha3	0.02
odom_alpha4	0.02
odom_alpha5	0.05
transform_tolerance	0.5
gui_publish_rate	10
laser_model_type	likelihood_field
laser_max_beams	30
laser_z_hit	0.8

laser_z_rand	0.2
laser_sigma_hit	0.2
laser_z_short	0.05
laser_z_max	0.05
laser_lambda_short	0.1
min_particles	500
max_particles	5000
kld_err	0.05

Node `move_base` nạp các tham số điều hướng và tính chi phí từ hai bản đồ

- **global_costmap** sử dụng bản đồ tĩnh để tính toán vùng di chuyển an toàn toàn cục với các lớp như *obstacle layer* và *inflation layer*.
- **local_costmap** cập nhật chi phí cục bộ từ dữ liệu LiDAR thời gian thực, giúp robot phản ứng nhanh với vật cản mới.

Chúng tôi sử dụng `global_planner` là `NavfnROS`, đảm bảo tìm đường đi tối ưu trên toàn bản đồ. `local_planner` được cấu hình là `TEB Local Planner` (Time Elastic Band) – một thuật toán tối ưu đa mục tiêu, hỗ trợ tốt cho robot holonomic như robot omni 3 bánh. TEB cho phép điều chỉnh linh hoạt các tham số như:

- `xy_goal_tolerance`, `yaw_goal_tolerance`: xác định độ chính xác khi đến đích.
- `max_vel_x`, `max_vel_y`, `max_vel_theta`: giới hạn tốc độ.
- `weight_obstacle`, `min_obstacle_dist`: xử lý né vật cản.

Bảng 4. 7 Cấu hình tham số cho `teb_local_planner`

Tham số	Giá trị
<code>odom_topic</code>	<code>odom</code>
<code>footprint_model.type</code>	<code>circular</code>
<code>footprint_model.radius</code>	<code>0.2</code>
<code>holonomic_robot</code>	<code>TRUE</code>
<code>max_vel_x</code>	<code>0.2</code>
<code>max_vel_y</code>	<code>0.2</code>

max_vel_theta	0.5
acc_lim_x	0.2
acc_lim_y	0.2
acc_lim_theta	0.5
xy_goal_tolerance	0.05
yaw_goal_tolerance	0.05
free_goal_vel	TRUE
dt_ref	0.2
dt_hysteresis	0.02
min_samples	4
global_plan_viapoint_sep	0.3
weight_viapoint	1
feasibility_check_no_poses	5
allow_init_with_backwards_motion	TRUE
weight_optimaltime	1
weight_shortest_path	0
weight_obstacle	50
weight_dynamic_obstacle	10
min_obstacle_dist	0.3
include_costmap_obstacles	TRUE
costmap_obstacles_behind_robot_dist	1

Các tham số như `inflation_radius`, `path_distance_bias`, `goal_distance_bias`, `max_vel_x` và `max_rot_vel` được điều chỉnh phù hợp với chuyển động đa hướng của robot omni. Với cấu hình `holonomic_robot=true`, robot có thể di chuyển ngang linh hoạt, tận dụng tốt đặc tính bánh omni.

Để xử lý các tình huống như mắc kẹt hoặc không tìm được đường đi, chúng tôi cấu hình các hành vi khôi phục (`recovery_behaviors`) như: (`clear_costmap_recovery`) (`rotate_recovery`).

Khi gặp sự cố, `move_base` sẽ tự động kích hoạt các hành vi này để robot tiếp tục nhiệm vụ.

Bảng 4. 8 Cấu hình tham số recovery behaviors

Tham số	Giá trị
recovery_behaviors[0].name	clear_costmap
recovery_behaviors[0].type	clear_costmap_recovery/ClearCostmapRecovery
recovery_behaviors[1].name	rotate_recovery
recovery_behaviors[1].type	rotate_recovery/RotateRecovery
rotate_recovery.max_rotation_deg	45
rotate_recovery.rotation_velocity	0.5 (rad/s)

4.2.4 Mobile Application

Chúng tôi phát triển ứng dụng di động bằng React Native, hướng đến tạo ra một giao diện thân thiện và trực quan, giúp y tá dễ dàng điều khiển và giám sát robot omni 3 bánh trong quy trình giao thuốc. Mục tiêu chính của ứng dụng bao gồm: kết nối và giao tiếp với robot, gửi lệnh di chuyển, và theo dõi trạng thái thực hiện nhiệm vụ.

Để thiết lập kết nối mạng, chúng tôi sử dụng thư viện react-native-zeroconf nhằm quét và phát hiện các thiết bị hỗ trợ mDNS/Bonjour trên cùng mạng WiFi. Khi y tá khởi chạy ứng dụng, module Zeroconf tự động tìm kiếm service ROS Bridge do robot cung cấp, thu thập địa chỉ IP và cổng của robot. Sau đó, ứng dụng khởi tạo kết nối WebSocket đến ROS Bridge Server, mở kênh hai chiều cho việc gửi nhận message ROS qua JSON.

Về phần gửi lệnh, chúng tôi tích hợp roslibjs (thông qua roslib trên React Native) để xây dựng các Publisher và ServiceClient. Đặc biệt, khi y tá chọn vị trí mục tiêu (ví dụ: gần vị trí hiện tại hoặc phòng bệnh trong lịch hẹn), ứng dụng tạo một thông điệp PoseStamped với header.frame_id là "map" và tọa độ đích, rồi publish vào topic /move_base_simple/goal. Robot sẽ nhận thông điệp này, đưa vào pipeline navigation, và di chuyển đến vị trí đã chọn. Ngoài ra, để cung cấp tính năng lập danh sách điểm đích có thể di chuyển, ứng dụng gọi service getPoint để nhận mảng

waypoint khả dụng, sau đó hiển thị lên bản đồ tích hợp (component bản đồ MapView) để y tá chọn.

Phân hiển thị trạng thái thực hiện nhiệm vụ được xây dựng dựa trên việc subscribe vào các topic trạng thái chính: /move_base/status, /amcl_pose và topic tùy chỉnh như /robot/task_state. Ứng dụng lắng nghe các message JSON trả về từ ROS Bridge, cập nhật trạng thái động của robot (đang di chuyển, đã đến, đang giao thuốc, hoàn thành) lên giao diện người dùng. Chúng tôi thiết kế các component React như <TaskStatus>, <RobotPosition>, và <GoalList>, giúp y tá theo dõi chi tiết tiến độ và vận hành của robot theo thời gian thực.

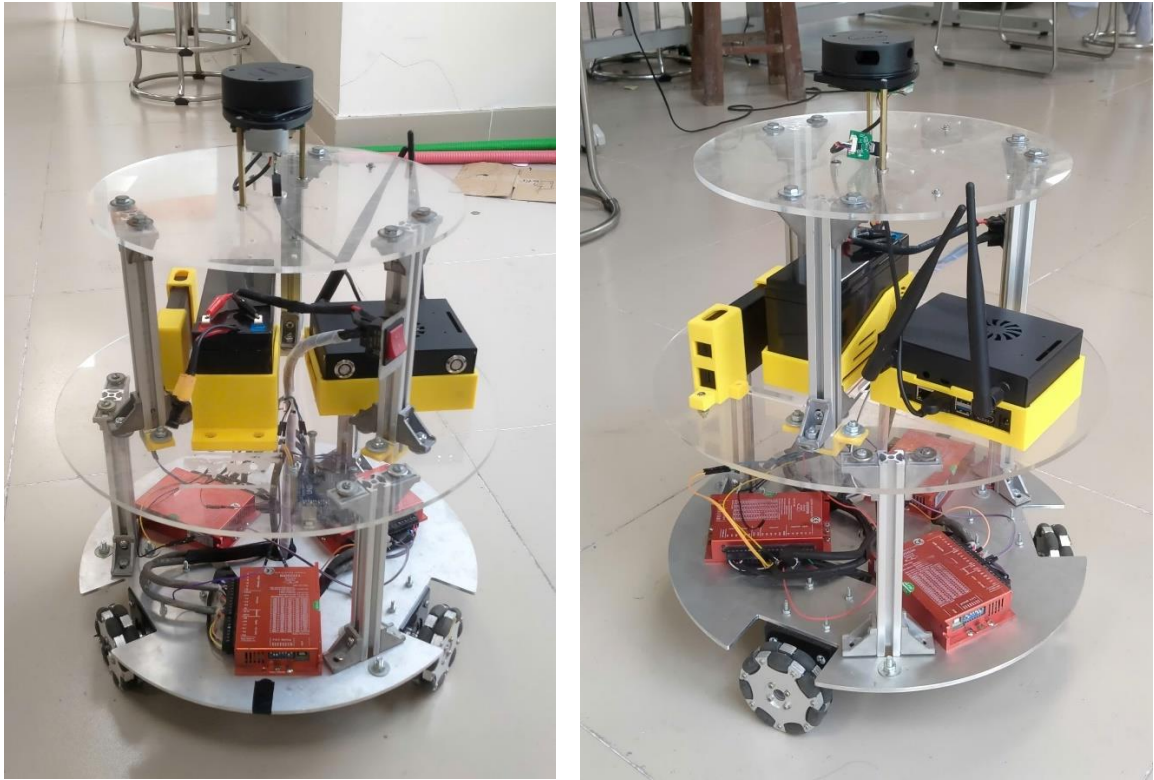
Về giao diện, chúng tôi áp dụng các nguyên tắc thiết kế Material Design, sử dụng các thư viện UI phổ biến như React Native Paper để đảm bảo tính nhất quán và khả năng mở rộng. Màn hình chính gồm bản đồ định vị robot, danh sách mục tiêu, và thanh điều hướng dưới cùng cho các chức năng: Kết nối, Gửi lệnh, Theo dõi và Cài đặt. Chúng tôi cũng tối ưu hóa trải nghiệm người dùng bằng cơ chế tái kết nối tự động khi mất mạng, hiển thị thông báo lỗi và hướng dẫn y tá kiểm tra kết nối WiFi hoặc khởi động lại robot.

Tóm lại, ứng dụng mobile React Native kết nối chặt chẽ với hệ thống ROS qua ROS Bridge, cung cấp đầy đủ chức năng kết nối, gửi lệnh di chuyển và giám sát trạng thái, giúp y tá vận hành robot omni 3 bánh một cách dễ dàng và hiệu quả.

Chương 5. TRIỂN KHAI MÔ HÌNH VÀ KẾT QUẢ

5.1 Mô hình robot hoàn thiện

Sau khi thiết kế và gia công các bộ phận của phần cứng và chuẩn bị đầy đủ các linh kiện và các thành khác. Tiến hành việc lắp ráp robot di động omni giao thuốc trong bệnh viện hoàn chỉnh như hình 5.1 dưới đây.



Hình 5. 1 Mô hình robot di động omni giao thuốc trong bệnh viện

Robot của chúng tôi có khả năng di chuyển linh hoạt trong không gian làm việc với thiết kế dạng model 3 bánh omni.

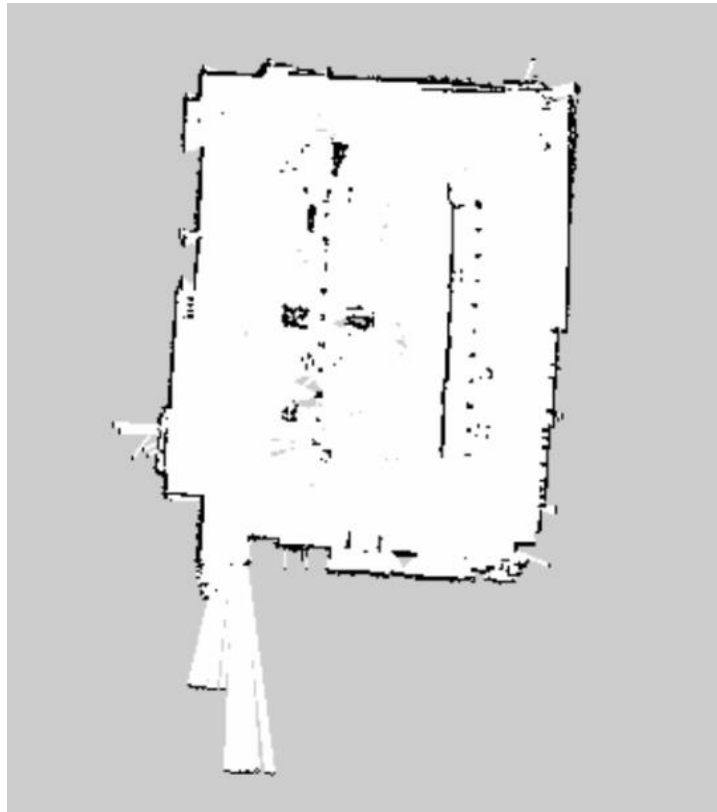
Có khả năng né các vật và như con người xung quanh với LiDAR và sử dụng thuật toán Dijkstra để tự động tìm đường đi tối ưu đến đích.

5.2 Kết quả xây dựng bản đồ với Hector SLAM

Sau khi hoàn tất phần cứng, robot được triển khai chạy gói Hector SLAM để xây dựng bản đồ 2D của môi trường:

Môi trường thử nghiệm: một khu vực phòng học không gian làm việc của robot, có các góc rẽ và vật cản.

- Robot di chuyển quanh khu vực, LiDAR thu thập dữ liệu khoảng cách và Hector SLAM xử lý để tạo bản đồ.
- Kết quả: bản đồ 2D được xây dựng có độ chính xác tương đối cao, chi tiết các khu vực và vật cản.



Hình 5. 2 Bản đồ 2D không gian làm việc của robot

Qua hình bản đồ đã thể hiện được hình dáng tổng thể môi trường làm việc là hình chữ nhật, thể hiện rõ khung viền tường của khu vực di chuyển.

Vùng trắng đều, cho thấy robot đã di chuyển bao phủ được phần lớn không gian.

Tuy nhiên ta cũng có thể thấy được những hạn chế

Ở các vùng giữa bản đồ những nhiều vùng đen lớn nhỏ rời rạc không liền mạch cho thấy môi trường nhiều vật cản nhỏ, chân bàn, ghế,... Các cạnh bên của bản đồ không hoàn toàn sắc nét

Căn chỉnh trục bản đồ chưa tốt khiến bản đồ bị xoay nhẹ (không song song với trục ngang dọc của ảnh), điều này có thể ảnh hưởng đến **định hướng** khi dùng navigation.

5.3 Kết quả bám đường của thuật toán Ted Local Planner

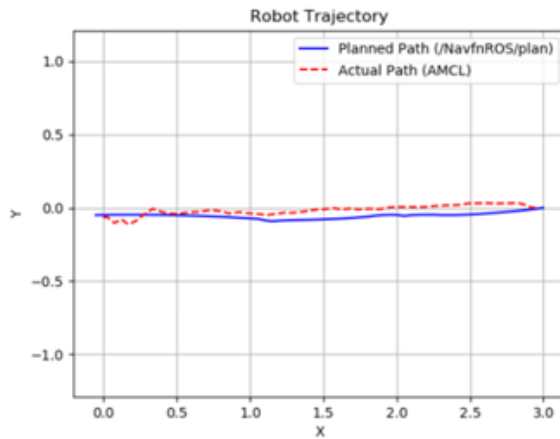
Robot được điều khiển tự động di chuyển đến các vị trí định trước thông qua gói move_base kết hợp thuật toán TEB Local Planner, việc tối ưu các tham số của thuật toán giúp robot có khả năng bám đường tốt hơn:

Thông số Teb Local Planner ban đầu:

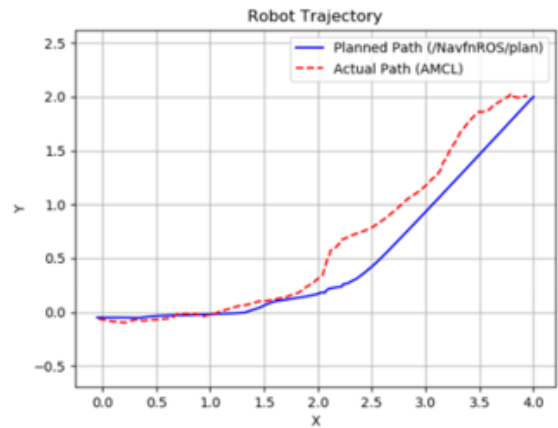
Bảng 5. 1 Tham số Ted local Planner ban đầu

Nhóm	Tham số	Giá trị	Mô tả
Vận tốc & Gia tốc	max_vel_x	0.2	Vận tốc tối đa theo trục X (m/s)
	max_vel_y	0.2	Vận tốc tối đa theo trục Y (m/s)
	max_vel_theta	0.5	Vận tốc quay tối đa (rad/s)
	acc_lim_x	0.2	Gia tốc tối đa theo trục X (m/s^2)
	acc_lim_y	0.2	Gia tốc tối đa theo trục Y (m/s^2)
	acc_lim_theta	0.5	Gia tốc quay tối đa (rad/s^2)
Mục tiêu	xy_goal_tolerance	0.05	Sai số cho phép theo vị trí X-Y (m)
	yaw_goal_tolerance	0.05	Sai số cho phép theo góc quay (rad)
	free_goal_vel	TRUE	Cho phép đến goal với bất kỳ vận tốc nào
Thời gian & Mẫu	dt_ref	0.2	Khoảng thời gian giữa các bước tối ưu (s)
	dt_hysteresis	0.02	Độ trễ cập nhật dt_ref
	min_samples	4	Số mẫu tối thiểu trong quỹ đạo
	feasibility_check_no_poses	5	Số pose kiểm tra khả thi của quỹ đạo
Theo kế hoạch toàn cục	global_plan_viapoint_sep	0.3	Tạo 1 via point mỗi 30cm
	weight_viapoint	1.0	Trọng số để bám sát đường global plan
	allow_init_with_backwards_motion	TRUE	Cho phép bắt đầu bằng chuyển động lùi

Biểu đồ so sánh giữa quỹ đạo dự kiến (Planned Path) và quỹ đạo thực tế (Actual Path) của robot sau khi truyền tham số ban đầu cho TEB Local Planner.



Quỹ đạo thẳng



Quỹ đạo cong

Hình 5. 3 Quỹ đạo dự kiến và quỹ đạo thực tế của robot với thông số lần 1

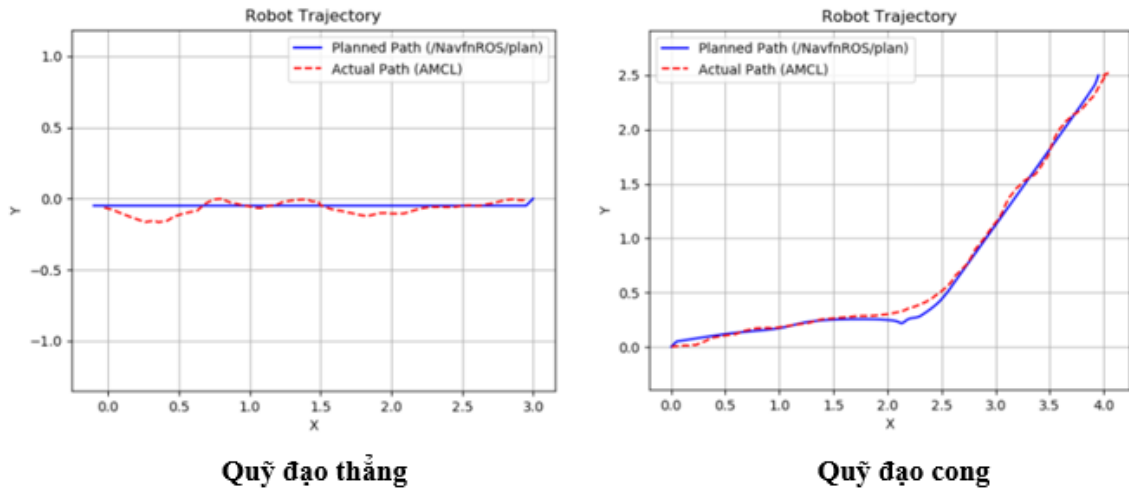
Với các giá trị thông số ban đầu có thể thấy quỹ đạo của robot được biểu diễn trên đồ thị có xu hướng lệch về phía bên trái đối với của quỹ đạo thẳng và cong.

Tối ưu thông số Teb Local Planner lần 2

Bảng 5. 2 Tối ưu thông số Ted local Planner lần 2

Tham số	Giá trị cũ	Giá trị mới	Ghi chú
max_vel_x	0.2	0.3	Tăng tốc độ tiến
max_vel_y	0.2	0.3	Tăng tốc độ ngang
xy_goal_tolerance	0.05	0.03	Giảm sai số vị trí, yêu cầu chính xác hơn
min_samples	4	3	Tăng số mẫu tối ưu, giúp mượt hơn
global_plan_viapoint_sep	0.3	0.2	Tạo via-point dày hơn (mỗi 20cm)
weight_viapoint	1.0	4.0	Ghi đè cuối là 4.0, tăng trọng số bám plan
weight_optimaltime	1.0	2.0	Ưu tiên tối ưu thời gian cao hơn
weight_obstacle	50.0	10.0	Giảm trọng số tránh vật tĩnh
weight_dynamic_obstacle	10.0	15.0	Tăng trọng số tránh vật động
min_obstacle_dist	0.3	0.25	Cho phép đi gần vật cản hơn
global_plan_overwrite_orientation	Mặc định	FALSE	Thêm mới: không ghi đè hướng global plan
global_plan_viapoint_look_ahead	Mặc định	1.0	Thêm mới: tạo via-point trong tầm nhìn 1m
max_global_plan_lookahead_dist	Mặc định	2.5	Thêm mới: giới hạn khoảng nhìn xa của global plan

Biểu đồ so sánh giữa quỹ đạo dự kiến (Planned Path) và quỹ đạo thực tế (Actual Path) của robot sau khi tối ưu tham số cho TEB Local Planner lần 2.



Hình 5. 4 Quỹ đạo dự kiến và quỹ đạo thực tế của robot với thông số lần 2

Với các giá trị thông số tối ưu lần 2 có thể thấy quỹ đạo của robot được biểu diễn trên đồ thị đối với quỹ đạo thẳng còn vọt lố tương đối lớn, ngược lại với quỹ đạo cong robot thể hiện khả năng bám đường tốt.

Tối ưu thông số TEB Local Planner cuối cùng

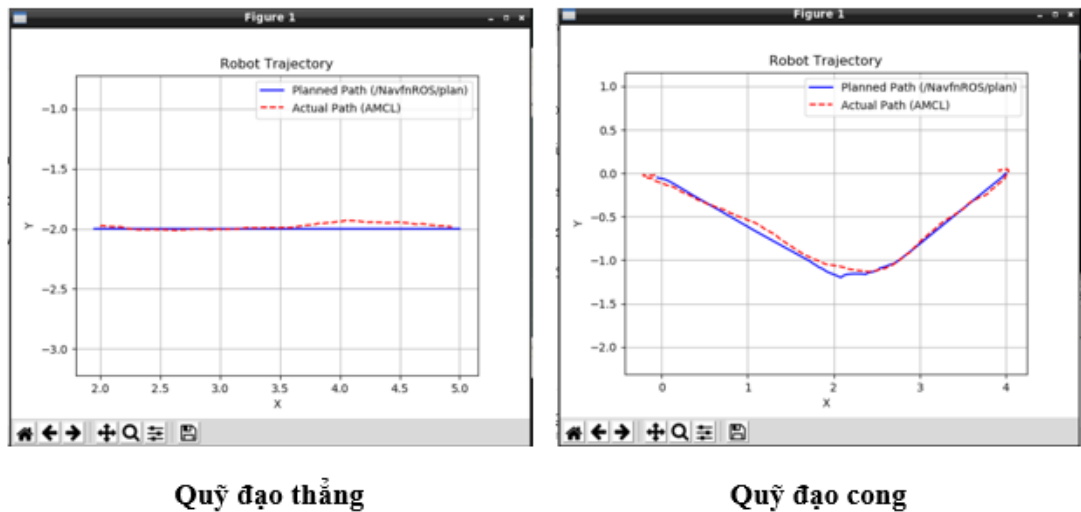
Bảng 5. 3 Tối ưu thông số Ted local Planner cuối cùng

Nhóm	Tham số	Giá trị cũ	Giá trị mới	Ghi chú
Vận tốc & Gia tốc	max_vel_x	0.3	0.2	Giảm tốc độ tiến
	max_vel_y	0.3	0.2	Giảm tốc độ ngang
	max_vel_theta	0.5	0.35	Giảm tốc độ quay
	acc_lim_x	0.1	0.15	Tăng giới hạn gia tốc trục X
	acc_lim_y	0.1	0.15	Tăng giới hạn gia tốc trục Y
	acc_lim_theta	0.5	0.15	Giảm giới hạn gia tốc quay
	max_vel_x_backwards	—	0.2	Thêm mới: tốc độ lùi tối đa
	holonomic_robot	—	TRUE	Thêm mới: robot hỗ trợ di chuyển đa hướng
Mục tiêu	xy_goal_tolerance	0.02	0.1	Giảm yêu cầu chính xác vị trí
	yaw_goal_tolerance	0.05	0.3	Giảm yêu cầu chính xác hướng quay
	free_goal_vel	TRUE	FALSE	Không cho phép có vận tốc tại điểm đích
	complete_global_plan	—	TRUE	Thêm mới: yêu cầu thực hiện toàn bộ kế hoạch toàn cục
Thời gian & Mẫu	dt_ref	0.18	0.2	Tăng khoảng thời gian giữa các mẫu
	dt_hysteresis	0.02	0.1	Tăng ngưỡng thay đổi dt_ref
	min_samples	4	4	Không đổi
	feasibility_check_no_poses	5	5	Không đổi
Global Plan & Via-point	global_plan_viapoint_sep	0.25	-1	Tắt tạo viapoint tự động
	global_plan_overwrite_orientation	FALSE	FALSE	Không đổi

Nhóm	Tham số	Giá trị cũ	Giá trị mới	Ghi chú
	weight_viapoint	5.0	3	Giảm trọng số theo viapoint
	allow_init_with_backwards_motion	TRUE	FALSE	Không cho phép bắt đầu bằng chuyển động lùi
	max_global_plan_lookahead_dist	2.5	3.0	Tăng khoảng nhìn theo kế hoạch toàn cục
	global_plan_prune_distance	—	1.0	Thêm mới: cắt bớt global plan phía sau robot
Tối ưu hóa quỹ đạo	weight_optimaltime	1.0	3	Tăng trọng số tối ưu thời gian
	weight_shortest_path	1.0	1.0	Không đổi
	weight_obstacle	10.0	100	Tăng mạnh trọng số tránh vật cản
	weight_dynamic_obstacle	15.0	10	Giảm trọng số vật cản động
	weight_inflation	—	0.2	Thêm mới: tránh vùng lân cận vật cản
	weight_kinematics_nh	—	1000	Thêm mới: ràng buộc động học phi holonomic
	weight_acc_lim_x	—	1	Thêm mới: trọng số giới hạn gia tốc X
	weight_acc_lim_theta	—	1	Thêm mới: trọng số giới hạn gia tốc quay
Robot Footprint	footprint_model.type	—	circular	Thêm mới: mô hình robot hình tròn
	footprint_model.radius	—	0.2	Thêm mới: bán kính robot
Vật cản	min_obstacle_dist	—	0.2	Thêm mới: khoảng cách tối thiểu đến vật cản
	inflation_dist	—	0.15	Thêm mới: vùng ảnh hưởng vật cản

Nhóm	Tham số	Giá trị cũ	Giá trị mới	Ghi chú
	include_costmap_obstacles	—	TRUE	Thêm mới: sử dụng vật cản từ costmap
	include_dynamic_obstacles	—	TRUE	Thêm mới: bật tránh vật cản động
	dynamic_obstacle_inflation_dist	—	0.4	Thêm mới: mở rộng vùng vật cản động
Homotopy	enable_homotopy_class_planning	—	TRUE	Thêm mới: lập kế hoạch nhiều lớp topological
	max_number_classes	—	4	Thêm mới: số hướng đi homotopy tối đa
	selection_cost_hysteresis	—	1.0	Thêm mới: tránh chuyển hướng khi chi phí chưa khác biệt nhiều
Khôi phục & Dao động	shrink_horizon_backup	—	TRUE	Thêm mới: co rút quỹ đạo khi gặp lỗi
	oscillation_recovery	—	TRUE	Thêm mới: bật chế độ chống dao động
	oscillation_v_eps	—	0.1	Thêm mới: ngưỡng tốc độ để phát hiện dao động
	oscillation_omega_eps	—	0.1	Thêm mới: ngưỡng tốc độ quay phát hiện dao động
	oscillation_recovery_min_duration	—	10	Thêm mới: thời gian phục hồi tối thiểu khi phát hiện dao động

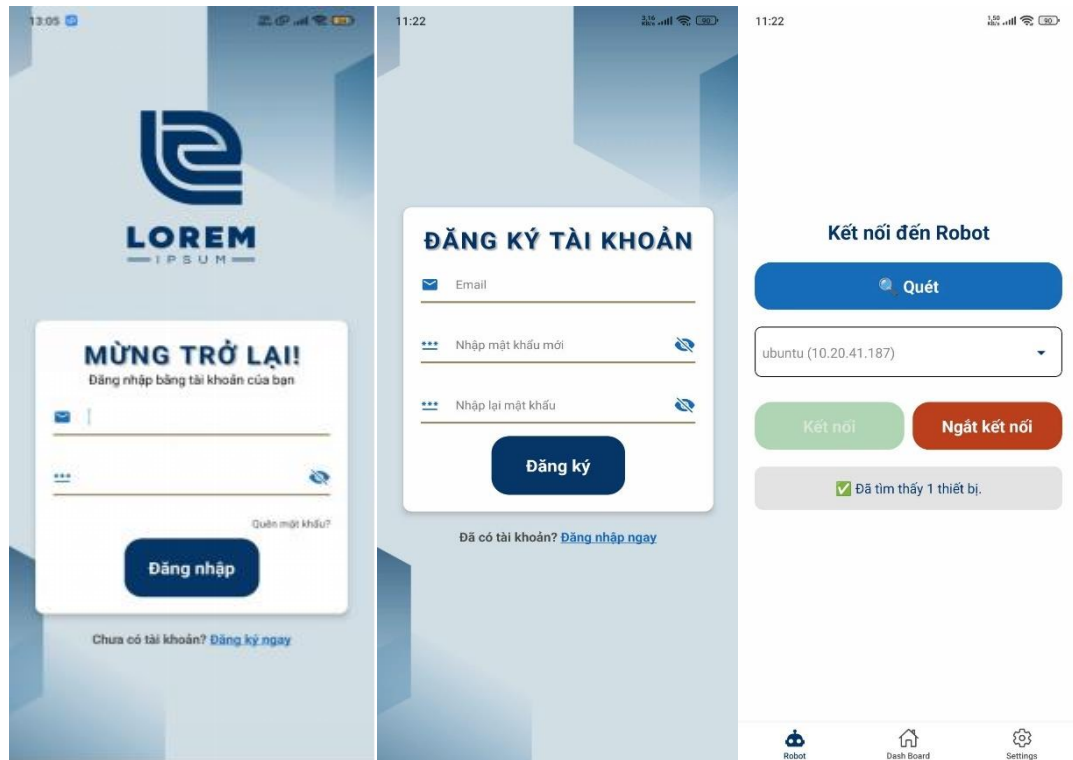
Biểu đồ so sánh giữa quỹ đạo dự kiến (Planned Path) và quỹ đạo thực tế (Actual Path) của robot sau khi tối ưu tham số cho TEB Local Planner lần cuối.



Hình 5. 5 Quỹ đạo dự kiến và quỹ đạo thực tế của robot với thông số tối ưu

Với các giá trị thông số tối ưu lần cuối cho thấy robot với có khả năng bám đường tốt và ổn định ở cả hai quỹ đạo.

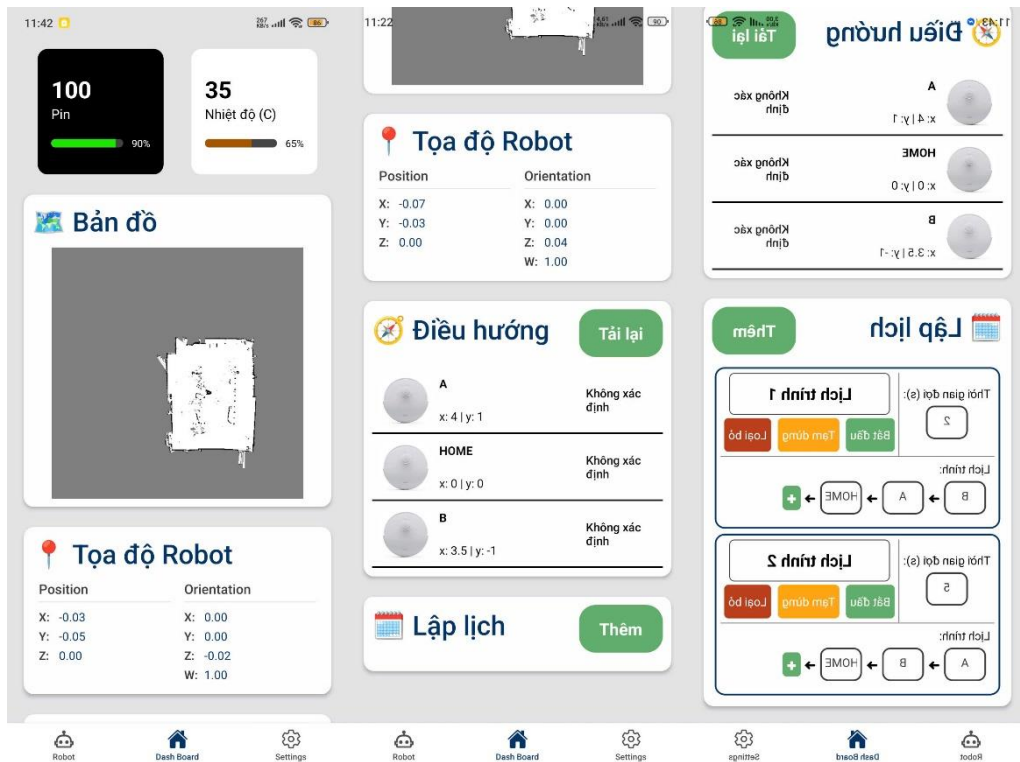
5.4 Mobile app



Hình 5. 6 Màn hình đăng nhập và kết nối robot



Hình 5. 7 Màn hình setting, thông tin tài khoản



Hình 5. 8 Màn hình Dashboard



Robot 3 Bánh Omni

Robot 3 bánh omni là loại robot di chuyển đa hướng sử dụng 3 bánh xoay đa hướng (omni wheels), cho phép di chuyển trơn tru theo mọi hướng mà không cần quay thân robot.

Thông số kỹ thuật	
Kích thước (WxHxD)	400 x 400 x 560 mm
Trọng lượng	7.5 kg
Tốc độ tối đa	0.2 m/s
Pin	Li-ion 24V,10Ah
Động cơ	DC Motor 100 W x3
Giao tiếp	WiFi WedSocket / ROS
Cảm biến	Lidar, IMU, Encoder

Tính năng nổi bật

- Di chuyển đa hướng linh hoạt
- Hỗ trợ điều khiển qua ROS
- Tích hợp cảm biến khoảng cách
- Pin hoạt động liên tục 3-4 giờ



Robot 3 Bánh Omni

Robot 3 bánh omni là loại robot di chuyển đa hướng sử dụng 3 bánh xoay đa hướng (omni wheels), cho phép di chuyển trơn tru theo mọi hướng mà không cần quay thân robot.

Thông số kỹ thuật	
Kích thước (WxHxD)	400 x 400 x 560 mm
Trọng lượng	7.5 kg
Tốc độ tối đa	0.2 m/s
Pin	Li-ion 24V,10Ah
Động cơ	DC Motor 100 W x3
Giao tiếp	WiFi WedSocket / ROS
Cảm biến	Lidar, IMU, Encoder

Tính năng nổi bật

- Di chuyển đa hướng linh hoạt
- Hỗ trợ điều khiển qua ROS
- Tích hợp cảm biến khoảng cách
- Pin hoạt động liên tục 3-4 giờ

Hình 5. 9 Thông số kỹ thuật và các tính năng của robot

Chương 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1 Kết luận

6.1.1 Đã đạt được

Trong đồ án này, nhóm đã nghiên cứu, thiết kế và triển khai một hệ thống robot di động omni phục vụ cho việc phân phối thuốc tự động trong môi trường bệnh viện.

Cụ thể, nhóm đã đạt được các kết quả sau:

- Thiết kế bản vẽ kỹ thuật tổng thể và hoàn thành chế tạo phần cứng robot di động 3 bánh omni giao thuốc trong bệnh viện. Phần cứng gọn và chắc chắn, tích hợp đầy đủ các module điều khiển, nguồn, động cơ và cảm biến đủ điều kiện lập trình thực hiện nhiệm vụ đề ra.
- Lập trình điều khiển robot dựa trên phương pháp động học nghịch, từ chuyển động tổng thể mong muốn của robot tính toán ra vận tốc từng bánh xe để điều khiển giúp robot di chuyển theo các hướng chỉ định.
- Đọc giá trị Encoder và IMU bánh xe phục vụ cho việc tính toán odometry định vị robot trên bản đồ
- Dùng LiDAR áp dụng thuật toán SLAM cùng nền tảng thư viện mạnh mẽ hỗ trợ của hệ điều ROS xây dựng bản đồ môi trường làm việc xung quanh của robot, set up các địa điểm giao và nhận thuốc.
- Phát triển giao diện ứng dụng nền tảng di động cho y tá dùng quản lý theo dõi robot thời gian thực các thông số như tốc độ, tình trạng pin, vị trí hiện tại của robot, giám sát tiến độ vận chuyển thuốc.

6.1.2 Chưa đạt được

Phương trình động học cho robot chưa tối ưu hoàn toàn cho robot chuyển động một cách chính xác nhất, do đó các yếu tố như tốc độ, phi tuyến và sai số thực tế như độ trượt của bánh xe, lực cản mặt sàn có thể ảnh hưởng đến khả năng chuyển động chính xác của robot.

Dù đã có ý tưởng ứng dụng thị giác máy (computer vision) vào việc hỗ trợ định hướng phát hiện và tránh va chạm với con người, nhưng hệ thống vẫn chưa tích hợp

thành công camera ZED và chưa triển khai các thuật toán xử lý ảnh phục vụ cho chức năng này.

Hệ thống hiện chưa được tích hợp các chức năng an toàn chủ động như nút dừng khẩn cấp, cảnh báo nguy hiểm.

Các tiêu chuẩn an toàn quốc tế như ISO 13482 và ISO 13849 mới dừng ở mức định hướng nghiên cứu, chưa được áp dụng và kiểm chứng trên mô hình thực tế.

6.2 Hướng phát triển

Việc ứng dụng robot di động trong hệ thống phân phối thuốc chỉ là một bước đầu trong tiềm năng triển khai rộng rãi robot tự hành vào hạ tầng logistics thông minh trong bệnh viện. Việc tiếp tục nâng cao và phát triển hệ thống robot giúp ngày một tăng chất lượng dịch vụ và mang lại nhiều lợi ích cho bệnh viện cũng như bệnh nhân.

- Phát triển tính năng sạc tự động và trạm sạc cho robot. Giúp robot tự động đến nạp năng lượng khi sắp hết hoặc lúc không vận hành mà không cần sự can thiệp không cần thiết của con người.
- Phát triển và nâng cấp bộ xử lý điều khiển của robot để đáp ứng các yêu cầu tính toán ngày càng cao trong các tác vụ điều hướng. Các nền tảng có hiệu năng cao hơn như Jetson Xavier NX hoặc Jetson Orin Nano, nhằm tăng khả năng xử lý song song các tác vụ như nhận dạng hình ảnh, chạy AI inference, hoặc điều phối đa robot trong thời gian thực.
- Nâng cấp lên các phiên bản hệ điều hành cao hơn như ROS Noetic (hoặc ROS 2 Foxy/Humble) để tận dụng hiệu suất xử lý, khả năng bảo mật, và hỗ trợ lâu dài. Chuyển từ ROS 1 sang ROS 2 tăng tính linh hoạt, mở rộng mạng robot, giao tiếp thời gian thực hiệu quả hơn.
- Phát triển tích hợp camera 3D ZED2 vào hệ thống robot, camera được hỗ trợ rất tốt bởi nền tảng NVIDIA và các dòng Jetson. sử dụng thư viện ZED SDK kết hợp ROS để triển khai các ứng dụng nhận diện con người, hỗ trợ tương tác an toàn hơn trong môi trường bệnh viện.

- Tích hợp màn hình hiển thị và chức năng giao tiếp bằng giọng nói trên robot nhằm nâng cao khả năng tương tác người – máy, giúp hệ thống thân thiện và dễ sử dụng hơn đối với người dùng.
- Hoàn thiện và mở rộng hệ thống an toàn, bao gồm kiểm thử và chứng nhận theo tiêu chuẩn ISO 13482 và ISO 13849, bổ sung các cảm biến dự phòng để phát hiện sự cố, cơ chế tự chẩn đoán lỗi và kết nối hệ thống cảnh báo từ xa. Đồng thời nghiên cứu tích hợp chức năng tự động gửi thông báo đến nhân viên vận hành hoặc bộ phận kỹ thuật khi phát hiện tình huống nguy hiểm.
- Bổ sung cảnh báo thời gian thực qua ứng dụng di động để thông báo sự cố và vị trí robot cho người vận hành ngay cả khi không ở hiện trường. Tích hợp nút dừng khẩn cấp và cảnh báo đa phương thức (âm thanh, đèn LED).

TÀI LIỆU THAM KHẢO

- [1] Bộ Khoa học và Công nghệ (MOST – Ministry of Science and Technology), "Chính thức đưa vào sử dụng Hệ thống robot y tế vận chuyển," 4 2020. [Online]. Available: <https://www.most.gov.vn/vn/tin-tuc/20087/chinh-thuc-dua-vao-su-dung-he-thong-robot-y-te-van-chuyen.aspx>. [Accessed 11 7 2025].
- [2] L. Hà, "'Giảng viên chế tạo robot hỗ trợ trong bệnh viện,'" VnExpress," 1 11 2024. [Online]. Available: <https://vnexpress.net/giang-vien-che-cao-robot-ho-tro-trong-benh-vien-4811075.html>. [Accessed 11 7 2025].
- [3] Kitti Thamrongaphichartkul, Nitisak Worrasittichai, Teeraya Prayongrak, Supachai Vongbunyong, "A Framework of IoT Platform for Autonomous Mobile Robot in Hospital Logistics Applications," *2020 15th International Joint Symposium on Artificial Intelligence and Natural Language Processing (iSAI-NLP)*, pp. pp. 1-4, 2020.
- [4] Akshet Patel, Pranav Sharma, Princy Randhawa , "MedBuddy: The Medicine Delivery Robot," *2021 9th International Conference on Reliability, Infocom Technologies and Optimization (Trends and Future Directions) (ICRITO)*, pp. pp. 1-4, 2021.
- [5] L. T. Sang, *Động học robot di động*, Ho Chi Minh: HUTECH , 2024.
- [6] T. Nguyen, "Tổng Quan ROS Cơ Bản," 4 Jan 2023. [Online]. Available: <https://robodev.blog/tong-quan-ve-series-ros-co-ban>. [Accessed 20 June 2025].
- [7] I. Synopsys, "What is LiDAR?," n.d.. [Online]. Available: <https://www.synopsys.com/glossary/what-is-lidar.html>. [Accessed 25 June 2025].
- [8] n. Refsnes Data AS, "Dijkstra's Algorithm," [Online]. Available: https://www.w3schools.com/dsa/dsa_algo_graphs_dijkstra.php. [Accessed 7 7 2025].
- [9] Jiayi Wang et al., "Illustration of the TEB algorithm: sequence of AGV poses forming a trajectory," 2021. [Online]. Available: https://www.researchgate.net/figure/Illustration-of-the-TEB-algorithm-The-TEB-is-a-sequence-of-AGV-poses-forming-a_fig2_356638054?utm_source=chatgpt.com. [Accessed 7 7 2025].
- [10] W. M. e. al., "Overview of the ROS navigation stack including the teb local planner," 1 2019. [Online]. Available: https://www.researchgate.net/figure/Overview-of-the-ROS-navigation-stack-4-including-the-teb-local-planner_fig1_326233414. [Accessed 7 7 2025].

Phụ lục. CODE HỆ THỐNG

Chương trình động học

```
#include <Arduino.h>
#include <TimerOne.h>
#include <math.h> // Để gọi fabs(), round()

// Định nghĩa chân cho từng động cơ
#define BACK_DIR_PIN 2
#define BACK_PUL_PIN 3
#define HEAD1_DIR_PIN 4
#define HEAD1_PUL_PIN 5
#define HEAD2_DIR_PIN 7
#define HEAD2_PUL_PIN 6

unsigned long lastEncTime = 0;

const double PULSES_PER_REV = 5000; // Số xung cho 1 vòng quay của
động cơ
const unsigned int TICK_INTERVAL = 20; //  $\mu$ s

// Các biến đếm cho từng động cơ (volatile vì dùng trong ISR)
volatile uint32_t toggleCountHEAD1;
volatile uint32_t toggleCountHEAD2;
volatile uint32_t toggleCountBACK;

volatile uint32_t countHEAD1 = 0;
volatile uint32_t countHEAD2 = 0;
volatile uint32_t countBACK = 0;
```



```

// Trạng thái chân PUL (HIGH/LOW)
bool head1_state = false;
bool head2_state = false;
bool back_state = false;

// Thông số tốc độ (rad/s) của các động cơ
double head1_speed_rad;
double head2_speed_rad;
double back_speed_rad;

// 1) Tạo mask để lọc các bit PUL khỏi PORTD.
// Chúng ta sẽ giữ nguyên các bit khác của PORTD (nếu đang dùng), chỉ thay
đổi các bit này.
// BACK_PUL_PIN=3 → PD3, HEAD2_PUL_PIN=5 → PD5,
HEAD1_PUL_PIN=6 → PD6.
constexpr uint8_t PUL_MASK = (1 << BACK_PUL_PIN) // PD3
| (1 << HEAD2_PUL_PIN) // PD5
| (1 << HEAD1_PUL_PIN); // PD6

// 2) Hàm helper để cập nhật đồng thời 3 chân PUL.
// head1_state, head2_state, back_state là bool (0 hoặc 1).
// Dựa vào đó dựng newBits rồi ghi thẳng lên PORTD.
inline void updateAllPULPins(bool head1_state, bool head2_state, bool
back_state)
{
    // Bước 1: Tạo giá trị mới cho 3 bit PUL trong PORTD
    uint8_t newBits = (head1_state ? (1 << HEAD1_PUL_PIN) : 0) | (head2_state
? (1 << HEAD2_PUL_PIN) : 0) | (back_state ? (1 << BACK_PUL_PIN) : 0);

    // Bước 2: Ghi lên PORTD: xóa các bit trong PUL_MASK, rồi OR với newBits
    PORTD = (PORTD & ~PUL_MASK) | newBits;

```

```

    }
    // 3) ISR được gọi mỗi TICK_INTERVAL  $\mu$ s.
    //     Mỗi khi countX >= toggleCountX, đảo trạng thái headX_state rồi gọi
    updateAllPULPins().
    void timerISR()
    {
        bool toggledHEAD1 = false;
        bool toggledHEAD2 = false;
        bool toggledBACK = false;

        // HEAD1
        if (++countHEAD1 >= toggleCountHEAD1)
        {
            head1_state = !head1_state;
            countHEAD1 = 0;
            toggledHEAD1 = true;
        }

        // HEAD2
        if (++countHEAD2 >= toggleCountHEAD2)
        {
            head2_state = !head2_state;
            countHEAD2 = 0;
            toggledHEAD2 = true;
        }

        // BACK
        if (++countBACK >= toggleCountBACK)
        {

```

```

    back_state = !back_state;
    countBACK = 0;
    toggledBACK = true;
}

// Nếu có ít nhất một motor cần cập nhật xung, ghi đồng thời cả 3 chân PUL
if (toggledHEAD1 || toggledHEAD2 || toggledBACK)
{
    updateAllPULPins(head1_state, head2_state, back_state);
}
}

// Hàm tính delay ( $\mu$ s) dựa trên tốc độ rad/s
double calculateDelay(double speed_rad)
{
    speed_rad = fabs(speed_rad); // Dùng fabs để xử lý chính xác kiểu double/float
    // Serial.println("Delay: ");
    // Serial.println((PI * 1e6) / (speed_rad * PULSES_PER_REV));
    return round((PI * 1e6) / (speed_rad * PULSES_PER_REV));
}

// Hàm kích hoạt động cơ với tốc độ đã cho
void runWithSpeed(double head1_v, double back_v, double head2_v)
{
    Timer1.detachInterrupt(); // Dừng Timer ISR để cấu hình

    // HEAD1: Cấu hình chiều quay (DIR) và tính toggleCount
    if (head1_v == 0)
    {

```

```

toggleCountHEAD1 = 0xFFFFFFFF; // Giá trị rất lớn để không bao giờ
toggle
}
else
{
// head1_v >= 0 → DIR pin = LOW
// head1_v < 0 → DIR pin = HIGH
digitalWrite(HEAD1_DIR_PIN, head1_v >= 0 ? LOW : HIGH);
toggleCountHEAD1      =      (uint32_t)(calculateDelay(head1_v)      /
TICK_INTERVAL);
}

// HEAD2: Cấu hình chiều quay (DIR) và tính toggleCount
if (head2_v == 0)
{
toggleCountHEAD2 = 0xFFFFFFFF;
}
else
{
digitalWrite(HEAD2_DIR_PIN, head2_v >= 0 ? LOW : HIGH);
toggleCountHEAD2      =      (uint32_t)(calculateDelay(head2_v)      /
TICK_INTERVAL);
}

// BACK: Cấu hình chiều quay (DIR) và tính toggleCount
if (back_v == 0)
{
toggleCountBACK = 0xFFFFFFFF;
}

```

```

else
{
    digitalWrite(BACK_DIR_PIN, back_v >= 0 ? LOW : HIGH);
    toggleCountBACK      =      (uint32_t)(calculateDelay(back_v)      /
TICK_INTERVAL);
}

```

// Reset bộ đếm xung và kéo PUL về LOW trước khi attach ISR

```
countHEAD1 = countHEAD2 = countBACK = 0;
```

```
head1_state = head2_state = back_state = false;
```

```
updateAllPULPins(head1_state, head2_state, back_state);
```

// Kích hoạt lại ngắt Timer để bắt đầu cấp xung cho các motor

```
Timer1.attachInterrupt(timerISR);
```

```
}
```

// Hàm kiểm tra xem chuỗi có phải là số hợp lệ không

```
bool isNumber(const String &s)
```

```
{
```

```
    String str = s;
```

```
    str.trim();
```

```
    if (str.length() == 0)
```

```
        return false;
```

```
    bool seenDot = false;
```

```
    for (uint16_t i = 0; i < str.length(); i++)
```

```
    {
```

```
        char c = str.charAt(i);
```

```
        if (c == '+' || c == '-')
```

```

{
    if (i != 0)
        return false; // Dấu +/- chỉ được ở đầu
    }
    else if (c == '.')
    {
        if (seenDot)
            return false; // Không cho phép 2 dấu chấm
        seenDot = true;
    }
    else if (!isDigit(c))
    {
        return false;
    }
}
return true;
}

```

// Hàm phân tích lệnh và lấy các thông số vận tốc

```
bool parseCommand(const String &cmd)
```

```

{
    String command = cmd;
    command.trim();
    if (!command.startsWith(":") || !command.endsWith(":"))
    {
        Serial.println("Lệnh không hợp lệ: thiếu ký tự ':' ở đầu hoặc cuối");
        return false;
    }
    // Loại bỏ ký tự ':' ở đầu và cuối

```

```

command = command.substring(1, command.length() - 1);

// Lấy commandType trước dấu '_'
int firstUnderscore = command.indexOf('_');
if (firstUnderscore == -1)
{
    Serial.println("Lệnh không hợp lệ: thiếu commandType");
    return false;
}
String commandType = command.substring(0, firstUnderscore);
if (commandType != "GO")
{
    Serial.print("Lệnh không được hỗ trợ: ");
    Serial.println(commandType);
    return false;
}

// Lấy phần param sau dấu '_' đầu tiên
command = command.substring(firstUnderscore + 1);
int firstSep = command.indexOf('_');
int secondSep = command.lastIndexOf('_');
if (firstSep == -1 || secondSep == -1 || firstSep == secondSep)
{
    Serial.println("Lệnh GO không hợp lệ: thiếu tham số hoặc định dạng không đúng");
    return false;
}

String head1Str = command.substring(0, firstSep);

```

```

String backStr = command.substring(firstSep + 1, secondSep);
String head2Str = command.substring(secondSep + 1);

// Kiểm tra xem cả 3 tham số có phải số không
if (!isNumber(head1Str) || !isNumber(backStr) || !isNumber(head2Str))
{
    Serial.println("Lệnh không hợp lệ: các tham số không phải là số");
    return false;
}

// Chuyển thành số thực
head1_speed_rad = head1Str.toFloat();
back_speed_rad = backStr.toFloat();
head2_speed_rad = head2Str.toFloat();

return true;
}

void setup()
{
    Serial.begin(115200);

    pinMode(HEAD1_DIR_PIN, OUTPUT);
    digitalWrite(HEAD1_DIR_PIN, LOW);
    pinMode(HEAD1_PUL_PIN, OUTPUT);
    digitalWrite(HEAD1_PUL_PIN, LOW);

    pinMode(HEAD2_DIR_PIN, OUTPUT);
    digitalWrite(HEAD2_DIR_PIN, LOW);

```



```

pinMode(HEAD2_PUL_PIN, OUTPUT);
digitalWrite(HEAD2_PUL_PIN, LOW);

pinMode(BACK_DIR_PIN, OUTPUT);
digitalWrite(BACK_DIR_PIN, LOW);
pinMode(BACK_PUL_PIN, OUTPUT);
digitalWrite(BACK_PUL_PIN, LOW);

// Khởi tạo Timer với khoảng thời gian TICK_INTERVAL  $\mu$ s
Timer1.initialize(TICK_INTERVAL);
// Đặt lại cả 3 PUL = LOW (cho chắc chắn)
updateAllPUL Pins(false, false, false);
lastEncTime = millis();
}
void loop()
{
  if (Serial.available())
  {
    String cmd = Serial.readStringUntil('\n');
    if (parseCommand(cmd))
    {
      Serial.println("CMD: " + cmd);
      runWithSpeed(head1_speed_rad, back_speed_rad, head2_speed_rad);
    }
  }
  delay(1); // Thêm một chút thời gian để tránh quá tải Serial
  // runWithSpeed(6.2831,6.2831,6.2831);
  // delay(1000);
  // runWithSpeed(0,0,0);

```

```
// delay(10000);  
}
```

Chương trình esp32 đọc Encoder và IMU

```
#include <Arduino.h>
```

```
#include <TimerOne.h>
```

```
#include <math.h> // Để gọi fabs(), round()
```

```
// Định nghĩa chân cho từng động cơ
```

```
#define BACK_DIR_PIN 2
```

```
#define BACK_PUL_PIN 3
```

```
#define HEAD1_DIR_PIN 4
```

```
#define HEAD1_PUL_PIN 5
```

```
#define HEAD2_DIR_PIN 7
```

```
#define HEAD2_PUL_PIN 6
```

```
unsigned long lastEncTime = 0;
```

```
const double PULSES_PER_REV = 5000; // Số xung cho 1 vòng quay của  
động cơ
```

```
const unsigned int TICK_INTERVAL = 20; //  $\mu$ s
```

```
// Các biến đếm cho từng động cơ (volatile vì dùng trong ISR)
```

```
volatile uint32_t toggleCountHEAD1;
```

```
volatile uint32_t toggleCountHEAD2;
```

```
volatile uint32_t toggleCountBACK;
```

```
volatile uint32_t countHEAD1 = 0;
```

```
volatile uint32_t countHEAD2 = 0;
```

```
volatile uint32_t countBACK = 0;
```

```

// Trạng thái chân PUL (HIGH/LOW)
bool head1_state = false;
bool head2_state = false;
bool back_state = false;

// Thông số tốc độ (rad/s) của các động cơ
double head1_speed_rad;
double head2_speed_rad;
double back_speed_rad;

// DFRobot_BMX160 bmx160;

//

```

```

// 1) Tạo mask để lọc các bit PUL khỏi PORTD.
// Chúng ta sẽ giữ nguyên các bit khác của PORTD (nếu đang dùng), chỉ thay
đổi các bit này.
// BACK_PUL_PIN=3 → PD3, HEAD2_PUL_PIN=5 → PD5,
HEAD1_PUL_PIN=6 → PD6.
//

```

```

constexpr uint8_t PUL_MASK = (1 << BACK_PUL_PIN) // PD3
| (1 << HEAD2_PUL_PIN) // PD5
| (1 << HEAD1_PUL_PIN); // PD6

```

```

//

```

```

// 2) Hàm helper để cập nhật đồng thời 3 chân PUL.
// head1_state, head2_state, back_state là bool (0 hoặc 1).
// Dựa vào đó dựng newBits rồi ghi thẳng lên PORTD.
//

```

```

inline void updateAllPULPins(bool head1_state, bool head2_state, bool
back_state)
{
    // Bước 1: Tạo giá trị mới cho 3 bit PUL trong PORTD
    uint8_t newBits = (head1_state ? (1 << HEAD1_PUL_PIN) : 0) | (head2_state
? (1 << HEAD2_PUL_PIN) : 0) | (back_state ? (1 << BACK_PUL_PIN) : 0);

    // Bước 2: Ghi lên PORTD: xóa các bit trong PUL_MASK, rồi OR với newBits
    PORTD = (PORTD & ~PUL_MASK) | newBits;
}

//

```

```

// 3) ISR được gọi mỗi TICK_INTERVAL  $\mu$ s.
// Mỗi khi countX >= toggleCountX, đảo trạng thái headX_state rồi gọi
updateAllPULPins().
//

```

```

void timerISR()
{
    bool toggledHEAD1 = false;
    bool toggledHEAD2 = false;
    bool toggledBACK = false;

    // HEAD1
    if (++countHEAD1 >= toggleCountHEAD1)
    {
        head1_state = !head1_state;
        countHEAD1 = 0;
        toggledHEAD1 = true;
    }

    // HEAD2
    if (++countHEAD2 >= toggleCountHEAD2)
    {
        head2_state = !head2_state;
        countHEAD2 = 0;
        toggledHEAD2 = true;
    }

    // BACK
    if (++countBACK >= toggleCountBACK)
    {
        back_state = !back_state;
        countBACK = 0;
        toggledBACK = true;
    }
}

```

```

// Nếu có ít nhất một motor cần cập nhật xung, ghi đồng thời cả 3 chân PUL
if (toggledHEAD1 || toggledHEAD2 || toggledBACK)
{
    updateAllPULPins(head1_state, head2_state, back_state);
}
}

// Hàm tính delay ( $\mu$ s) dựa trên tốc độ rad/s
double calculateDelay(double speed_rad)
{
    speed_rad = fabs(speed_rad); // Dùng fabs để xử lý chính xác kiểu double/float
    // Serial.println("Delay: ");
    // Serial.println((PI * 1e6) / (speed_rad * PULSES_PER_REV));
    return round((PI * 1e6) / (speed_rad * PULSES_PER_REV));
}

// Hàm kích hoạt động cơ với tốc độ đã cho
void runWithSpeed(double head1_v, double back_v, double head2_v)
{
    Timer1.detachInterrupt(); // Dừng Timer ISR để cấu hình

    // HEAD1: Cấu hình chiều quay (DIR) và tính toggleCount
    if (head1_v == 0)
    {
        toggleCountHEAD1 = 0xFFFFFFFF; // Giá trị rất lớn để không bao giờ
toggle
    }
    else

```

```

{
    // head1_v >= 0 → DIR pin = LOW
    // head1_v < 0 → DIR pin = HIGH
    digitalWrite(HEAD1_DIR_PIN, head1_v >= 0 ? LOW : HIGH);
    toggleCountHEAD1      =      (uint32_t)(calculateDelay(head1_v)      /
TICK_INTERVAL);
}

// HEAD2: Cấu hình chiều quay (DIR) và tính toggleCount
if (head2_v == 0)
{
    toggleCountHEAD2 = 0xFFFFFFFF;
}
else
{
    digitalWrite(HEAD2_DIR_PIN, head2_v >= 0 ? LOW : HIGH);
    toggleCountHEAD2      =      (uint32_t)(calculateDelay(head2_v)      /
TICK_INTERVAL);
}

// BACK: Cấu hình chiều quay (DIR) và tính toggleCount
if (back_v == 0)
{
    toggleCountBACK = 0xFFFFFFFF;
}
else
{
    digitalWrite(BACK_DIR_PIN, back_v >= 0 ? LOW : HIGH);

```

```

        toggleCountBACK      =      (uint32_t)(calculateDelay(back_v)      /
TICK_INTERVAL);
    }

    // Reset bộ đếm xung và kéo PUL về LOW trước khi attach ISR
    countHEAD1 = countHEAD2 = countBACK = 0;
    head1_state = head2_state = back_state = false;
    updateAllPULPins(head1_state, head2_state, back_state);

    // Kích hoạt lại ngắt Timer để bắt đầu cấp xung cho các motor
    Timer1.attachInterrupt(timerISR);
}

// Hàm kiểm tra xem chuỗi có phải là số hợp lệ không
bool isNumber(const String &s)
{
    String str = s;
    str.trim();
    if (str.length() == 0)
        return false;

    bool seenDot = false;
    for (uint16_t i = 0; i < str.length(); i++)
    {
        char c = str.charAt(i);
        if (c == '+' || c == '-')
        {
            if (i != 0)
                return false; // Dấu +/- chỉ được ở đầu
        }
    }
}

```



```

    }
    else if (c == '.')
    {
        if (seenDot)
            return false; // Không cho phép 2 dấu chấm
        seenDot = true;
    }
    else if (!isDigit(c))
    {
        return false;
    }
}
return true;
}

```

// Hàm phân tích lệnh và lấy các thông số vận tốc

```
bool parseCommand(const String &cmd)
```

```

{
    String command = cmd;
    command.trim();
    if (!command.startsWith(":") || !command.endsWith(":"))
    {
        Serial.println("Lệnh không hợp lệ: thiếu ký tự ':' ở đầu hoặc cuối");
        return false;
    }
}

```

// Loại bỏ ký tự ':' ở đầu và cuối

```
command = command.substring(1, command.length() - 1);
```

// Lấy commandType trước dấu '_'

```

int firstUnderscore = command.indexOf('_');
if (firstUnderscore == -1)
{
    Serial.println("Lệnh không hợp lệ: thiếu commandType");
    return false;
}
String commandType = command.substring(0, firstUnderscore);
if (commandType != "GO")
{
    Serial.print("Lệnh không được hỗ trợ: ");
    Serial.println(commandType);
    return false;
}

// Lấy phần param sau dấu '_' đầu tiên
command = command.substring(firstUnderscore + 1);
int firstSep = command.indexOf('_');
int secondSep = command.lastIndexOf('_');
if (firstSep == -1 || secondSep == -1 || firstSep == secondSep)
{
    Serial.println("Lệnh GO không hợp lệ: thiếu tham số hoặc định dạng không
đúng");
    return false;
}

String head1Str = command.substring(0, firstSep);
String backStr = command.substring(firstSep + 1, secondSep);
String head2Str = command.substring(secondSep + 1);

```

```

// Kiểm tra xem cả 3 tham số có phải số không
if (!isNumber(head1Str) || !isNumber(backStr) || !isNumber(head2Str))
{
    Serial.println("Lệnh không hợp lệ: các tham số không phải là số");
    return false;
}

// Chuyển thành số thực
head1_speed_rad = head1Str.toFloat();
back_speed_rad = backStr.toFloat();
head2_speed_rad = head2Str.toFloat();

return true;
}

void setup()
{
    Serial.begin(115200);

    pinMode(HEAD1_DIR_PIN, OUTPUT);
    digitalWrite(HEAD1_DIR_PIN, LOW);
    pinMode(HEAD1_PUL_PIN, OUTPUT);
    digitalWrite(HEAD1_PUL_PIN, LOW);

    pinMode(HEAD2_DIR_PIN, OUTPUT);
    digitalWrite(HEAD2_DIR_PIN, LOW);
    pinMode(HEAD2_PUL_PIN, OUTPUT);
    digitalWrite(HEAD2_PUL_PIN, LOW);
}

```

```

pinMode(BACK_DIR_PIN, OUTPUT);
digitalWrite(BACK_DIR_PIN, LOW);
pinMode(BACK_PUL_PIN, OUTPUT);
digitalWrite(BACK_PUL_PIN, LOW);

// Khởi tạo Timer với khoảng thời gian TICK_INTERVAL  $\mu$ s
Timer1.initialize(TICK_INTERVAL);

// Đặt lại cả 3 PUL = LOW (cho chắc chắn)
updateAllPUL Pins(false, false, false);

lastEncTime = millis();
}

void loop()
{
  if (Serial.available())
  {
    String cmd = Serial.readStringUntil('\n');
    if (parseCommand(cmd))
    {
      Serial.println("CMD: " + cmd);
      runWithSpeed(head1_speed_rad, back_speed_rad, head2_speed_rad);
    }
  }
  delay(1); // Thêm một chút thời gian để tránh quá tải Serial
}

```

Chương trình tính toán tạo Odometry và IMU

```
#!/usr/bin/env python
# -*- coding: utf-8 -*-
"""
ROS Melodic Python node to read IMU and encoder data from UART,
parse custom DATA_... format, and publish to /imu/data_raw,
/imu/mag, and /odom_raw topics at configurable rate.
Features:
- Calibrate IMU: gyro (deg/s -> rad/s), mag ( $\mu$ T -> T), accel unchanged (m/s2)
- Compute odometry from encoder counts via forward kinematics
"""
import rospy
import serial
import numpy as np
from math import pi
from sensor_msgs.msg import Imu, MagneticField
from nav_msgs.msg import Odometry
from geometry_msgs.msg import Quaternion, TransformStamped
import tf
import tf2_ros

# Global constants (SI units)
WHEEL_RADIUS = 0.041 # wheel radius in meters (0.041 m)
L = 0.180 # distance from wheel center to robot center in meters
SIG_DEG = [60.0, 180.0, 300.0] # wheel angles in degrees
SIG = [angle * pi/180.0 for angle in SIG_DEG] # convert to radians

def build_jacobian():
    # Forward kinematics matrix J (3x3)
```

```

sin1, sin2, sin3 = np.sin(SIG)
cos1, cos2, cos3 = np.cos(SIG)
J = (WHEEL_RADIUS / 3.0) * np.array([
    [-2*sin1, -2*sin2, -2*sin3],
    [-2*cos1, -2*cos2, -2*cos3],
    [-1.0/L, -1.0/L, -1.0/L]
])
return J

```

```

class ImuOdomNode:
    def __init__(self):
        rospy.init_node('odom_imu_uart_node')
        # Params
        self.port = rospy.get_param('~port', '/dev/ttyTHS1')
        self.baudrate = rospy.get_param('~baudrate', 115200)
        self.timeout = rospy.get_param('~timeout', 1)
        self.rate_hz = rospy.get_param('~rate', 30)
        self.ticks_per_rev = rospy.get_param('~ticks_per_rev', 998)

        # Publishers
        self.imu_pub = rospy.Publisher('/imu/data_raw', Imu, queue_size=10)
        self.mag_pub = rospy.Publisher('/imu/mag', MagneticField,
queue_size=10)
        self.odom_pub = rospy.Publisher('/odom_raw', Odometry,
queue_size=10)

        # Serial
        try:

```

```

        self.ser = serial.Serial(self.port, self.baudrate, timeout=self.timeout)
        rospy.loginfo("Opened serial port: %s and baudrate: %s", self.port,
self.baudrate)
        self.ser.write(b'r')
        line = self.ser.readline().decode('utf-8', errors='ignore')
        rospy.loginfo(line)
    except serial.SerialException as e:
        rospy.logerr("Failed to open %s, error: %s",self.port, e)
        rospy.signal_shutdown("Serial port error")
        return

# Precompute matrix
self.J = build_jacobian()

# State for odometry
self.x = 0.0
self.y = 0.0
self.theta = 0.0

# Initialize in __init__:
self.orientation = Quaternion(0, 0, 0, 1) # Quaternion identity
self.tf_broadcaster = tf2_ros.TransformBroadcaster()

now = rospy.Time.now().to_sec()
self.prev_time = now
self.prev_ticks = np.zeros(3, dtype=float)

```

```

def parse_line(self, line):

    # Expect: DATA_gx_gy_gz_ax_ay_az_mx_my_mz_c1_c2_c3
    if not line.startswith('DATA_'):
        return None
    parts = line.strip().split('_')[1:]
    if len(parts) != 12:
        return None
    vals = []
    try:
        # first 9 floats, last 3 ints
        vals = [float(p) for p in parts[:9]]
        counts = [int(p) for p in parts[9:]]
        return vals + counts
    except ValueError:
        return None

def spin(self):
    rate = rospy.Rate(self.rate_hz)
    dt = 1.0 / self.rate_hz
    DEG2RAD = pi/180.0

    while not rospy.is_shutdown():
        self.ser.write(b'o')
        line = self.ser.readline().decode('utf-8', errors='ignore')
        # rospy.loginfo(line)
        data = self.parse_line(line)
        if data is None:
            rate.sleep()

```



```

        continue

    if rospy.is_shutdown():
        break

    # unpack
    gx, gy, gz, ax, ay, az, mx, my, mz, chead1, cback, chead2 = data
    now = rospy.Time.now()
    curr_time = now.to_sec()

    # --- Publish IMU ---
    imu = Imu()
    imu.header.stamp = now
    imu.header.frame_id = 'imu_link'
    imu.orientation = self.orientation
    # gyro: deg/s -> rad/s
    imu.angular_velocity.x = gx * DEG2RAD
    imu.angular_velocity.y = gy * DEG2RAD
    imu.angular_velocity.z = gz * DEG2RAD
    # accel: m/s^2 unchanged
    imu.linear_acceleration.x = ax + 0.5
    imu.linear_acceleration.y = ay - 0.5
    imu.linear_acceleration.z = az - 0.3

    self.imu_pub.publish(imu)

    # --- Publish Magnetometer ---
    mag = MagneticField()
    mag.header.stamp = now

```

```

mag.header.frame_id = 'imu_link'
#  $\mu$ T -> T
mag.magnetic_field.x = mx * 1e-6
mag.magnetic_field.y = my * 1e-6
mag.magnetic_field.z = mz * 1e-6
self.mag_pub.publish(mag)

# --- Compute Odometry ---
dt = curr_time - self.prev_time
if dt <= 0:
    dt = 1.0 / self.rate_hz
self.prev_time = curr_time

ticks_now = np.array([thead1, cback, thead2], dtype=float)
delta_ticks = ticks_now - self.prev_ticks
self.prev_ticks = ticks_now

# omega_i (rad/s)
omega = (2.0 * pi * delta_ticks) / (self.ticks_per_rev * dt)

vel = self.J.dot(omega)
vx, vy, omega_z = vel.tolist()

# Integrate pose
dx = (vx * np.cos(self.theta) - vy * np.sin(self.theta)) * dt
dy = (vx * np.sin(self.theta) + vy * np.cos(self.theta)) * dt
dtheta = omega_z * dt
self.x += dx
self.y += dy

```

```

        self.theta += dtheta
        # publish odometry
        odom = Odometry()
        odom.header.stamp = now
        odom.header.frame_id = 'odom'
        odom.child_frame_id = 'base_link'
        # pose
        odom.pose.pose.position.x = self.x
        odom.pose.pose.position.y = self.y
        odom.pose.pose.position.z = 0.0
        quat = tf.transformations.quaternion_from_euler(0, 0, self.theta)
        odom.pose.pose.orientation = Quaternion(*quat)
        # twist
        odom.twist.twist.linear.x = vx
        odom.twist.twist.linear.y = vy
        odom.twist.twist.angular.z = omega_z
        self.odom_pub.publish(odom)

    rate.sleep()

if __name__ == '__main__':
    try:
        node = ImuOdomNode()
        if not rospy.is_shutdown():
            node.spin()
    except rospy.ROSInterruptException:
        pass

```

Chương trình thuật toán tìm và bám đường

```
base_local_planner: "teb_local_planner/TebLocalPlannerROS"
```

base_global_planner: "navfn/NavfnROS"

TebLocalPlannerROS:

odom_topic: odom

Trajectory

teb_autosize: True

dt_ref: 0.2

dt_hysteresis: 0.1

max_samples: 500

allow_init_with_backwards_motion: False

max_global_plan_lookahead_dist: 3.0

global_plan_overwrite_orientation: False

global_plan_viapoint_sep: -1

global_plan_prune_distance: 1

exact_arc_length: False

feasibility_check_no_poses: 5

publish_feedback: False

Robot

holonomic_robot: true

max_vel_x: 0.2

max_vel_x_backwards: 0.2

max_vel_y: 0.2

max_vel_theta: 0.35

acc_lim_x: 0.15

acc_lim_y: 0.15

```

acc_lim_theta: 0.15
min_turning_radius: 0.0 # diff-drive robot (can turn on place!)

footprint_model:
  type: "circular"
  radius: 0.2 # ? Nên d?t:  $0.41 - 0.25 = \sim 0.16$  ? ho?c đúng kích thu?c robot

# GoalTolerance

xy_goal_tolerance: 0.1
yaw_goal_tolerance: 0.1
free_goal_vel: False
complete_global_plan: True

# Obstacles

min_obstacle_dist: 0.2 # This value must also include our robot radius, since
footprint_model is set to "point".
min_samples: 4
inflation_dist: 0.15
include_costmap_obstacles: True
costmap_obstacles_behind_robot_dist: 1.5
obstacle_poses_affected: 15

dynamic_obstacle_inflation_dist: 0.4
include_dynamic_obstacles: True

costmap_converter_plugin: ""
costmap_converter_spin_thread: True

```

costmap_converter_rate: 5

Optimization

no_inner_iterations: 5

no_outer_iterations: 4

optimization_activate: True

optimization_verbose: False

penalty_epsilon: 0.1

obstacle_cost_exponent: 4

weight_max_vel_x: 2

weight_max_vel_theta: 1

weight_acc_lim_x: 1

weight_acc_lim_theta: 1

weight_kinematics_nh: 1000

weight_kinematics_turning_radius: 1

weight_optimaltime: 3 # must be > 0

weight_shortest_path: 1

weight_obstacle: 100

weight_inflation: 0.2

weight_dynamic_obstacle: 10

weight_dynamic_obstacle_inflation: 0.2

weight_viapoint: 3

weight_adapt_factor: 2

Homotopy Class Planner

enable_homotopy_class_planning: True

enable_multithreading: True

max_number_classes: 4
selection_cost_hysteresis: 1.0
selection_prefer_initial_plan: 0.9
selection_obst_cost_scale: 100.0
selection_alternative_time_cost: False

roadmap_graph_no_samples: 15
roadmap_graph_area_width: 5
roadmap_graph_area_length_scale: 1.0
h_signature_prescaler: 0.5
h_signature_threshold: 0.1
obstacle_heading_threshold: 0.45
switching_blocking_period: 0.0
viapoints_all_candidates: True
delete_detours_backwards: True
max_ratio_detours_duration_best_duration: 3.0
visualize_hc_graph: False
visualize_with_time_as_z_axis_scale: False

Recovery

shrink_horizon_backup: True
shrink_horizon_min_duration: 10
oscillation_recovery: True
oscillation_v_eps: 0.1
oscillation_omega_eps: 0.1
oscillation_recovery_min_duration: 10
oscillation_filter_duration: 10